



Computer Architecture & Organisation

IFT222

Covenant Centre for Open
and Distance eLearning

ccodel.covenantuniversity.edu.ng

Table of Contents

Module One: Fundamentals Of Computer Architecture And Organization	1
Unit1 — Computer Architecture Versus Computer Organization	2
Unit 2 — Data Representation In Digital Computers	15
Unit 3 — Floating Pointing Number & Other Representations	32
Unit 4 — Instruction Set Architecture (ISA)	44
Module Two: Processor Design And Performance	60
Unit 1 — Processor Components	61
Unit 2 — CPU Performance Metrics	74
Unit 3 — Improving System Performance Through Pipelining	92
Module Three: Computer Memory Organization	111
Unit 1 — Computer Memory Overview	112
Unit 1 — Semiconductor Memory	122
Unit 3 — Cache Memory Design And Organization	129
Module Four: Basic And Advanced Processor Architectures	148
Unit 1 — RISC Versus CISC Architectures	149
Unit 2 — Parallel Architectures And Interconnection Networks	160
Answers To Self-Assessment Questions	170
Module One	171
Module Two	173
Module Three	174
Module Four	177
GLOSSARY	178

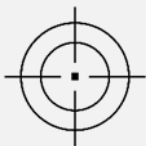
Module One

Fundamentals of Computer Architecture and Organization

Unit 1 - Computer Architecture Versus Computer Organisation

INTRODUCTION

Computers have had a huge impact on the manner in which society operates. Increased efficiency, through automation and organisation, only scratches the surface of what computing technology has afforded humanity. In fact, the technology has progressed so much that people not utilising computing technology in their businesses are at a disadvantage, as the computer today has an almost endless array of applications and functions. This unit introduces the basic concepts of a computer, generally in terms of what a computer is, parts of a computer and an overview of computer architecture and organisation. It also provides some definitions on the key terminologies of the course as well as a foundation of what to expect in the course.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Define computer architecture and computer organisation.
- Differentiate clearly between both concepts.
- Explain the relationship between architecture and organisation.
- Identify components of each concept.

1.1.1 What is a Computer?

Many users of the computer know what the computer is used for as it applies to their field of endeavour. But not everyone is able to give a proper definition of a computer. The main subject of discussion in this course is the computer. It is therefore important that we begin by providing a clear definition of this important tool.

Definition 1:

A computer is a programmable device that stores, retrieves, and processes data.

Definition 2:

A computer is a machine or device that performs processes, calculations and operations based on instructions provided by a software or hardware program.

Definition 3:

A computer may be defined as a sophisticated electronic calculating machine that accepts data, processes the data according to a list of internally stored instructions and produces the resulting output information.

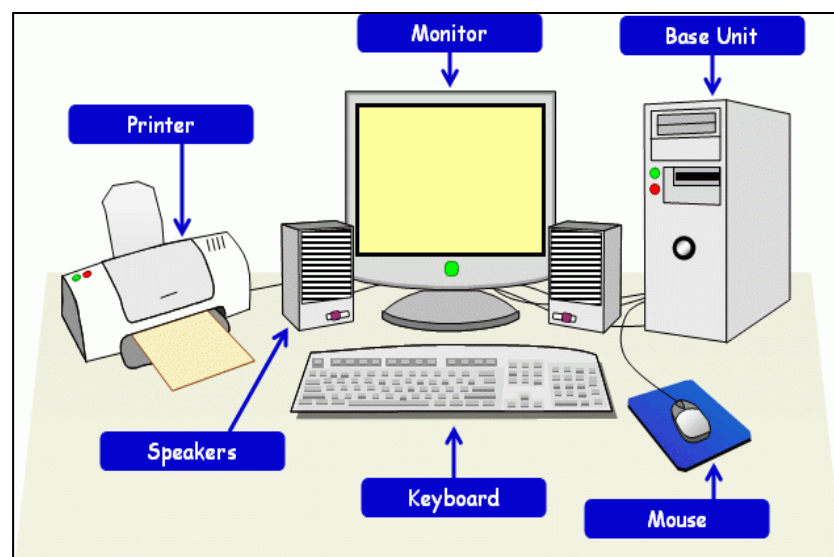


Figure 1: Parts of a Computer System

1.1.2 Components of a Computer System

Figure 1 shows the various parts of a computer from an external point of view. For the most part, all personal computers consist of the same basic components: a Central Processing Unit (CPU), memory, circuit board, storage, and input/output devices. All computer applications make use of these components in different ways and combinations. It also turns out that almost every digital device uses the same set of

components, so examining the personal computer will give us insight into the structure of a variety of digital devices. Thinking about computers in terms of their basic components and the functionality associated with each component reveals the primary functions of the computer in general. Remember that the computer is simply an electronic device that manipulates data. There are four primary functions of a computer: input, output, processing, and storage. These functions describe what is meant by the term “manipulate” when used to describe what a computer is.

1.1.3 Computer System Design

Computer systems are designed using two closely related but distinct concepts, namely:

- Computer Architecture
- Computer Organisation

These concepts define how a computer system functions and operates internally.

1.1.4 Computer Architecture

Computer architecture refers simply to the attributes of a computer visible to a programmer. That is, it refers to those attributes that have a direct impact on the logical execution of a computer program. For example, it is an architectural design issue whether a computer will have a multiply instruction. But it is an organisational issue whether that instruction will be implemented by a special multiply unit or by a mechanism that makes repeated use of the add unit of the system.

Thus, computer architecture refers to the functional design and logical structure of a computer system. It describes the following aspects of a computer:

- Instruction set
- Data types
- Addressing modes
- Register organisation

Computer Architecture also answers the question: “What does the computer do?”. It therefore serves as a blueprint for system design. Examples of architectural attributes include the instruction set, the number of bits used to represent various data types (e.g., numbers, characters), I/O mechanisms, and memory addressing techniques.

1.1.5 Computer Organization

Computer organisation refers to the operational units and their interconnections that realise the architectural specifications. Organisational attributes include those hardware details transparent to the programmer, such as control signals, interfaces between the computer and peripherals, and the memory technology used.

Computer organisation refers to the physical implementation of the system, including:

- Hardware components
- Interconnections
- Control signals
- Memory structure

Thus, Computer Organisation answers the question: "How does the computer do it? It ensures the architecture is translated into working hardware.

Computer architecture can be likened to a building design, while computer organisation may be likened to the actual construction or implementation of the building.

1.1.6 Components of Computer Architecture and Computer Organisation

A computer system is not just a collection of hardware parts; it is a carefully designed system governed by two fundamental concepts, namely Computer Architecture and Computer Organisation, as discussed in the previous section

These two concepts work together to define what a computer system does and how efficiently it performs tasks. Understanding the components of these two vital concepts is crucial for designing efficient hardware systems, writing optimised software and improving system performance.

1.1.7 Components of Computer Architecture

Computer architecture defines the logical structure and functional behavior of a computer system. These components are typically visible to programmers and determine how software interacts with hardware.

Instruction Set Architecture (ISA)

The Instruction Set Architecture (ISA) is the core of computer architecture. It acts as the interface between hardware and software and defines the following:

The set of instructions a processor can execute

- Instruction formats (e.g., ADD, LOAD, STORE)
- Data types (integer, floating point)
- Registers available
- Addressing modes

Real-Life Applications

- **Software Compatibility**

Applications compiled for a specific ISA (e.g., x86 or ARM) will only run on processors supporting that ISA. Example: Mobile apps are built for ARM-based processors in smartphones.

- **Operating Systems**

An OS is designed for a specific ISA.

Example: Windows OS is optimized for x86 architecture systems.

- **Game Development**

Game engines optimize performance based on processor instruction sets.

Different organizations (hardware implementations) can support the same ISA but vary in performance.

Key Insight

A single architecture can have multiple organizations.

Example: Two processors may use the same ISA but differ in:

- Cache size
- Pipeline depth
- Clock speed

Data Types and Representation

Defines how data is stored and interpreted:

- Integer formats (signed/unsigned)
- Floating-point representation
- Character encoding (ASCII, Unicode)

Real-Life Applications

- **Financial Systems:** Accurate floating-point representation is critical for banking software.
- **Multimedia Systems:** Video and image processing rely on data formats (e.g., RGB encoding).
- **Scientific Computing:** Precision in floating-point calculations is essential for simulations.

Addressing Modes

Addressing modes determine how operands are accessed during instruction execution.

Examples:

- Immediate addressing
- Direct addressing
- Indirect addressing
- Register addressing

Real-Life Applications

- **Efficient Program Execution:** Compilers choose addressing modes to optimise speed.
- **Embedded Systems:** Efficient memory access is crucial in devices with limited memory.
- **Gaming Systems:** Fast memory access improves frame rates and responsiveness.

Register Set

Registers are small, fast storage locations inside the CPU.

Types include:

- General-purpose registers
- Special-purpose registers (Program Counter, Stack Pointer)

Real-Life Applications

- **High-Speed Computation:** Programs execute faster when data is stored in registers instead of memory.

- Real-Time Systems: Critical systems (e.g., air traffic control) rely on fast register operations.
- Mobile Devices: Efficient register usage improves battery life and performance.

System Design (e.g., Von Neumann vs Harvard)

Defines how memory and processing units are structured.

Von Neumann Architecture: Shared memory for data and instructions

This processor configuration consists of a control unit, an arithmetic logic unit, a register set, and a memory region containing program instructions and data. The key feature distinguishing the von Neumann architecture from the Harvard architecture is the use of a single area of memory for program instructions and the data acted upon by those Instructions. It is conceptually straightforward for programmers, and relatively easier for circuit designers, to locate all of the code and data a program requires in a single memory region. Figure 1 shows the elements of the Von Neumann architecture.

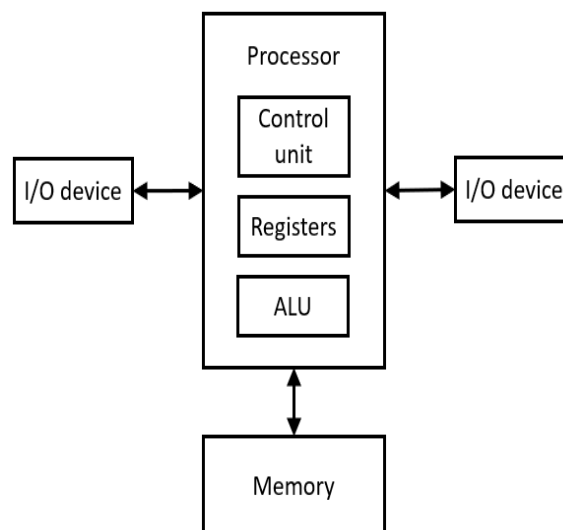


Figure 2: Von Neumann Architecture (source: Ledin, 2020)

Harvard Architecture: Separate memory for data and instructions

A strict Harvard architecture uses one address space and memory bus for program instructions and a separate address space and memory bus for data. This configuration has the immediate benefit of enabling simultaneous access to instructions and data, thereby implementing a form of parallelism. Of course, this enhancement comes at the expense of essentially duplicating the number of address lines, data

lines, and control signals that must be implemented by the processor to access both of the memory regions. The following diagram shows the layout of a processor implementing the Harvard architecture. Figure 2 shows the diagram of the Harvard architecture.

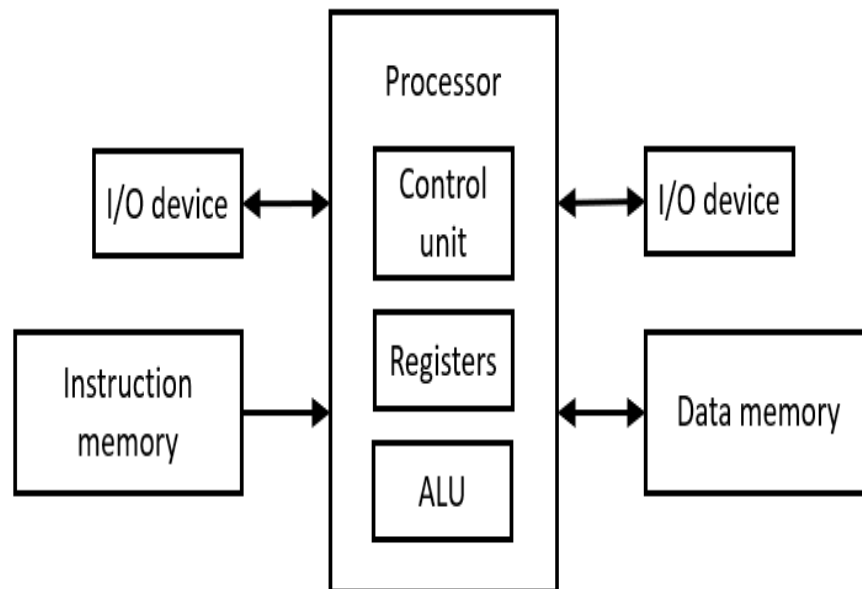


Figure 3: Harvard architecture (source: Ledin, 2020)

The Harvard architecture potentially provides a higher performance level by parallelising accesses to instructions and data. This architecture also removes the entire class of security issues associated with maliciously executing data as program instructions, as long as the instruction memory cannot be modified by program instructions. This assumes that the program memory is loaded with instructions in a trustworthy manner.

Real-Life Applications

- Von Neumann Systems: Used in general-purpose computers (laptops, desktops).
- Harvard Systems: Used in microcontrollers and embedded systems for faster access.

Example: Microcontrollers in washing machines or cars use the Harvard architecture.

1.1.8 Components of Computer Organisation

Computer organisation focuses on the physical implementation of architectural concepts.

Control Unit

The control unit directs the operations of the processor by:

- Fetching instructions
- Decoding instructions
- Generating control signals

Real-Life Applications

CPU Operation: Every application (e.g., web browsing) relies on the control unit to execute instructions.

Automation Systems: Industrial machines depend on control signals for operation.

Robotics: Control units coordinate sensor input and motor actions.

Arithmetic Logic Unit (ALU)

The ALU performs:

- Arithmetic operations (addition, subtraction)
- Logical operations (AND, OR, NOT)

Real-Life Applications

- **Calculators:** Perform arithmetic operations via ALU.
- **Gaming Systems:** Physics calculations and rendering rely heavily on ALU operations.
- **AI Systems:** Neural network computations involve large-scale arithmetic processing.

Memory Organization

Refers to how memory is structured and accessed:

- Cache memory
- Main memory (RAM)
- Secondary storage

Real-Life Applications

- **Smartphones:** Faster apps depend on an efficient memory hierarchy.
- **Web Browsers:** Cache memory improves page loading speed.

- **Cloud Computing:** Efficient memory organisation supports large-scale data processing.

Input/Output (I/O) Systems

Handles communication between the computer and external devices.

Includes:

- Input devices (keyboard, mouse)
- Output devices (monitor, printer)

Real-Life Applications

- **User Interaction:** Typing, clicking, and displaying output.
- **Medical Equipment:** Sensors and monitors rely on I/O systems.
- **Smart Homes:** Devices communicate with sensors and actuators.

System Buses and Interconnections

Buses are communication pathways that transfer:

- Data
- Addresses
- Control signals

Types:

- Data bus
- Address bus
- Control bus

Real-Life Applications

- **Motherboard Communication:** All components communicate via buses.
- **High-Speed Systems:** Faster buses improve overall system performance.
- **Gaming PCs:** High-speed data transfer supports graphics and processing.

Pipeline and Parallelism

Techniques to improve performance:

- **Pipelining:** overlapping instruction execution
- **Parallelism:** multiple tasks executed simultaneously

Real-Life Applications

- **Modern CPUs:** Use pipelines to execute multiple instructions efficiently.
- **Multicore Processors:** Run multiple applications simultaneously.
- **AI and Data Centers:** Parallel processing handles massive workloads.

1.1.9 Comparison of Computer Architecture and Organisation

Feature	Computer Architecture	Computer Organization
Focus	Functional behavior	Physical implementation
Question answered	What does the system do?	How does the system work?
Level	High-level	Low-level
Visibility	Visible to the programmer	Hidden from the programmer
Examples	ISA, registers	Circuits, buses, memory hardware
Design stage	First	After architecture

In-Text Questions

1. What is the primary focus of computer architecture?
2. What does computer organisation describe?
3. Why can multiple systems share the same architecture?

In-Text Answers

1. It focuses on the functional behavior and logical design of the computer system.
2. It describes the physical arrangement and implementation of hardware components.
3. Because architecture defines functionality, while organisation defines implementation, which can vary.

1.1.10 Conclusion

You have learnt in this unit what a computer is, including its functions. You have also learnt what most people do not usually bother about, which is the architecture and organisation of a computer system. This knowledge has, in no small measure, changed your perspective or view of a computer. You can now describe or discuss a computer system in more technical terms.

Summary

In this unit, we have considered a number of things that concern a computer system, including its definition, functions, architecture and organisation. The primary functions of the computer include input, output, storage, and processing. These functions are what the phrase “manipulate data” refers to when describing a computer.

Finally, computer architecture represents the view of a computer from the perspective of a programmer, and it is about what the computer can do, while organisation is about how the computer is structured in order to carry out the expected functionalities.

Self-Assessment Questions

1. Define computer architecture.
2. Define computer organisation.
3. Differentiate between architecture and organisation.
4. Give two examples of architectural components.
5. Give two examples of organisational components.
6. Why is architecture important to programmers?
7. Can two computers have the same architecture but different performance?
8. What is ISA?

Unit Activities

Students analyse two systems based on the same instruction set (e.g., Intel-based systems) but with different internal designs.

Instructions:

Two computers both use the same instruction set but differ in:

- Cache size
- Clock speed
- Memory hierarchy

Students must:

- Identify which aspects relate to architecture
- Identify which relate to the organisation
- Explain how differences affect performance

Guiding Questions:

- Why do both systems run the same programs?
- What causes performance differences?
- Which component belongs to architecture vs organisation?

Tutor-Marked Assignment

- Provide a practical explanation of computer architecture and computer organisation. Include cogent examples in your explanation.
- Discuss other features/functions performed by modern-day computers apart from the ones discussed in this unit. Make sure you supply appropriate references for your source of information.

References/Further Readings

Computer Organization and Architecture: Designing for performance by William Stallings, Tenth edition (2016). Read all sections in Chapter One

http://home.ustc.edu.cn/~leedsong/reference_books_tools/Computer%20Organization%20and%20Architecture%2010th%20-%20William%20Stallings.pdf

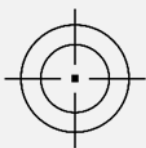
Computer Organization and Architecture: Evolutionary Concepts, Principles and Designs by Pranabananda Chakraborty, 2021

Computer Organization and Design: The Hardware/Software Interface: RISC-V Edition - Read all sections in Chapter One

Unit 2 — Data Representation in Digital Computers

INTRODUCTION

This unit introduces the concept of data representation, which describes the various ways in which computers can store and manipulate numbers, characters, and non-numeric data. Computers are built of electronic circuitry. Information is represented by currents, voltage differences, or (for permanent storage) the orientation of magnetic material. A single electronic circuit or piece of magnetic material is often regarded as being in one of two states, so it can be used to represent a binary digit. Thus, numbers are usually represented within a computer in base two (2).



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain how data is represented in digital computers
- Convert between number systems (binary, decimal, octal, hexadecimal)
- Represent signed and unsigned integers

1.2.1 Concepts of Data Representation

Data and instructions cannot be entered and processed directly into computers using human language. Any type of data (numbers, letters, special symbols, sound or pictures) must first be converted into machine-readable form, i.e. binary form. Due to this reason, it is important to understand how a computer, together with its peripheral devices, handles data in its electronic circuits, on magnetic media and in optical devices.

Electronic components, such as a microprocessor, are made up of millions of electronic circuits. The availability of high voltage (on) in these circuits is interpreted as '1' while a low voltage (off) is interpreted as '0'. This concept can be compared to switching on and off an electric circuit. When the switch is closed, the high voltage in the circuit causes the bulb to light ('1' state). On the other hand, when the switch is open, the bulb goes off ('0' state). This forms a basis for describing data representation in digital computers using the binary number system.

1.2.2 Number Systems

The number system is a way to represent or express numbers. There are various types of number systems, such as the whole numbers and the real numbers. But in the context of computers, we define other types of number systems. They are: decimal, binary, octal and hexadecimal number systems

Human beings use decimal (base 10) and duodecimal (base 12) number systems for counting and measurements. Computers use the binary (base 2) number system, as they are made from binary digital components (known as transistors) operating in two states - on and off. In computing, we also use hexadecimal (base 16) or octal (base 8) number systems as a compact form for representing binary numbers.

Decimal (Base 10) Number System

The decimal number system has ten symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9, called digits. It uses positional notation. That is, the least-significant digit (right-most digit) is of the order of 10^0 (units or ones), the second right-most digit is of the order of 10^1 (tens), the third right-most digit is of the order of 10^2 (hundreds), and so on, where ^ denotes exponent. For example,

$$735 = 700 + 30 + 5 = 7 \times 10^2 + 3 \times 10^1 + 5 \times 10^0$$

We shall denote a decimal number with an optional suffix D if ambiguity arises.

Binary (Base 2) Number System

Binary number system has two symbols: 0 and 1, called bits. It is also a positional notation, for example,

$$10110B = 10000B + 1000B + 100B + 10B + 0B = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

We shall denote a binary number with a suffix B. Some programming languages denote binary numbers with prefix 0b or 0B (e.g., 0b1001000), or prefix b with the bits quoted (e.g., b'10001111').

A binary digit is called a bit. Eight bits is called a byte (why 8-bit unit? Probably because $8=2^3$).

Hexadecimal (Base 16) Number System

The hexadecimal number system uses 16 symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F, called hex digits. It is a positional notation, for example,

$$A3EH = A00H + 30H + EH = 10 \times 16^2 + 3 \times 16^1 + 14 \times 16^0$$

We shall denote a hexadecimal number (in short, hex) with a suffix H. Some programming languages denote hex numbers with prefix 0x or 0X (e.g., 0x1A3C5F), or prefix x with hex digits quoted (e.g., x'C3A4D98B').

Each hexadecimal digit is also called a hex digit. Most programming languages accept lowercase 'a' to 'f' as well as uppercase 'A' to 'F'.

Computers use the binary system in their internal operations, as they are built from binary digital electronic components with 2 states - on and off. However, writing or reading a long sequence of binary bits is cumbersome and error-prone (try to read this binary string: 1011 0011 0100 0011 0001 1101 0001 1000B, which is the same as hexadecimal B343 1D18H). The hexadecimal system is used as a compact form or shorthand for binary bits. Each hex digit is equivalent to 4 binary bits, i.e., shorthand for 4 bits, as follows:

Hexadecimal	Binary	Decimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
A	1010	10
B	1011	11
C	1100	12
D	1101	13
E	1110	14
F	1111	15

1.2.3 Number Conversion from One Base to Another

It is possible to convert a number in one base to a number in another base. In this section, various scenarios are presented.

Conversion from decimal to any base

To convert an integer into a digit string in a base, we repeatedly divide the number by the base. The digits correspond to the remainders of these divisions, and are generated from right to left.

Conversion from Hexadecimal to Binary

Replace each hex digit by the 4 equivalent bits (as listed in the above table), for examples,

A3C5H = 1010 0011 1100 0101B

102AH = 0001 0000 0010 1010B

Conversion from Binary to Hexadecimal

Starting from the right-most bit (least-significant bit), replace each group of 4 bits by the equivalent hex digit (pad the left-most bits with zero if necessary), for example,

$$1001001010B = 0010\ 0100\ 1010B = 24AH$$

$$10001011001011B = 0010\ 0010\ 1100\ 1011B = 22CBH$$

It is important to note that a hexadecimal number provides a compact form or shorthand for representing binary bits.

Conversion from Base r to Decimal (Base 10)

Given a n-digit base r number: $d_{n-1}d_{n-2}d_{n-3}...d_2d_1d_0$ (base r), the decimal equivalent is given by:

$$d_{n-1} \times r^{n-1} + d_{n-2} \times r^{n-2} + \dots + d_1 \times r^1 + d_0 \times r^0$$

For examples,

$$A1C2H = 10 \times 16^3 + 1 \times 16^2 + 12 \times 16^1 + 2 = 41410 \text{ (base 10)}$$

$$10110B = 1 \times 2^4 + 1 \times 2^2 + 1 \times 2^1 = 22 \text{ (base 10)}$$

Conversion from Decimal (Base 10) to Another Base r

Use repeated division/remainder. For example,

To convert 261(base 10) to hexadecimal:

$$261/16 \Rightarrow \text{quotient}=16 \text{ remainder}=5$$

$$16/16 \Rightarrow \text{quotient}=1 \text{ remainder}=0$$

$$1/16 \Rightarrow \text{quotient}=0 \text{ remainder}=1 \text{ (quotient}=0 \text{ stop)}$$

Hence, $261D = 105H$ (Collect the hex digits from the remainder in reverse order)

The above procedure is actually applicable to conversion between any 2 base systems. For example,

To convert 1023(base 4) to base 3:

$$1023(\text{base } 4)/3 \Rightarrow \text{quotient}=25D \text{ remainder}=0$$

$$25D/3 \Rightarrow \text{quotient}=8D \text{ remainder}=1$$

$$8D/3 \Rightarrow \text{quotient}=2D \text{ remainder}=2$$

$2D/3 \Rightarrow \text{quotient}=0 \text{ remainder}=2 \text{ (quotient}=0 \text{ stop)}$

Hence, $1023(\text{base } 4) = 2210(\text{base } 3)$

Conversion between Two Number Systems with Fractional Part

1. Separate the integral and the fractional parts.
2. For the integral part, divide by the target radix repeatedly, and collect the remainder in reverse order.
3. For the fractional part, multiply the fractional part by the target radix repeatedly, and collect the integral part in the same order.

Example 1: Decimal to Binary

Convert $18.6875D$ to binary

Integral Part = $18D$

$18/2 \Rightarrow \text{quotient}=9 \text{ remainder}=0$

$9/2 \Rightarrow \text{quotient}=4 \text{ remainder}=1$

$4/2 \Rightarrow \text{quotient}=2 \text{ remainder}=0$

$2/2 \Rightarrow \text{quotient}=1 \text{ remainder}=0$

$1/2 \Rightarrow \text{quotient}=0 \text{ remainder}=1 \text{ (quotient}=0 \text{ stop)}$

Hence, $18D = 10010B$

Fractional Part = $.6875D$

$.6875 \times 2 = 1.375 \Rightarrow \text{whole number is } 1$

$.375 \times 2 = 0.75 \Rightarrow \text{whole number is } 0$

$.75 \times 2 = 1.5 \Rightarrow \text{whole number is } 1$

$.5 \times 2 = 1.0 \Rightarrow \text{whole number is } 1$

Hence $.6875D = .1011B$

Combine, $18.6875D = 10010.1011B$

Example 2: Decimal to Hexadecimal

Convert 18.6875D to hexadecimal

Integral Part = 18D

$18/16 \Rightarrow \text{quotient}=1 \text{ remainder}=2$

$1/16 \Rightarrow \text{quotient}=0 \text{ remainder}=1 \text{ (quotient}=0 \text{ stop)}$

Hence, 18D = 12H

Fractional Part = .6875D

$.6875 \times 16 = 11.0 \Rightarrow \text{whole number is 11D (BH)}$

Hence .6875D = .BH

Combine, 18.6875D = 12.BH

1.2.4 Signed and Unsigned Integer Representation

Integers are whole numbers or fixed-point numbers with the radix point fixed after the least-significant bit. They are in contrast to real numbers or floating-point numbers, where the position of the radix point varies. It is important to take note that integers and floating-point numbers are treated differently in computers. They have different representations and are processed differently (e.g., floating-point numbers are processed in a so-called floating-point processor).

Computers use a fixed number of bits to represent an integer. The commonly-used bit-lengths for integers are 8-bit, 16-bit, 32-bit or 64-bit. Besides bit-lengths, there are two representation schemes for integers:

- Unsigned Integers: can represent zero and positive integers.
- Signed Integers: can represent zero, positive and negative integers.

A programmer usually needs to decide on the bit length and representation scheme for the integers, depending on the application's requirements. Suppose that you need a counter for counting a small quantity from 0 up to 200, you might choose the 8-bit unsigned integer scheme, as there are no negative numbers involved

Unsigned Integer Data Representation

Unsigned integers can represent zero and positive integers, but not negative integers. The value of an unsigned integer is interpreted as "the magnitude of its underlying binary pattern".

Example 1: Suppose that $n=8$ and the binary pattern is 0100 0001B, the value of this unsigned integer is $1 \times 2^0 + 1 \times 2^6 = 65D$.

Example 2: Suppose that $n=16$ and the binary pattern is 0001 0000 0000 1000B, the value of this unsigned integer is $1 \times 2^3 + 1 \times 2^{12} = 4104D$.

An n -bit pattern can represent 2^n distinct integers. An n -bit unsigned integer can represent integers from 0 to $(2^n)-1$, as tabulated below:

n	Mini- mum	Maximum
8	0	$(2^8)-1$ (=255)
16	0	$(2^{16})-1$ (=65,535)
32	0	$(2^{32})-1$ (=4,294,967,295) (9+ digits)
64	0	$(2^{64})-1$ (=18,446,744,073,709,551,615) (19+ digits)

Signed Integer Data Representation

Signed integers can represent zero, positive integers, as well as negative integers. Three representation schemes are available for signed integers:

- Sign-Magnitude representation
- 1's Complement representation
- 2's Complement representation

In all the above three schemes, the most-significant bit (msb) is called the sign bit. The sign bit is used to represent the sign of the integer, with 0 for positive integers and 1 for negative integers. The magnitude of the integer, however, is interpreted differently in different schemes.

n-bit Sign Integers in Sign-Magnitude Representation

In sign-magnitude representation (Figure 4):

- The most-significant bit (msb) is the sign bit, with a value of 0 representing a positive integer and 1 representing a negative integer.
- The remaining $n-1$ bits represent the magnitude (absolute value) of the integer. The absolute value of the integer is interpreted as "the magnitude of the $(n-1)$ -bit binary pattern".

Example 1: Suppose that $n=8$ and the binary representation is 0 100 0001B.

Sign bit is 0 $\Rightarrow$ positive

Absolute value is 100 0001B = 65D

Hence, the integer is +65D

Example 2: Suppose that $n=8$ and the binary representation is 1 000 0001B.

Sign bit is 1 $\Rightarrow$ negative

Absolute value is 000 0001B = 1D

Hence, the integer is -1D

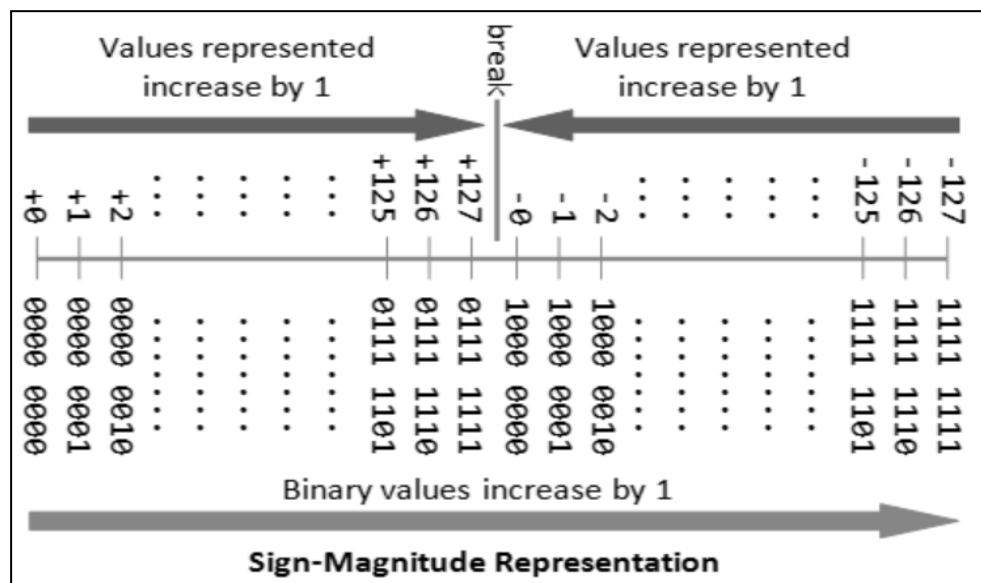


Figure 4: Sign-Magnitude Representation

The drawbacks of sign-magnitude representation are:

1. There are two representations (0000 0000B and 1000 0000B) for the number zero, which could lead to inefficiency and confusion.
2. Positive and negative integers need to be processed separately.

n-bit Sign Integers in 1's Complement Representation

In 1's complement representation:

Again, the most significant bit (msb) is the sign bit, with value of 0 representing positive integers and 1 representing negative integers. The remaining $n-1$ bits represent the magnitude of the integer, as follows:

- for positive integers, the absolute value of the integer is equal to "the magnitude of the (n-1)-bit binary pattern".
- for negative integers, the absolute value of the integer is equal to "the magnitude of the complement (inverse) of the (n-1)-bit binary pattern" (hence called 1's complement).

Example 1: Suppose that $n=8$ and the binary representation 0 100 0001B.

Sign bit is 0 $\Rightarrow$ positive

Absolute value is 100 0001B = 65D

Hence, the integer is +65D

Example 2: Suppose that $n=8$ and the binary representation 1 000 0001B.

Sign bit is 1 $\Rightarrow$ negative

Absolute value is the complement of 000 0001B, i.e., 111 1110B = 126D

Hence, the integer is -126D

Example 3: Suppose that $n=8$ and the binary representation 0 000 0000B.

Sign bit is 0 $\Rightarrow$ positive

Absolute value is 000 0000B = 0D

Hence, the integer is +0D

Example 4: Suppose that $n=8$ and the binary representation 1 111 1111B.

Sign bit is 1 $\Rightarrow$ negative

Absolute value is the complement of 111 1111B, i.e., 000 0000B = 0D

Hence, the integer is -0D

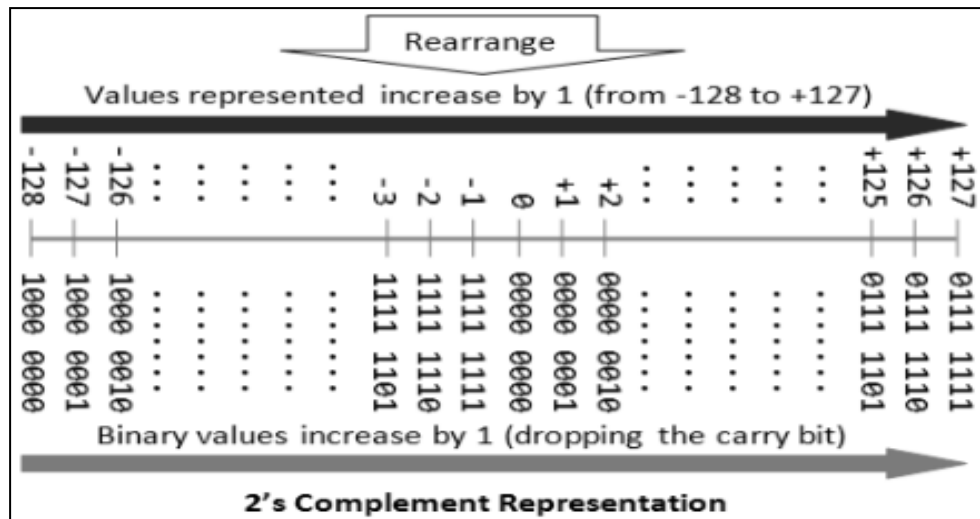


Figure 6b: Two's Complement Representation

1.2.5 How computers uses 2's complement for signed Integers

We have discussed three representations for signed integers: signed-magnitude, 1's complement and 2's complement. Computers use 2's complement in representing signed integers. This is because:

- There is only one representation for the number zero in 2's complement, instead of two representations in sign-magnitude and 1's complement.
- Positive and negative integers can be treated together in addition and subtraction. Subtraction can be carried out using the "addition logic".

Example 1: Addition of Two Positive Integers: Suppose that $n=8$, $65D + 5D = 70D$

$65D \rightarrow 0100\ 0001B$

$5D \rightarrow 0000\ 0101B$

$0100\ 0110B \rightarrow 70D$ (OK)

Example 2: Subtraction is treated as Addition of a Positive and a Negative Integers: Suppose that $n=8$, $65D - 5D = 65D + (-5D) = 60D$

$65D \rightarrow 0100\ 0001B$

$-5D \rightarrow 1111\ 1011B(+)$

$0011\ 1100B \rightarrow 60D$ (discard carry - OK)

Example 3: Addition of Two Negative Integers: Suppose that $n=8$, $-65D - 5D = (-65D) + (-5D) = -70D$

-65D → 1011 1111B

-5D → 1111 1011B(+

1011 1010B → -70D (discard carry - OK)

Figure 7 explains how the 2's complement works. By re-arranging the number line, values from -128 to +127 are represented contiguously by ignoring the carry bit.

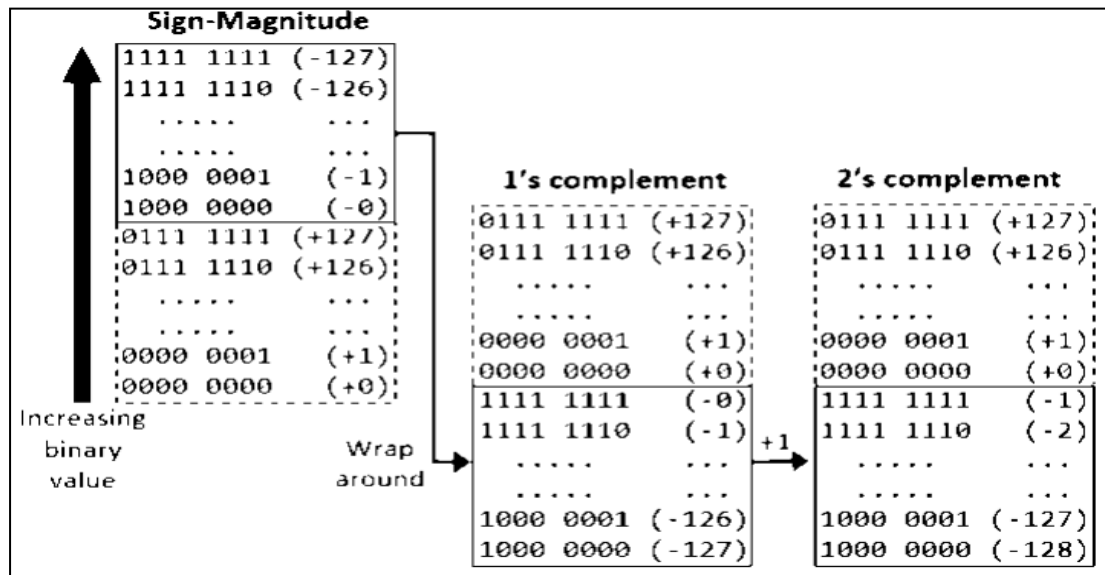


Figure 7: All three Signed Representations

1.2.6 Overflow and Underflow in 2's Complement

Sometimes a value cannot be represented in the limited number of bits allowed.

Examples:

- For unsigned, 3 bits: 8 would require at least 4 bits (1000)
- For S-M, 4 bits: 8 would require at least 5 bits (01000)

When a value cannot be represented in the number of bits allowed/specified, we say that an overflow has occurred. Overflow occurs when doing arithmetic operations.

Overflow is a situation in which the exponent is too large to be represented in the Exponent field, while Underflow is a situation in which the number is too small to be represented in the Exponent field.

Arithmetic underflow can occur when the true result of a floating-point operation is smaller in magnitude (that is, closer to zero) than the smallest value representable as a normal floating-point number in the target datatype.

Underflow can, in part, be regarded as a negative overflow of the exponent of the floating-point value. For example, if the exponent part can represent values from -127 to 127 , then a result with a value less than -127 may cause underflow.

Because of the fixed precision (i.e., fixed number of bits), an n -bit 2's complement signed integer has a certain range. For example, for $n=8$, the range of 2's complement signed integers is -128 to $+127$. During addition (and subtraction), it is important to check whether the result exceeds this range, in other words, whether overflow or underflow has occurred.

Example 1: Overflow: Suppose that $n=8$, $127D + 2D = 129D$ (overflow - beyond the range)

$127D \rightarrow 0111\ 1111B$

$2D \rightarrow 0000\ 0010B(+$

$1000\ 0001B \rightarrow -127D$ (wrong)

Example 2: Underflow: Suppose that $n=8$, $-125D - 5D = -130D$ (underflow - below the range)

$-125D \rightarrow 1000\ 0011B$

$-5D \rightarrow 1111\ 1011B(+$

$0111\ 1110B \rightarrow +126D$ (wrong)

1.2.7 Determining the Range of Values for an n -bit Using 2's Complement

An n -bit 2's complement signed integer can represent integers from $-2^{(n-1)}$ to $+2^{(n-1)}-1$, as tabulated. Take note that the scheme can represent all the integers within the range, without any gaps. In other words, there is no missing integer within the supported range.

n	minimum	Maximum
8	$-(2^7)$ ($=-128$)	$+(2^7)-1$ ($=+127$)
16	$-(2^{15})$ ($=-32,768$)	$+(2^{15})-1$ ($=+32,767$)
32	$-(2^{31})$ ($=-2,147,483,648$)	$+(2^{31})-1$ ($=+2,147,483,647$) (9+ digits)
64	$-(2^{63})$ ($=-9,223,372,036,854,775,808$)	$+(2^{63})-1$ ($=+9,223,372,036,854,775,807$) (18+ digits)

1.2.8 Conclusion

Understanding data representation is fundamental to computer science. It enables efficient data processing, storage, and communication within digital systems. Mastery of number systems and integer representation forms the foundation for more advanced topics such as computer architecture, programming, and data encoding.

Summary

Data in computers is represented using binary digits. Different number systems (binary, decimal, octal, hexadecimal) are used for various purposes. Conversion between number systems is essential for understanding computing processes. Integers can be represented as signed or unsigned. Two's complement is the most widely used method for signed numbers.

Self-Assessment Questions

1. What is a byte?
2. Convert 20_{10} to binary.
3. Convert 1110_2 to decimal.
4. What is the range of 8-bit unsigned integers?
5. Represent -3 in 4-bit two's complement.
6. What is hexadecimal used for?

Unit Activities

Activity 1

Represent -12 using:

- Sign-magnitude
- One's complement
- Two's complement (8-bit)

Activity 2

Convert: $7B_{16}$ to binary 101110_2 to octal

Tutor-Marked Assignment

1. Explain how data is represented in digital systems with examples of different data types.
2. Convert the following:
 - 255_{10} to binary
 - 110011_2 to decimal
 - $7F_{16}$ to decimal
3. Discuss the differences between binary, octal, and hexadecimal systems.
4. Explain and compare:
 - Sign-magnitude
 - One's complement
 - Two's complement
5. Perform binary addition and detect overflow: $1110 + 0111$ (4-bit system)
6. Analyse the advantages of two's complement over other signed representations.

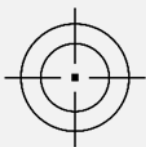
References For Further Reading

Stallings, W. – Computer Organization and Architecture
Mano, M. M. – Digital Design
Tanenbaum, A. S. – Structured Computer Organization
Patterson, D. A., & Hennessy, J. L. – Computer Organization and Design
Null, L., & Lobur, J. – The Essentials of Computer Organization and Architecture
Tocci, R. J. – Digital Systems: Principles and Applications

Unit 3: Floating Point Numbers & Other Representations

INTRODUCTION

In digital computer systems, all data, whether numerical or textual, must be represented in binary form for storage, processing, and transmission. Different types of data require different representation techniques to ensure accuracy, efficiency, and compatibility across systems.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

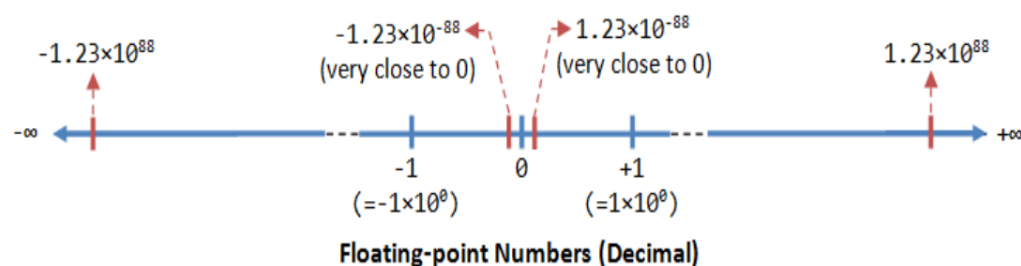
- Explain floating-point number representation and its components.
- Convert decimal numbers into normalised binary floating-point form.
- Describe and apply BCD and Excess-N codes.
- Encode and decode characters using ASCII and EBCDIC.

1.3.1 Floating Point Number Representation

The term floating point is derived from the fact that there is no fixed number of digits before and after the decimal point; that is, the decimal point can float. There are also representations in which the number of digits before and after the decimal point is set, called fixed-point representations. In general, floating-point representations are slower and less accurate than fixed-point representations, but they can handle a larger range of numbers.

A floating-point number (or real number) can represent a very large (1.23×10^{88}) or a very small (1.23×10^{-88}) value. It could also represent very large negative numbers (-1.23×10^{88}) and very small negative numbers (-1.23×10^{-88}), as well as zero, as illustrated:

Because mathematics with floating-point numbers requires a great deal of computing power, many microprocessors come with a chip, called a floating-point unit (FPU), specialised for performing floating-point arithmetic. FPUs are also called math coprocessors and numeric coprocessors.



A floating-point number is typically expressed in scientific notation, with a fraction (F) and an exponent (E) of a certain radix (r), in the form of $F \times r^E$. Decimal numbers use radix of 10 ($F \times 10^E$); while binary numbers use radix of 2 ($F \times 2^E$).

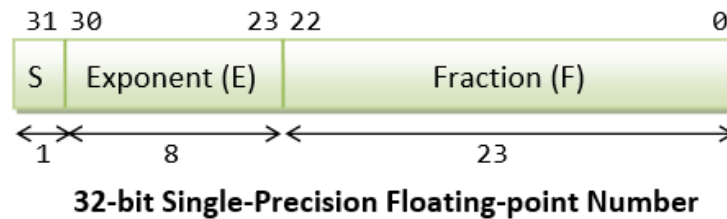
Representation of a floating-point number is not unique. For example, the number 55.66 can be represented as 5.566×10^1 , 0.5566×10^2 , 0.05566×10^3 , and so on. The fractional part can be normalised. In the normalised form, there is only a single non-zero digit before the radix point. For example, decimal number 123.4567 can be normalised as 1.234567×10^2 ; binary number 1010.1011B can be normalised as $1.0101011B \times 2^3$.

In computers, floating-point numbers are represented in scientific notation of fraction (F) and exponent (E) with a radix of 2, in the form of $F \times 2^E$. Both E and F can be positive as well as negative. Modern computers adopt the IEEE 754 standard for representing floating-point numbers. There are two representation schemes: 32-bit single-precision and 64-bit double-precision.

1.3.2 IEEE-754 32-bit Single-Precision Floating-Point Numbers

In 32-bit single-precision floating-point representation:

- The most significant bit is the sign bit (S), with 0 for positive numbers and 1 for negative numbers.
- The following 8 bits represent the exponent (E).
- The remaining 23 bits represent the fraction (F).



a) Normalised Form

Let's illustrate with an example, suppose that the 32-bit pattern is 1 1000 0001 011 0000 0000 0000 0000 0000, with:

- $S = 1$
- $E = 1000\ 0001$
- $F = 011\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000$

In the normalised form, the actual fraction is normalised with an implicit leading 1 in the form of 1.F. In this example, the actual fraction is $1.011\ 0000\ 0000\ 0000\ 0000\ 0000 = 1 + 1 \times 2^{-2} + 1 \times 2^{-3} = 1.375D$.

The sign bit represents the sign of the number, with $S=0$ for positive and $S=1$ for a negative number. In this example with $S=1$, this is a negative number, i.e., $-1.375D$.

In normalised form, the actual exponent is $E-127$ (so-called excess-127 or bias-127). This is because we need to represent both positive and negative exponents. With an 8-bit E, ranging from 0 to 255, the excess-127 scheme could provide an actual exponent of -127 to 128. In this example, $E-127=129-127=2D$.

Hence, the number represented is $-1.375 \times 2^2 = -5.5D$.

Example 1: Suppose that the IEEE-754 32-bit floating-point representation pattern is 0 10000000 110 0000 0000 0000 0000 0000.

Sign bit $S = 0 \Rightarrow$ positive number

$E = 1000\ 0000B = 128D$ (in normalized form)

Fraction is $1.11B$ (with an implicit leading 1) $= 1 + 1 \times 2^{-1} + 1 \times 2^{-2} = 1.75D$

The number is $+1.75 \times 2^{(128-127)} = +3.5D$

Example 2: Suppose that IEEE-754 32-bit floating-point representation pattern is 1 01111110 100 0000 0000 0000 0000 0000.

Sign bit $S = 1 \Rightarrow$ negative number

$E = 0111\ 1110B = 126D$ (in normalized form)

Fraction is $1.1B$ (with an implicit leading 1) $= 1 + 2^{-1} = 1.5D$

The number is $-1.5 \times 2^{(126-127)} = -0.75D$

Example 3: Suppose that IEEE-754 32-bit floating-point representation pattern is 1 01111110 000 0000 0000 0000 0000 0001.

Sign bit $S = 1 \Rightarrow$ negative number

$E = 0111\ 1110B = 126D$ (in normalized form)

Fraction is $1.000\ 0000\ 0000\ 0000\ 0000\ 0001B$ (with an implicit leading 1) $= 1 + 2^{-23}$

The number is $-(1 + 2^{-23}) \times 2^{(126-127)} = -0.5000000059604644775390625$ (may not be exact in decimal!)

Example 4 (De-Normalized Form): Suppose that IEEE-754 32-bit floating-point representation pattern is 1 00000000 000 0000 0000 0000 0000 0001.

Sign bit $S = 1 \Rightarrow$ negative number

$E = 0$ (in de-normalized form)

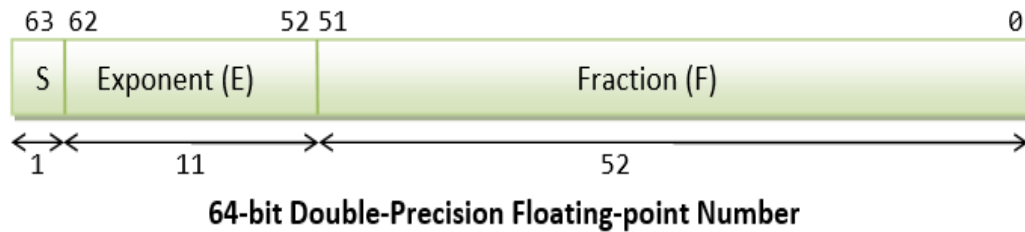
Fraction is $0.000\ 0000\ 0000\ 0000\ 0000\ 0001B$ (with an implicit leading 0) $= 1 \times 2^{-23}$

The number is $-2^{-23} \times 2^{(-126)} = -2 \times (-149) \approx -1.4 \times 10^{-45}$

1.3.3 IEEE-754 64-bit Double-Precision Floating-Point Numbers

The representation scheme for 64-bit double-precision is similar to that of 32-bit single-precision:

- The most significant bit is the sign bit (S), with 0 for positive numbers and 1 for negative numbers.
- The following 11 bits represent the exponent (E).
- The remaining 52 bits represent the fraction (F).



The value (N) is calculated as follows:

- Normalized form: For $1 \leq E \leq 2046$, $N = (-1)^S \times 1.F \times 2^{(E-1023)}$.
- Denormalized form: For $E = 0$, $N = (-1)^S \times 0.F \times 2^{(-1022)}$. These are in the denormalised form.
- For $E = 2047$, N represents special values, such as $\pm\text{INF}$ (infinity), NaN (not a number).

1.3.4 Other Types of Representation

In this unit, we discuss how the computer represents other data types like graphics, characters, alphanumeric data, images, etc.

BCD- Binary Coded Decimal

The binary coded decimal (BCD) is a type of binary code used to represent a given decimal number in an equivalent binary form. The main advantage of this scheme is that it allows easy conversion to decimal digits for printing or display and faster calculations. There are three different types:

- 8 4 2 1
- 5 4 2 1
- 4 2 2 1

The relationship between decimal numbers and the BCD is as given in the table below

Decimal digit	BCD Code											
	8	4	2	1	4	2	2	1	5	4	2	1
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	0	0	0	1	0
3	0	0	1	1	0	0	1	1	0	0	1	1
4	0	1	0	0	1	0	0	0	0	1	0	0
5	0	1	0	1	0	0	1	1	1	0	0	0
6	0	1	1	0	1	1	0	0	1	0	0	1
7	0	1	1	1	1	1	0	1	1	0	1	0
8	1	0	0	0	1	1	1	0	1	0	1	1
9	1	0	0	1	1	1	1	1	1	1	0	0

Figure 8: Binary Coded Decimal

The numbers 8,4,2,1 in 8421 BCD, 4, 2, 2, 1 in 4221 BCD and 5, 4, 2 and 1 in 5421 BCD represent weights of the relevant bits. BCD is a weighted code; each binary digit in the four-bit group representing a given decimal digit is assigned a weight. The sum of the weights of those binary digits whose value is 1 is equal to the decimal digit which they represent.

Example:

The 8421 BCD code for 9.2 is 1001.0010.

The 4221 BCD code for 9.2 is 1111.0010.

The 5421 BCD code for 9.2 is 1100.0010.

BCD code is useful for outputting to displays that are always numeric (0 to 9), such as those found in digital clocks or digital voltmeters.

Excess-n Code

Excess-n, also called XS-n, is a non-weighted code used to express decimal numbers. It is another important binary code. It is particularly significant for arithmetic operations as it overcomes the shortcomings encountered while using the 8421 BCD code to add two decimal digits whose sum exceeds 9. This code is used in some old computers.

For instance, the Excess-3 code for a given decimal number is determined by adding '3' to each decimal digit in the given number and then replacing each digit of the newly found decimal number by its four-bit binary equivalent.

Example: XS-3 code of 24 is obtained as

$$\begin{array}{r}
 2 \quad 4 \\
 +3 \quad +3 \\
 \hline
 5 \quad 7
 \end{array}$$

0101 0111

Thus, XS-3 code of 24 is 0101 0111.

It is important to note that the excess-n code is self-complementing. This means the 1's complement of an Excess- 3 number for instance, is the Excess- 3 code for the 9's complement of the corresponding decimal number.

For example, the Excess- 3 code for decimal 6 is 1001. The 1's complement of 1001 is 0110, which is the Excess-3 code for decimal 3, and 3 is the 9's complement of 6.

This property of Excess-3 code makes it useful in some arithmetic operations.

Example:

The Excess-3 code for 27 is 0101 1010.

Similarly, Excess-3 code for $(597)_{10}$ and $(14.57)_{10}$ is

$(597)_{10} = (1000\ 1100\ 1010)$

$(14.57)_{10} = (0100\ 0111.1000\ 1010)$

Alphanumeric Data Representation

Earlier computers were used only for the purpose of calculations, i.e. they were only used as a calculating device. Computers can now represent numbers and other information, such as names, addresses, item descriptions, etc. Such information is represented using letters and symbols. A computer is a digital system and can only deal with 1's and 0's. To deal with letters and symbols, they use alphanumeric codes.

Alphanumeric codes, also called character codes, are binary codes used to represent alphanumeric data. The codes write alphanumeric data, including letters of the alphabet, numbers, mathematical symbols and punctuation marks, in a form that is understandable and processable by a computer.

Using these codes, we can interface input-output devices such as keyboards, monitors, printers, etc. with a computer. Examples include:

- ASCII code
- EBCDIC code
- Unicode

ASCII Code

ASCII stands for American Standard Code for Information Interchange. It's a 7-bit character code where every single bit represents a unique character. ASCII printable characters in character code 32-127. Codes 32-127 are common for all the different variations of the ASCII table, they are called printable characters, represent letters, digits, punctuation marks, and a few miscellaneous symbols. Character 127 represents the command DEL.

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100			96	60	140		
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	\$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	I	I	104	68	150	i	i
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	J	J	105	69	151	j	j
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	K	K	106	70	152	k	k
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	L	L	107	71	153	l	l
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	M	M	108	72	154	m	m
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	N	N	109	73	155	n	n
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	O	O	110	74	156	o	o
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	P	P	111	75	157	p	p
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	Q	Q	112	76	160	q	q
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	R	R	113	77	161	r	r
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	S	S	114	78	162	s	s
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	T	T	115	79	163	t	t
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	U	U	116	80	164	u	u
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	V	V	117	81	165	v	v
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	W	W	118	82	166	w	w
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	X	X	119	83	167	x	x
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	Y	Y	120	84	170	y	y
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Z	Z	121	85	171	z	z
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	[	[	122	86	172	{	{
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	\	\	123	87	173		
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	^	^	124	88	174	~	~
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	_	_	125	89	175		
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	`	`	126	90	176		
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137			127	91	177	DEL	DEL

Source: www.LookupTables.com

Figure 9: ASCII Codes

Extended Binary Coded Decimal Interchange Code (EBCDIC)

Extended Binary Coded Decimal Interchange Code (EBCDIC) is an 8-bit character encoding used mainly on IBM mainframe and IBM midrange computer operating systems. EBCDIC descended from the code used with punched cards and the corresponding 6-bit binary-coded decimal code used with most of IBM's computer peripherals of the late 1950s and early 1960s. The figure below represents elements of ASCII and EBCDIC.

A S C I I	E B C D I C	G r a p h i c	A S C I I	E B C D I C	G r a p h i c	A S C I I	E B C D I C	G r a p h i c	A S C I I	E B C D I C	G r a p h i c	A S C I I	E B C D I C	G r a p h i c	A S C I I	E B C D I C	G r a p h i c
00 00 NUL	20 40 SP		40 7C @			60 79 .			80 00 NUL	A0 40 SP		C0 7C @			E0 79 .		
01 01 SOH	21 5A !		41 C1 A			61 81 a			81 01 SOH	A1 5A !		C1 C1 A			E1 81 a		
02 02 STX	22 7F "		42 C2 B			62 82 b			82 02 STX	A2 7F "		C2 C2 B			E2 82 b		
03 03 ETX	23 7B #		43 C3 C			63 83 c			83 03 ETX	A3 7B #		C3 C3 C			E3 83 c		
04 37 BOT	24 5B \$		44 C4 D			64 84 d			84 37 EOT	A4 5B \$		C4 C4 D			E4 84 d		
05 2D ENQ	25 6C %		45 C5 E			65 85 e			85 2D ENQ	A5 6C %		C5 C5 E			E5 85 e		
06 2E ACK	26 50 &		46 C6 F			66 86 f			86 2E ACK	A6 50 &		C6 C6 F			E6 86 f		
07 2F BEL	27 7D '		47 C7 G			67 87 g			87 2F BEL	A7 7D '		C7 C7 G			E7 87 g		
08 16 BS	28 8D (		48 C8 H			68 88 h			88 16 BS	A8 8D (		C8 C8 H			E8 88 h		
09 05 HT	29 5D)		49 C9 I			69 89 i			89 05 HT	A9 5D)		C9 C9 I			E9 89 i		
0A 25 LF	2A 5C)	*	4A D1 J			6A 91 j			AA 25 LF	AA 5C)	*	CA D1 J			EA 91 j		
0B 0B VT	2B 4E +		4B D2 K			6B 92 k			8B 0B VT	AB 4E +		CB D2 K			EB 92 k		
0C 0C FF	2C 6B .		4C D3 L			6C 93 l			8C 0C FF	AC 6B .		CC D3 L			EC 93 l		
0D 0D CR	2D 60 -		4D D4 M			6D 94 m			8D 0D CR	AD 60 -		CD D4 M			ED 94 m		
0E 0E SO	2E 4B _		4E D5 N			6E 95 n			8E 0E SO	AE 4B _		CE D5 N			EE 95 n		
0F 0F SI	2F 61 /		4F D6 O			6F 96 o			8F 0F SI	AF 61 /		CF D6 O			EF 96 o		
10 10 DLE	30 F0 0		50 D7 P			70 97 p			90 10 DLE	B0 F0 0		D0 D7 P			F0 97 p		
11 11 DC1	31 F1 1		51 D8 Q			71 98 q			91 11 DC1	B1 F1 1		D1 D8 Q			F1 98 q		
12 12 DC2	32 F2 2		52 D9 R			72 99 r			92 12 DC2	B2 F2 2		D2 D9 R			F2 99 r		
13 13 DC3	33 F3 3		53 E2 S			73 A2 s			93 13 DC3	B3 F3 3		D3 E2 S			F3 A2 s		
14 3C DC4	34 F4 4		54 E3 T			74 A3 t			94 3C DC4	B4 F4 4		D4 E3 T			F4 A3 t		
15 3D NAK	35 F5 5		55 E4 U			75 A4 u			95 3D NAK	B5 F5 5		D5 E4 U			F5 A4 u		
16 32 SYN	36 F6 6		56 E5 V			76 A5 v			96 32 SYN	B6 F6 6		D6 E5 V			F6 A5 v		
17 26 ETB	37 F7 7		57 E6 W			77 A6 w			97 26 ETB	B7 F7 7		D7 E6 W			F7 A6 w		
18 18 CAN	38 F8 8		58 E7 X			78 A7 x			98 18 CAN	B8 F8 8		D8 E7 X			F8 A7 x		
19 19 EM	39 F9 9		59 E8 Y			79 A8 y			99 19 EM	B9 F9 9		D9 E8 Y			F9 A8 y		
1A 3F SUB	3A 7A :		5A E9 Z			7A A9 z			9A 3F SUB	BA 7A :		DA E9 Z			FA A9 z		
1B 27 ESC	3B 5E ;		5B AD [			7B C0 [			9B 27 ESC	BB 5E ;		DB AD [			FB C0 [		
1C 1C FS	3C 4C <		5C BD \			7C A0 \			9C 1C FS	BC 4C <		DC E0 \			FC A0 \		
1D 1D GS	3D 7E =		5D BD]			7D D0]			9D 1D GS	BD 7E =		DD BD]			FD D0]		
1E 1E RS	3E 6E >		5E 5F ^			7E A1 ~			9E 1E RS	BE 6E >		DE 5F ^			FE A1 ~		
1F 1F US	3F 6F ?		5F 6D -			7F 3F ?			9F 1F US	BF 6F ?		DF 6D -			FF 3F ?		

Figure 10: EBCDIC Codes

In-Text Questions

1. Why is floating-point representation preferred over fixed-point?
2. What are the components of floating-point representation?
3. Why is normalisation important in floating-point numbers?
4. What is the main idea behind BCD?
5. What is Excess-3 code?
6. What is the difference between ASCII and EBCDIC?

In-Text Answers

1. Because it can represent a much wider range of values, including very large and very small numbers.
2. Sign bit, exponent, and mantissa.
3. It ensures a unique representation and maximises precision.
4. Each decimal digit is represented separately using a 4-bit binary code.
5. It is a non-weighted code obtained by adding 3 to each decimal digit before converting to binary.
6. ASCII is widely used and standardized globally, while EBCDIC is mainly used in IBM mainframes and has a different binary structure.

1.3.5 Conclusion

Efficient data representation is fundamental to computer architecture and system design. Floating-point representation supports complex numerical computations, while BCD and Excess-N provide alternative numeric encoding methods. ASCII and EBCDIC enable textual data processing across systems. A solid understanding of these techniques allows students to design better algorithms, optimize storage, and ensure accurate data processing in real-world applications.

Summary

This topic explores how computers represent both numerical and textual data. Floating-point representation enables efficient handling of real numbers using scientific notation principles. Other schemes like BCD and Excess-N simplify decimal operations and error detection. Character encoding systems such as ASCII and EBCDIC allow computers to process text. Each representation has its strengths and limitations depending on application requirements.

Self-Assessment Questions

1. Convert decimal 45 to BCD.
2. Convert decimal 7 to Excess-3 code.
3. What is the ASCII code for character 'A'?
4. List two advantages of floating-point representation.
5. What is the bias in Excess-3 code?
6. Mention one limitation of BCD.

Unit Activities

Activity 1: Floating-Point Conversion Practice

- Convert the decimal number 13.25 into normalised binary floating-point form.
- Identify sign, exponent, and mantissa.

Activity 2: BCD Encoding Exercise

- Encode the numbers 29, 84, and 156 using BCD.
- Compare the bit length with pure binary representation.

Activity 3: Character Encoding Task

1. Write a short word (e.g., "DATA") and convert each character into:
 - ASCII
 - (Conceptually) EBCDIC
2. Discuss differences in encoding.

Tutor-Marked Assignment

1. Explain the concept of floating-point representation and its components.
2. Convert the decimal number 18.75 into normalised binary floating-point form.
3. Represent the result using IEEE 754 single-precision format.
4. Explain the concept of Binary Coded Decimal (BCD).
5. Convert the decimal number 276 into BCD.
6. Compare BCD with pure binary representation (advantages and disadvantages).
7. Explain the concept of Excess-3 code and why it is called a biased code.
8. Convert the decimal number 49 into Excess-3 code.
9. State two advantages of Excess-3 over BCD.
10. Explain how characters are represented using ASCII.
11. Convert the word "CODE" into ASCII binary representation.
12. Compare ASCII and EBCDIC.
13. Given the decimal number 25 and the character 'Z':
 - Convert 25 into:
 - Binary
 - BCD
 - Excess-3
 - Convert 'Z' into its ASCII binary equivalent.
 - Briefly explain one scenario where each representation (floating-point, BCD, ASCII) is most suitable.

REFERENCES FOR FURTHER READING

William Stallings – Computer Organization and Architecture

Morris Mano – Computer System Architecture

Andrew S. Tanenbaum – Structured Computer Organization

IEEE Standard 754 – Floating-Point Arithmetic

ISO – Character Encoding Standards

IBM Documentation on EBCDIC Systems

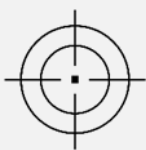
Patterson, D. A., & Hennessy, J. L. – Computer Organization and Design

Hamacher, Vranesic & Zaky – Computer Organization

Unit 4 - Instruction Set Architecture (ISA)

INTRODUCTION

In its basic operation, the CPU alternates between fetching instructions and executing instructions. The instructions cause the processor to manipulate the contents of specific programming model registers and of memory and I/O locations. There are different kinds/categories of instructions a processor operates on.



LEARNING OUTCOMES

1.4.1 Instruction Set Architecture (ISA)

The computer can do certain basic types of operations: load/store, arithmetic logic, test/branch, and input/output. Although the instructions which direct these operations in the CPU vary widely from processor to processor, certain common characteristics are found in all processor instruction sets.

Load/Store Instructions

Load/store operations are those which move data between a register in the processor and a memory location (or another register). They are often collectively referred to as data movement instructions. The two terms load and store are often confused and should be carefully distinguished from each other.

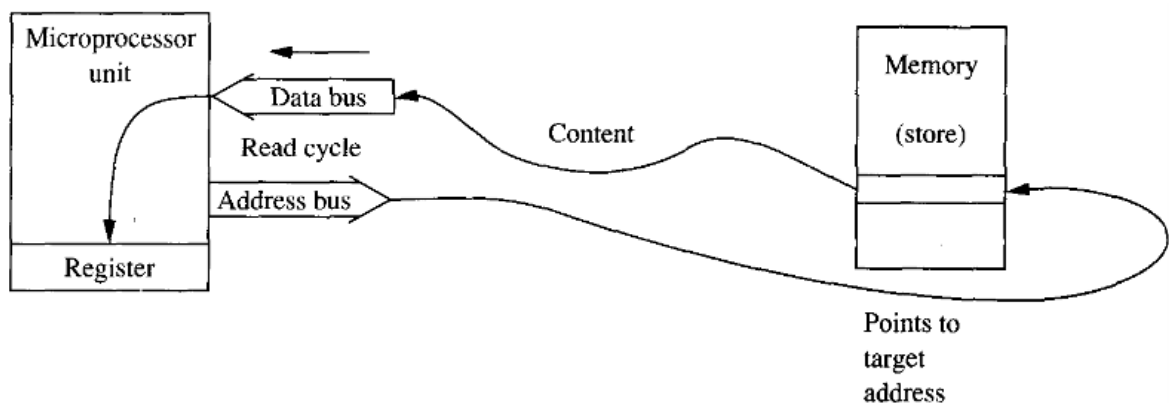


Figure 12a: Load operation

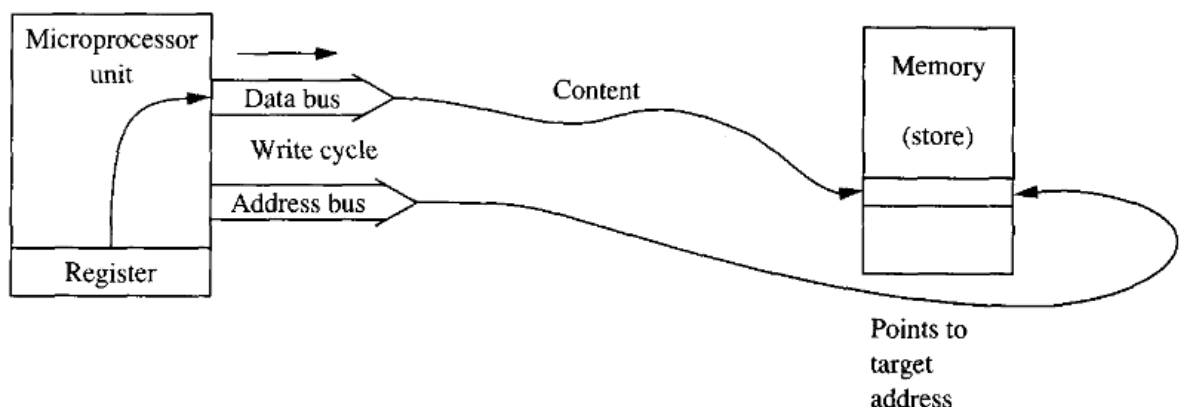


Figure 12b: Store Operation

One of the universal characteristics of instruction execution is that the source does not get changed, only the destination. Thus, in a load operation the processor copies data into a register, and in a store operation the processor copies data from a register into a memory location.

Examples of load/store instructions in the MC6809 processor are **LDA, STA, LDB, STB, LDD, STD**, and so forth.

Examples of load/store instructions in the MC68000 processor are

- MOVE (for either direction of data movement),
- MOVEM (for moving the content of several registers into or out of memory with a single instruction), and
- EXG (for exchanging register contents). These instructions and others like them must include information identifying the source and destination.

Arithmetic/Logic Instructions

Arithmetic/logic operations provide the primary data-processing capabilities of the computer. The minimum arithmetic operations required are those of addition and subtraction.

All processors, implement the basic logic operations of AND, OR, and exclusive-OR on a bit-by-bit basis between two operands. With operations requiring two operands, smaller microprocessors usually require that one of the operands must be in a destination register initially, while the other may be in another register or in memory. Their instructions may identify at most a single operand in memory, as shown in Figure 13.

They support such operations as ADDing or ANDing to processor registers as illustrated but not to memory locations.

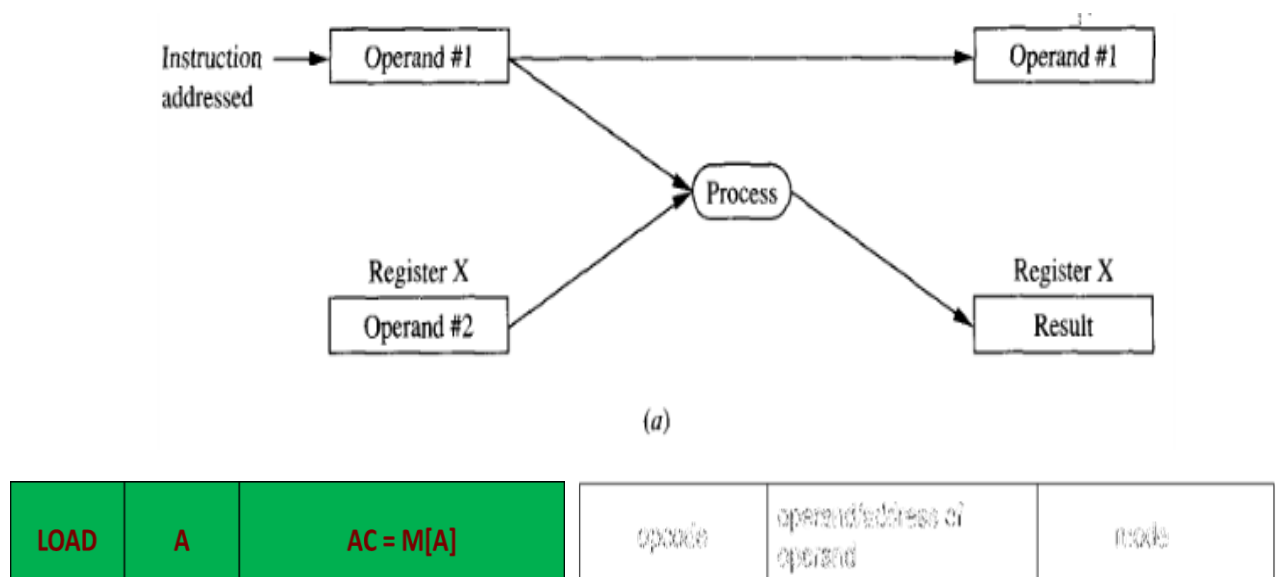
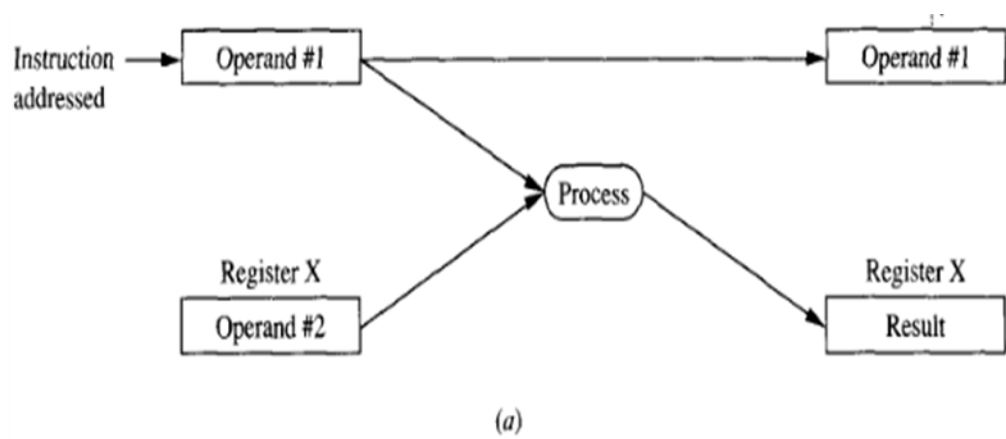
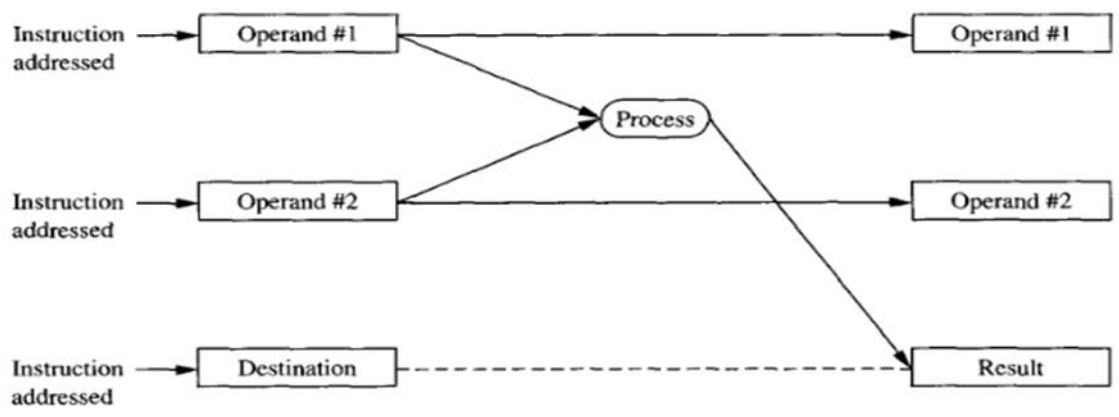


Figure 13: Instruction formats –One Addresses



LOAD	A	AC = M[A]	opcode	operand address of operand	mode
------	---	-----------	--------	----------------------------	------

Store operation Figure 14: Instruction formats –Two



ADD	R1, A, B	R1 = M[A] + M[B]	opcode	Destination address	Source address	Source address	mode
-----	----------	------------------	--------	---------------------	----------------	----------------	------

Figure 15: Instruction formats – Three Addresses

- Some examples of two-operand arithmetic/logic instructions in the MC6809 are ADDB, SUBA, SUBD, ANDA, and EORA (exclusive-OR). Each of these requires one operand to be in an accumulator
- Some two-operand arithmetic/logic instructions in the MC68000 instruction set are ADD, AND, OR, SUB, and EOR.
- All of the two-operand instructions in the MC68000 must specify the source and the destination. With a few exceptions, these may both be registers or memory locations or one of each

Test/Branch Instructions

Test/branch operations provide the decision-making capabilities in computer programs.

The test is often performed on the result of some prior arithmetic/logic operation. It determines whether the result was zero or nonzero, positive or negative, or some other binary choice involving bits in the condition code register. During the execution of a test/branch instruction the processor examines the appropriate condition code bits to determine the result of the test.

Depending upon the result, the processor may or may not branch to a remote location for the next instruction. Examples of test/branch instructions are BCC (branch if carry is cleared), BEQ (branch if equal to 0), BMI (branch if minus), and BRA (branch always).

1.4.2 Instruction Formats

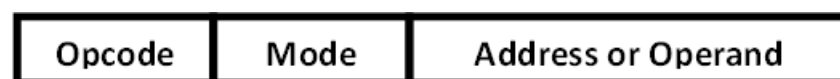
An instruction is usually made up of a combination of an operation code and some way of specifying an operand, most commonly by its location or address in memory.

Instruction Format is the layout of bits in an instruction

An instruction is divided into Opcode (function), operand(s) (implicit or explicit) and addressing mode.

- Instructions typically have 0,1,2 or 3 operands
- There is usually more than one instruction format in an instruction set

A computer will usually have a variety of instruction code formats. It is the function of the control unit within the CPU to interpret each instruction code and provide the necessary control functions needed to process the instruction.



The format of an instruction is usually depicted in a rectangular box symbolizing the bits of the instruction as they appear in memory words or in a control register. The bits of the instruction are divided into groups called fields. The most common fields found in instruction formats are:

1. An operation code field that specifies the operation to be performed.
 2. An address field that designates a memory address or a processor register.
 3. A mode field that specifies the way the operand or the effective address is determined.
- Other special fields are sometimes employed under certain circumstances, as for example a field that gives the number of shifts in a shift-type

instruction. The operation code field of an instruction is a group of bits that define various processor operations, such as add, subtract, complement, and shift.

- The bits that define the mode field of an instruction code specify a variety of alternatives for choosing the operands from the given address.
- Operations specified by computer instructions are executed on some data stored in memory or processor registers. Operands residing in memory are specified by their memory address. Operands residing in processor registers are specified with a register address.
- A register address is a binary number of k bits that defines one of ' 2^k ' registers in the CPU. Thus, a CPU with 16 processor registers R0 through R15 will have a register address field of four bits.

The binary number 0101, for example, will designate register R5. Computers may have instructions of several different lengths containing a varying number of addresses. The number of address fields in a computer's instruction format depends on its register organization.

1.4.3 CPU Organization

There are three categories of CPU organization based on the number of address fields in the instruction formats. They include:

- Single accumulator organization
- General register organization
- Stack organization

Single Accumulator organization

In an accumulator-type organization all operations are performed with an implied accumulator register. The instruction format in this type of computer uses one address field. For example, the instruction that specifies an arithmetic addition is defined by an assembly language instruction as ADD X where X is the address of the operand. The ADD instruction in this case results in the operation $AC \leftarrow AC + M[X]$. AC is the accumulator register and M[X] symbolizes the memory word located at address X.

General register organization

In a general register type of organization, the instruction format in this type of computer needs three register address fields. Thus, the instruction for an arithmetic

addition may be written in an assembly language as `ADD R1, R2, R3` to denote the operation $R1 \leftarrow R2 + R3$.

The number of address fields in the instruction can be reduced from three to two if the destination register is the same as one of the source registers. Thus the instruction `ADD R1, R2` would denote the operation $R1 \leftarrow R1 + R2$. Only register addresses for R1 and R2 need be specified in this instruction.

Computers with multiple processor registers use the move instruction with a mnemonic `MOV` to symbolize a transfer instruction. Thus the instruction `MOV R1, R2` denotes the transfer $R1 \leftarrow R2$ (or $R2 \leftarrow R1$, depending on the particular computer).

Thus transfer-type instructions need two address fields to specify the source and the destination. General register-type computers employ two or three address fields in.

Stack-based CPU Organization

The computers which use Stack-based CPU Organization are based on a data structure called stack. The stack is a list of data words. It uses Last in First out (LIFO) access method which is the most popular access method in most of the CPU. A register is used to store the address of the topmost element of the stack which is known as Stack pointer (SP). In this organization, ALU operations are performed on stack data. It means both the operands are always required on the stack. After manipulation, the result is placed in the stack.

The main two operations that are performed on the operators of the stack are Push and Pop. These two operations are performed from one end only.

PUSH

This operation results in inserting one operand at the top of the stack and it decrease the stack pointer register. The format of the PUSH instruction is:

`PUSH`

It inserts the data word at specified address to the top of the stack. It can be implemented as:

`//decrement SP by 1`

`SP <-- SP - 1`

`//store the content of specified memory address`

//into SP; i.e, at top of stack

SP <-- (memory address)

POP

This operation results in deleting one operand from the top of the stack and it increase the stack pointer register. The format of the POP instruction is:

POP

It deletes the data word at the top of the stack to the specified address. It can be implemented as:

//transfer the content of SP (i.e, at top most data)

//into specified memory location

(memory address) <-- SP

//increment SP by 1

SP <-- SP + 1

Operation type instruction does not need the address field in this CPU organization. This is because the operation is performed on the two operands that are on the top of the stack. For example:

SUB

This instruction contains the opcode only with no address field. It pops the two top data from the stack, subtracting the data, and pushing the result into the stack at the top.

PDP-11, Intel's 8085 and HP 3000 are some of the examples of the stack organized computers.

The advantages of Stack based CPU organization

- Efficient computation of complex arithmetic expressions.
- Execution of instructions is fast because operand data are stored in consecutive memory locations.
- Length of instruction is short as they do not have address field.

The disadvantages of Stack based CPU organization

- The size of the program increases.

1.4.4 Categories of Instruction Formats

The categories of instruction formats are based on the number of operands and they are discussed below.

Three-Address Instructions

Computers with three-address instruction formats can use each address field to specify either a processor register or a memory operand. The program in assembly language that evaluates $X = (A + B) * (C + D)$ is shown below, together with comments that explain the register transfer operation of each instruction.

```
ADD R1, A, B  R1 ← M[A] + M[B]
```

```
ADD R2, C, D  R2 ← M[C] + M[D]
```

```
MOL X, R1, R2 M[X] ← R1 * R2
```

- It is assumed that the computer has two processor registers, R1 and R2. The symbol M[A] denotes the operand at memory address symbolized by A.
- The advantage of the three-address format is that it results in short programs when evaluating arithmetic expressions.
- The disadvantage is that the binary-coded instructions require too many bits to specify three addresses.

Two-Address Instructions

Two-address instructions are the most common in commercial computers. Here again each address field can specify either a processor register or a memory word. The program to evaluate $X = (A + B) * (C + D)$ is as follows:

```
MOV R1, A  R1 ← M[A]
```

```
ADD R1, B  R1 ← R1 + M[B]
```

```
MOV R2, C  R2 ← M[C]
```

```
ADD R2, D  R2 ← R2 + M[D]
```

```
MOL R1, R2 R1 ← R1 * R2
```

```
MOV X, R1  M[X] ← R1
```

The MOV instruction moves or transfers the operands to and from memory and processor registers.

The first symbol listed in an instruction is assumed to be both a source and the destination where the result of the operation is transferred.

One-Address Instructions

One-address instructions use an implied accumulator (AC) register for all data manipulation. For multiplication and division there is a need for a second register. However, here we will neglect the second register and assume that the AC contains the result of all operations.

The program to evaluate $X = (A + B) * (C + D)$ is

```

LOAD A  AC ← M[A]

ADD B   AC ← AC + M[B]

STORE T M[T] ← AC

LOAD C  AC ← M[C]

ADD D   AC ← AC + M[D]

MOL T   AC ← AC * M[T]

STORE X M[X] ← AC
  
```

All operations are done between the AC register and a memory operand. T is the address of a temporary memory location required for storing the intermediate result.

Zero-Address Instructions

A stack-organized computer does not use an address field for the instructions ADD and MUL. The PUSH and POP instructions, however, need an address field to specify the operand that communicates with the stack. The following program shows how $X = (A + B) * (C + D)$ will be written for a stack organized computer. (TOS stands for top of stack).

```

PUSH A  TOS ← A

PUSH B  TOS ← B

ADD     TOS ← (A + B)

PUSH C  TOS ← C

PUSH D  TOS ← D
  
```

ADD TO $S \leftarrow (C + D)$

MOL TOS $\leftarrow (C + D) * (A + B)$

POP X $M[X] \leftarrow TOS$

To evaluate arithmetic expressions in a stack computer, it is necessary to convert the expression into reverse Polish notation. The name "zero-address" is given to this type of computer because of the absence of an address field in the computational instructions.

1.4.5 Addressing Modes

Addressing modes are an aspect of the instruction set architecture (ISA) in most CPU designs. The various addressing modes that are defined in a given ISA define how machine language instructions in that architecture identify the operand (or operands) of each instruction. Addressing Mode is a specification for generating the address of an operand at run time. RISC processors do not do any CPU operations (e.g. add, shift, mul, etc.) using memory registers—all operands must be in CPU registers.

Thus, addressing modes for memory operands only apply to "Load" and "Store" type instructions. CISC processors do almost all CPU operations (e.g. add, shift, mul, etc.) using both CPU registers and memory registers. Thus, addressing modes are a critical part of compiling almost all of the instructions. The effective address (EA) of an operand is the address in memory where the operand is stored and is usually a calculated value.

The operation field of an instruction specifies the operation to be performed.

This operation must be executed on some data stored in computer registers or memory words.

The way the operands are chosen during program execution is dependent on the addressing mode of the instruction. The addressing mode specifies a rule for interpreting or modifying the address field of the instruction before the operand is actually referenced.

Computers use addressing mode techniques for the purpose of accommodating one or both of the following provisions

- To give programming versatility to the user by providing such facilities as pointers to memory, counters for loop control, indexing of data, and program relocation.

- To reduce the number of bits in the addressing field of the instruction.

Numerical Example

- The two-word instruction at address 200 and 201 is a "load to AC" instruction with an address field equal to 500.
- The first word of the instruction specifies the operation code and mode, and the second word specifies the address part.
- PC has the value 200 for fetching this instruction. The content of processor register R1 is 400, and the content of an index register XR is 100.
- AC receives the operand after the instruction is executed. The figure lists a few pertinent addresses and shows the memory content at each of these addresses.

PC = 200	
R1 = 400	
XR = 100	
AC	

Address	Memory
200	Load to AC Mode
201	Address = 500
202	Next instruction
399	450
400	700
500	800
600	900
702	325
800	300

- The mode field of the instruction can specify any one of a number of modes. For each possible mode we calculate the effective address and the operand that must be loaded into AC.
- In the direct address mode, the effective address is the address part of the instruction 500 and the operand to be loaded into AC is 800. In the immediate mode the second word of the instruction is taken as the operand rather than an address, so 500 is loaded into AC. (The effective address in this case is 201)
- In the indirect mode the effective address is stored in memory at address 500. Therefore, the effective address is 800 and the operand is 300. In the relative mode the effective address is $500 + 202 = 702$ and the operand is 325.
- (Note that the value in PC after the fetch phase and during the execute phase is 202.) In the index mode the effective address is $XR + 500 = 100 +$

500 = 600 and the operand is 900. In the register mode the operand is in R1 and 400 is loaded into AC.

- (There is no effective address in this case.) In the register indirect mode the effective address is 400, equal to the content of R1 and the operand loaded into AC is 700.
- The auto increment mode is the same as the register indirect mode except that R1 is incremented to 401 after the execution of the instruction. The auto-decrement mode decrements R1 to 399 prior to the execution of the instruction.
- The operand loaded into AC is now 450. The table below lists the values of the effective address and the operand loaded into AC for the nine addressing modes.

Table: Tabular List of Numerical Example

Addressing Mode	Effective Address	Content of AC
Direct address	500	800
Immediate operand	201	500
Indirect address	800	300
Relative address	702	325
Indexed address	600	900
Register	—	400
Register indirect	400	700
Autoincrement	400	700
Autodecrement	399	450

1.4.6 Conclusion

In this unit, you have learnt the basic concept of instruction set architecture. The importance of designing an instruction set architecture has also been identified. Two additional topics covered in this are part of the design consideration of an ISA for a processor. Instruction format is concerned with how the instructions are presented in an ISA while addressing modes deals with how operand is made available at run-time. ISA is processor-specific which means, it can be used to describe or differentiate different versions or models of a processor.

Summary

This unit has discussed different aspects of an instruction set architecture such as instruction format, CPU organization, and addressing modes. Instruction set architecture is a protocol that defines how a computer machine is viewed by machine language programmer and compiler. As regards instruction formats cpu organization, three categories of design were discussed namely single accumulator-based, general register-based and stack-based organization. With respect to instruction format, four different were studied namely zero operand, one, two, and three operands types of instructions. Finally, the unit also covered addressing modes which specify how operands are provided at runtime. A number of addressing modes were discussed with practical numerical example to support.

Self-Assessment Questions

1. What is Instruction Set Architecture (ISA)?
2. What are the main components of an instruction format?
3. What is an opcode?
4. List four types of instructions.
5. What is immediate addressing mode?
6. What is the difference between direct and indirect addressing?
7. What type of instruction is "JMP 1000"?
8. Explain register addressing mode.
9. What is indexed addressing mode?
10. What is the purpose of instruction formats?
11. What happens during instruction decoding?
12. Analyze how instruction format affects processor performance.
13. Why is addressing mode important in ISA design?

Unit Activities

Activity 1: Instruction Format Analysis

Task: Analyze the following 32-bit instruction format:

| Opcode (8 bits) | Register (8 bits) | Address (16 bits) |

Break down the instruction:

00001100 00000010 00000000000010100

Questions:

1. What is the opcode?
2. Which register is used?
3. What is the memory address?

Activity 2: Instruction Classification

Task: Classify the following instructions:

Instruction	Type
MOV R1, R2	?
ADD R1, R3	?
JMP 2000	?
AND R1, R2	?

Expected Output:

- Data transfer
- Arithmetic
- Control
- Logical

Activity 3: Addressing Modes Identification

Task: Determine the addressing mode used:

Instruction	Addressing Mode
MOV R1, #10	?
MOV R1, R2	?
MOV R1, 2000	?
MOV R1, (R2)	?

Tutor-Marked Assignment

1. If an instruction contains four addresses, what might be the purpose of each address?
2. List and briefly explain five important instruction set design issues.
3. What types of operands are typical in machine instruction sets?
4. Consider a processor with 64 registers and an instruction set of size twelve. Each instruction has five distinct fields, namely, opcode, two source register identifiers, one destination register identifier, and a twelve-bit immediate value. Each instruction must be stored in memory in a byte-aligned fashion. If a program has 100 instructions, determine the amount of memory (in bytes) consumed by the program text.
5. A processor has 40 distinct instructions and 24 general purpose registers. A 32-bit instruction word has an opcode, two registers operands and an immediate operand. Determine the number of bits available for the immediate operand field.

References/Further Readings

Computer Organization and Architecture: Designing for performance by William Stallings, Tenth edition (2016). Read all sections in Chapter 12

http://home.ustc.edu.cn/~leedsong/reference_books_tools/Computer%20Organization%20and%20Architecture%2010th%20-%20William%20Stallings.pdf

Other internet resources may be used also.

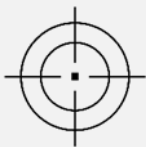
Module Two

Processor Design and Performance

Unit 1 — Processor Components

INTRODUCTION

The processor, also known as the Central Processing Unit (CPU), is the brain of a computer system. It is responsible for executing instructions, performing computations, and controlling the operations of other system components.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Describe the major components of a processor.
- Explain the roles of registers, ALU, and the control unit.
- Differentiate between hardwired and microprogrammed control units.
- Analyse the instruction cycle in detail.

2.1.1 Processor Components

A processor is composed of several key functional units that work together to execute programs efficiently. Understanding these components is essential for analysing system performance, instruction execution, and architectural design.

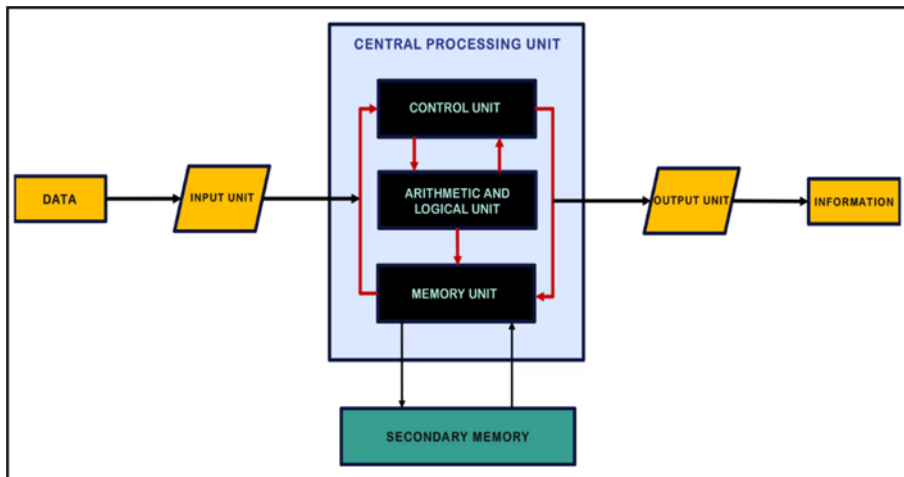


Figure 16: Components of Processor

Registers

Registers are small, high-speed storage locations inside the CPU used to store data, instructions, and addresses temporarily during execution.

Types of Registers

- General Purpose Registers (GPRs)
 - Store operands and intermediate results.
 - Example: R1, R2, etc.
- Special Purpose Registers
 - Program Counter (PC): Holds the address of the next instruction.
 - Instruction Register (IR): Holds current instruction.
 - Memory Address Register (MAR): Holds memory address.
 - Memory Data Register (MDR): Holds data from memory.
 - Accumulator (ACC): Stores arithmetic results.
 - Status/Flag Register: Holds condition flags (Zero, Carry, Overflow).

Scenario

A program performs repeated additions:

Instead of accessing RAM each time, operands are loaded into registers for faster execution.

Arithmetic Logic Unit (ALU)

The ALU is the component responsible for performing arithmetic and logical operations.

Functions of ALU

- Arithmetic: Addition, subtraction, multiplication, division
- Logical: AND, OR, NOT, XOR
- Comparison: Equal, greater than, less than

ALU Operations Example

R1 = 5, R2 = 3

ADD R1, R2 → R1 = 8

Control Unit (CU)

The Control Unit directs all operations of the CPU by generating control signals.

Functions

- Fetch instructions from memory
- Decode instructions
- Control data movement
- Coordinate ALU and registers

Control Signals Examples

- Read/Write signals
- ALU operation selection
- Register enable/disable

2.1.2 Hardwired vs Microprogrammed Control Unit

Hardwired Control Unit

A Hardwired Control Unit (HCU) is a type of control unit in which control signals are generated using fixed hardware circuits such as logic gates, decoders, multiplexers, and flip-flops. The control logic is physically wired into the processor, meaning it is not programmable.

How It Works

In a hardwired control unit:

- The instruction is fetched into the Instruction Register (IR).
- The opcode (operation code) is decoded using a decoder.
- Based on the opcode and the current timing signals, the control unit generates specific control signals.
- These signals directly control:
 - Registers (load, store, transfer)
 - ALU operations
 - Memory read/write operations

Key Components

- Instruction Register (IR): Holds the current instruction.
- Decoder: Interprets opcode.
- Control Logic Gates: Generate control signals.
- Timing/Sequence Counter: Ensures operations occur in the correct order.

Example of Operation

Instruction: ADD R1, R2

Execution in Hardwired Control:

- Opcode is decoded as ADD.
- Control signals are generated instantly:
 - Enable R1 and R2 outputs
 - Select the ALU addition operation
 - Store result back in R1

All these actions are performed through predefined hardware pathways.

Characteristics

- Control signals are generated by combinational logic circuits
- Execution is very fast
- No control memory is required
- Difficult to modify once designed

Advantages

- High Speed
 - No need to fetch microinstructions
 - Direct signal generation
- Efficient for Simple Architectures: works well with RISC processors
- Low Latency: faster instruction execution cycle

Disadvantages

- **Inflexibility:** Cannot easily add new instructions
- **Complex Design:** Becomes complicated for large instruction sets
- **Difficult to Debug:** Errors require hardware redesign

Microprogrammed Control Unit

A Microprogrammed Control Unit (MCU) is a type of control unit in which control signals are generated by executing a sequence of microinstructions stored in a special memory called control memory.

Instead of using fixed hardware logic (as in hardwired control), it uses a program-like approach to control CPU operations. The basic idea includes:

- Each machine-level instruction (e.g., ADD, LOAD) is broken down into smaller steps.
- These steps are implemented as microinstructions.
- A sequence of microinstructions forms a microprogram.
- The microprogram generates the required control signals step-by-step.

Key Components

- Control Memory
 - Stores microprograms.
 - Typically implemented using ROM or RAM.
- Microprogram Counter (μ PC): holds the address of the next microinstruction.

- Microinstruction Register (MIR): holds the current microinstruction.
- Control Address Register (CAR): points to control memory location.
- Sequencer (Control Logic): determines next microinstruction address.

How It Works

- Instruction is fetched into the Instruction Register (IR).
- Opcode is used to locate the starting address of the microprogram.
- Microinstructions are fetched sequentially from control memory.
- Each microinstruction generates specific control signals.
- Execution continues until the instruction is completed.

Example of Operation

Instruction: ADD R1, R2

Micro-operations:

- Load R1 into ALU input
- Load R2 into ALU input
- Perform addition
- Store result back in R1

Each step corresponds to a microinstruction.

Advantages

- Flexibility: eEasy to modify or add new instructions, and only the microcode needs updating
- Simpler Design: easier to implement complex instruction sets
- Better for CISC Architectures: handles complex instructions efficiently

Disadvantages

- Slower Speed: Requires fetching microinstructions
- Control Memory Overhead: Needs additional memory
- Extra Processing Step: Instruction → microinstructions → execution

Scenario

A processor manufacturer wants to fix a bug in instruction execution:

- Hardwired control: Redesign hardware
- Microprogrammed control: Update microcode (firmware patch)

2.1.3 Instruction Cycle Analysis

The instruction cycle (also called the fetch-decode-execute cycle) is the sequence of operations performed by the CPU to execute a single instruction from a program.

It is a continuous process that repeats for every instruction in memory, enabling program execution.

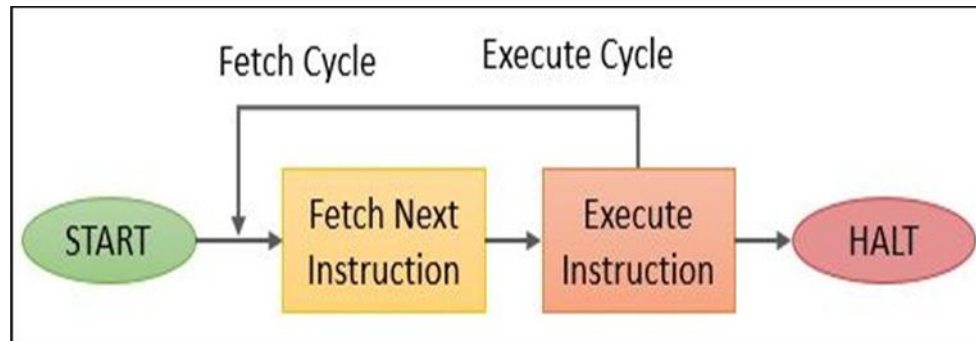


Figure 17: Instruction Cycle

Phases of the Instruction Cycle

1. Fetch
2. Decode
3. Execute
4. Memory Access (if needed)
5. Write Back

Fetch Stage

The CPU retrieves the next instruction from memory.

Operations

- The Program Counter (PC) contains the address of the next instruction.
- Address is sent to Memory Address Register (MAR).
- Instruction is fetched into the Memory Data Register (MDR).
- Instruction is transferred to the Instruction Register (IR).
- PC is incremented.

Example

If PC = 100, the CPU fetches the instruction stored at memory location 100.

Decode Stage

The control unit interprets the instruction.

Operations

- Opcode is decoded.
- Identify:
 - Operation type (ADD, LOAD, etc.)
 - Operands involved
- Determine required control signals.

Execute Stage

The actual operation is performed.

Operations

- ALU performs arithmetic/logic operations.
- Data may be transferred between registers.
- Branch conditions may be evaluated.

Example

ADD R1, R2 $\rightarrow R1 = R1 + R2$

Memory Access Stage (Optional)

Occurs if the instruction requires memory interaction.

Operations

Read data from memory (LOAD)

Write data to memory (STORE)

Write Back Stage

The result of execution is stored.

Operations

The result is written into a register or memory location.

Step-by-Step Example

Instruction: LOAD R1, 200

Execution Steps

- Fetch: Instruction fetched from memory
- Decode: Identify as a LOAD operation
- Execute: Address 200 prepared
- Memory Access: Data fetched from memory location 200
- Write Back: Data stored in R1

Another Example

Instruction: ADD R1, R2

Execution

- Fetch instruction
- Decode as ADD
- Execute addition in ALU
- Store result in R1

Types of Instruction Cycles

- Fetch Cycle
- Execute Cycle
- Interrupt Cycle (if an interrupt occurs)

Interrupt Cycle (Brief Note)

Occurs when an external/internal interrupt is detected

- CPU suspends current instruction
- Executes interrupt service routine (ISR)
- Returns to normal execution

Real-Life Analogy

Think of the instruction cycle like following a cooking process:

- Fetch → Pick recipe
- Decode → Understand instructions
- Execute → Cook
- Memory → Get ingredients if needed
- Write Back → Serve dish

Key Characteristics

- Sequential process
- Repeats continuously

- Controlled by the control unit
- Synchronised using clock cycles

Importance of the Instruction Cycle

- Determines CPU performance
- Affects execution speed
- Basis for advanced techniques like:
 - Pipelining
 - Parallel processing

Real-Life Scenario

- A banking application processes transactions:
- Registers store temporary balances
- ALU computes interest
- Control unit ensures correct instruction flow
- Instruction cycle repeats for each transaction

In-Text Questions

1. Why are registers faster than main memory?
2. What happens if an arithmetic operation exceeds the register size?
3. Does the control unit perform arithmetic operations?
4. Why is a microprogrammed control unit more flexible than a hardwired control unit?
5. Which control unit is used in modern complex processors?
6. Why is the Program Counter incremented after fetching an instruction?
7. What component is responsible for decoding instructions?
8. Do all instructions require memory access?
9. What happens if an interrupt occurs during execution?
10. Why is the PC incremented after fetch?

In-Text Answers

1. Registers are located inside the CPU and use faster circuitry, unlike main memory, which requires bus communication.

2. An overflow flag is set in the status register.
3. No, it only directs operations; the ALU performs computations.
4. Because its behavior is defined by microprograms stored in memory, which can be modified without changing hardware.
5. Microprogrammed control is often used for complex instruction sets (CISC).
6. To point to the next instruction in sequence.
7. The Control Unit.
8. No, only instructions like LOAD and STORE require it.
9. The CPU temporarily stops current execution and handles the interrupt.
10. To point to the next instruction in memory.

2.1.4 Conclusion

Understanding processor components is fundamental to computer architecture. It enables students to analyse system performance, design efficient processors, and understand how software interacts with hardware. Mastery of these concepts is essential for advanced topics like pipelining, parallel processing, and CPU optimisation.

Summary

- The CPU consists of registers, ALU, and a control unit.
- Registers provide fast data access.
- ALU performs computations.
- Control unit directs operations.
- Control can be hardwired or microprogrammed.
- The instruction cycle governs how instructions are executed step-by-step.

Self-Assessment Questions

1. What is the function of the MAR?
2. What is the difference between IR and PC?
3. Why is microprogrammed control flexible?
4. List two ALU operations.
5. What are the stages of the instruction cycle?

Unit Activities

Activity 1: Draw a block diagram of the CPU showing:

- Registers
- ALU
- Control Unit

Activity 2: Simulate instruction execution:

- LOAD R1, 100
- ADD R1, R2

Track register changes step-by-step.

Activity 3: Compare execution time:

- Hardwired vs Microprogrammed control

Tutor-Marked Assignment

1. Explain the roles of:

- Registers
- ALU
- Control Unit

Use diagrams and examples.

2. Compare hardwired and microprogrammed control units with real-world examples.

3. Trace the execution of:

LOAD R1, 200

ADD R1, R2

STORE R1, 300

4. Design a simple CPU model and describe how instructions are processed.

5. Discuss how the instruction cycle affects system performance.

6. Explain the importance of registers in high-speed processing.

References For Further Reading

Hennessy, J. L., & Patterson, D. A. – Computer Architecture: A Quantitative Approach

Stallings, W. – Computer Organization and Architecture

Mano, M. M. – Computer System Architecture

Tanenbaum, A. S. – Structured Computer Organization

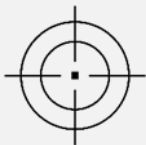
Patterson & Hennessy – Computer Organization and Design (RISC-V Edition)

Hamacher, Vranesic, Zaky – Computer Organization

Unit 2 — CPU Performance Metrics

INTRODUCTION

Performance metrics are essential tools used to evaluate and compare the efficiency of computer systems. In computer architecture, performance determines how fast a system executes programs and how efficiently it utilises hardware resources.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain clock speed and clock cycles.
- Define and compute Cycles Per Instruction (CPI).
- Evaluate performance using MIPS and benchmarking techniques.
- Apply Amdahl's Law to performance improvement scenarios.

2.2.1 Clock Speed and Clock Cycles

In computer systems, all operations performed by the CPU are synchronised using a clock signal. This clock acts like a metronome, coordinating when operations begin and end. Two fundamental concepts used to measure processor performance are:

- Clock Speed (Clock Rate)
- Clock Cycles

Clock speed and clock cycles form the foundation of CPU performance analysis. While higher clock speeds can improve performance, the number of cycles required per instruction plays an equally critical role. A balanced optimisation of both leads to efficient system design. Furthermore, understanding these concepts is essential because they directly influence how fast a processor can execute instructions.

Clock Speed (Clock Rate)

Clock speed is the number of clock cycles per second that a processor can execute.

Unit

- Measured in Hertz (Hz)
- Common units:
 - kHz = 10^3 Hz
 - MHz = 10^6 Hz
 - GHz = 10^9 Hz

Interpretation

- A 1 GHz processor performs 1 billion cycles per second
- A 3 GHz processor performs 3 billion cycles per second

Clock Signal

The clock signal is an electrical signal that oscillates between HIGH and LOW states.

Each complete oscillation represents one clock cycle.

Clock Period

Clock period is the time taken for one clock cycle.

Clock Period = $1/\text{Clock Speed}$

Example

If clock speed = 2 GHz:

Clock Period = $1/2 \times 10^9 = 0.5 \text{ ns}$

Clock Cycles

A clock cycle is the smallest unit of time in which a CPU performs a basic operation.

Each instruction requires one or more clock cycles.

Role in Instruction Execution

During each clock cycle:

- Data is transferred
- Operations are performed
- Control signals are activated

Number of Cycles per Instruction

Different instructions require different numbers of cycles:

Instruction Type	Cycles Required
Simple arithmetic	1–2 cycles
Memory access	3–5 cycles
Complex operations	More cycles

2.2.2 Relationship Between Clock Speed and Clock Cycles

Key Formula

CPU Execution Time = Number of Clock Cycles / Clock Speed

OR

CPU Time = Instruction Count × CPI × Clock Cycle Time

Interpretation

- Higher clock speed → shorter execution time
- More cycles → longer execution time

Important Insight

Performance depends on:

- Clock speed
- Number of cycles
- Instruction efficiency

Factors Affecting Clock Speed

- Processor design
- Heat dissipation
- Power consumption
- Semiconductor technology
- Pipeline depth

Sample Solved Questions

Example 1: Execution Time

A processor runs at 2 GHz and executes a program requiring 6 billion cycles. Find execution time.

Solution

$$\text{Execution Time} = 6 \times 10^9 / 2 \times 10^9$$

$$= 3 \text{ seconds}$$

Example 2: Clock Cycles

A program runs for 4 seconds on a 1 GHz processor. How many clock cycles were used?

Solution

$$\text{Cycles} = \text{Clock Speed} \times \text{Time}$$

$$= 1 \times 10^9 \times 4 = 4 \times 10^9 \text{ cycles}$$

Example 3: Clock Period

Find the clock period of a 2.5 GHz processor.

Solution

$$\text{Clock Period} = 1/2.5 \times 10^9$$

$$= 0.4 \text{ ns}$$

Example 4: Comparing Two Processors

Processor A:

- Clock speed = 3 GHz
- Cycles = 9 billion

Processor B:

- Clock speed = 2 GHz
- Cycles = 6 billion

Which is faster?

Solution

Processor A:

$$9 \times 10^9 / 3 \times 10^9 = 3 \text{ seconds}$$

Processor B:

$$6 \times 10^9 / 2 \times 10^9 = 3 \text{ seconds}$$

Conclusion: Both processors have equal performance.

Example 5: Effect of Optimisation

A program originally takes 10 billion cycles on a 2 GHz processor. After optimisation, the cycles are reduced to 6 billion. Find speedup.

Solution

Original time:

$$10 \times 10^9 / 2 \times 10^9 = 5 \text{ seconds}$$

New time:

$$6 \times 10^9 / 2 \times 10^9 = 3 \text{ seconds}$$

Speedup:

$$5/3 \approx 1.67$$

Example 6: Using CPI

A program executes 1 million instructions with an average CPI of 2 on a 1 GHz processor. Find execution time.

Solution

$$\text{Total Cycles} = 1 \times 10^6 \times 2 = 2 \times 10^6$$

$$\text{Time} = 2 \times 10^6 / 1 \times 10^9 = 2 \times 10^{-3}$$

$$= 0.002 \text{ seconds}$$

Common Misconceptions

1. Higher clock speed always means better performance → False
2. All instructions take one cycle → False
3. Clock speed alone determines performance → False

2.2.3 Cycles Per Instruction (CPI)

In evaluating processor performance, one of the most important metrics is Cycles Per Instruction (CPI). While clock speed tells us how fast the processor runs, CPI tells us how efficiently instructions are executed.

A processor with a high clock speed but poor CPI may perform worse than a slower processor with a low CPI. Therefore, CPI is essential for understanding real CPU performance.

Definition of CPI

Cycles Per Instruction (CPI) is the average number of clock cycles required to execute one instruction.

$$\text{CPI} = \text{Total Clock Cycles} / \text{Total Number of Instructions}$$

Interpretation of CPI

- CPI = 1 → Each instruction takes 1 cycle (very efficient)
- CPI > 1 → Instructions take multiple cycles
- Lower CPI = Better performance

Relationship Between CPI and Performance

The CPU execution time can be expressed as:

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

OR

$$\text{CPU Time} = (\text{Instruction Count} \times \text{CPI}) / \text{Clock Speed}$$

Key Insight

Performance depends on:

- Instruction count
- CPI
- Clock speed

Factors Affecting CPI

1. Instruction Type: Different instructions require different cycles:

Instruction Type	Typical CPI
Arithmetic	1
Memory access	2–5
Branch	2–4

2. Memory Access Delays

- Cache misses increase CPI
- Slow memory → more cycles

3. Pipeline Stalls

- Hazards (data, control, structural)
- Causes delays in execution

4. Processor Architecture

- RISC → lower CPI
- CISC → higher CPI (but fewer instructions)

5. Branch Prediction

- Poor prediction increases CPI due to pipeline flushing

Average CPI (Weighted CPI)

In real programs, instructions are mixed. So, CPI is computed as:

$$\text{Average CPI} = \sum (\text{CPI}_i \times \text{Frequency}_i)$$

Example

Instruction	CPI	Frequency
Arithmetic	1	50%
Memory	4	30%
Branch	2	20%

$$\text{CPI} = (1 \times 0.5) + (4 \times 0.3) + (2 \times 0.2) = 0.5 + 1.2 + 0.4 = 2.1$$

Examples

Example 1: Basic CPI Calculation

A program executes 200 instructions and takes 600 clock cycles. Find CPI.

Solution

$$\text{CPI} = 600/200 = 3$$

Example 2: Execution Time Using CPI

A processor executes:

- 1 million instructions
- CPI = 2
- Clock speed = 1 GHz

Find execution time.

Solution

$$\text{CPU Time} = 1 \times 10^6 \times 2 / 1 \times 10^9$$

$$= 2 \times 10^{-3}$$

$$= 0.002 \text{ seconds}$$

Example 3: Weighted CPI

Find average CPI.

Instruction Type	Count	CPI
Arithmetic	500	1
Memory	300	4
Branch	200	2

Solution

$$\text{Total instructions: } 500 + 300 + 200 = 1000$$

$$\text{Total cycles: } (500 \times 1) + (300 \times 4) + (200 \times 2) = 500 + 1200 + 400 = 2100$$

$$\text{CPI} = 2100/1000 = 2.1$$

Example 4: Effect of Reducing CPI

A program has:

- Instruction count = 2 million
- CPI = 3
- Clock speed = 1 GHz

If CPI is reduced to 2, find the speedup.

Solution

Original time: $2 \times 10^6 \times 3 / 10^9 = 0.006 \text{ s}$

New time: $2 \times 10^6 \times 2 / 10^9 = 0.004 \text{ s}$

Speedup: $0.006 / 0.004 = 1.5$

Example 5: Finding Instruction Count

A program runs in 0.01 seconds on a 2 GHz processor with CPI = 2. Find instruction count.

Solution

Instruction Count = (CPU Time × Clock Speed)/CPI

= $0.01 \times 2 \times 10^9 / 2$

= 0.01×10^9

= 10⁷

= 10 million instructions

Key Observations

- Lower CPI improves performance
- CPI varies across instruction types
- Memory delays significantly increase CPI
- Pipeline efficiency reduces CPI

2.2.4 MIPS and Benchmarking (CPI)

Performance evaluation is a fundamental aspect of computer architecture. Designers and users need reliable ways to measure how fast and efficiently a system executes programs. Two widely used approaches are:

- MIPS (Million Instructions Per Second) – a raw performance metric
- Benchmarking – a practical, workload-based evaluation method

While MIPS provides a quick estimate of CPU speed, benchmarking offers a more realistic assessment of system performance under real-world conditions.

Definition of MIPS

MIPS stands for Million Instructions Per Second, and it measures the rate at which a processor executes instructions.

$$\text{MIPS} = \text{Instruction Count} / (\text{Execution Time} \times 10^6)$$

Alternatively:

$$\text{MIPS} = \text{Clock Rate} / (\text{CPI} \times 10^6)$$

Where:

- Clock Rate = CPU frequency (Hz)
- CPI (Cycles Per Instruction) = average cycles per instruction

Interpretation of MIPS

- Higher MIPS → faster instruction execution
- Lower MIPS → slower performance

However, MIPS does NOT always reflect true performance, because different instructions have different complexities.

Example Calculation

Given:

- Instruction Count = 500 million
- Execution Time = 2 seconds

$$\text{MIPS} = 500 \times 10^6 / 2 \times 10^6 = 250 \text{ MIPS}$$

Advantages of MIPS

- Simple to calculate and understand
- Useful for comparing similar processors
- Provides quick performance estimation

Limitations of MIPS

MIPS has several critical drawbacks:

1. Instruction Set Dependency
 - Different architectures (e.g., RISC vs CISC) execute different numbers of instructions
 - A machine with fewer but more complex instructions may appear slower
2. Ignores Instruction Complexity
 - Not all instructions take the same time
 - Example: multiplication vs addition
3. Misleading Comparisons
 - A higher MIPS rating does not always mean better performance
4. Not Suitable for Real Applications
 - Real-world programs involve memory access, I/O, and branching

Benchmarking

Benchmarking is the process of evaluating a computer system's performance by running standardised programs (benchmarks) and measuring execution time or throughput.

Types of Benchmarks

1. Synthetic Benchmarks

- Artificial programs designed to test specific components
- Example: CPU arithmetic tests

Advantages:

- Easy to control
- Repeatable

Disadvantages:

- May not represent real workloads

2. Application Benchmarks

- Real-world programs
- Example:
 - Compilers
 - Web servers

- Scientific simulations

Advantages:

- Realistic performance measurement

Disadvantages:

- Complex to analyse

3. Kernel Benchmarks

- Focus on critical parts of applications
- Example: matrix multiplication

Popular Benchmark Suites

1. SPEC Benchmarks

- Industry-standard benchmarks
- Examples:
 - SPEC CPU – CPU performance
 - SPECint – integer performance
 - SPECfp – floating-point performance

2. TPC Benchmarks

- Used for database and transaction systems

3. LINPACK

- Measures floating-point computing power
- Used in ranking supercomputers

Performance Metrics in Benchmarking

Execution Time (Response Time): Time taken to complete a task

Throughput: Number of tasks completed per unit time

Speedup: $\text{Speedup} = \text{Old Execution Time} / \text{New Execution Time}$

Example of Benchmarking

Scenario:

- Program runs in 10 seconds on Machine A
- Program runs in 5 seconds on Machine B
- $\text{Speedup} = 10/5 = 2$

Interpretation: Machine B is 2× faster than Machine A.

Performance Evaluation Considerations

When evaluating performance, consider:

1. Workload Characteristics
 - Scientific vs business applications
2. System Components
 - CPU
 - Memory
 - I/O
3. Compiler Optimisation
 - Affects instruction count and execution time
4. Power and Cost
 - Performance is not the only metric

Real-Life Scenario

A company compares two processors:

- Processor A: 300 MIPS
- Processor B: 200 MIPS

However, benchmarking shows:

- Processor B completes real applications faster

Conclusion: Processor B is actually better despite lower MIPS.

2.2.5 Amdahl's law

In computer architecture, improving system performance is a major objective. However, optimising only a portion of a system does not always yield proportional improvements in overall performance. This limitation is captured by Amdahl's Law, which provides a mathematical framework for understanding the maximum possible speedup when only part of a system is enhanced.

Amdahl's Law is especially important in:

- Parallel processing
- Multi-core system design
- Performance optimization

Amdahl's Law states:

The overall performance improvement of a system is limited by the fraction of time the improved component is actually used.

Mathematical Expression

$$S = 1 / [(1 - P) + (P/k)]$$

Where:

- S = Overall speedup
- P = Fraction of the program that can be improved (parallelised)
- 1 - P = Fraction that cannot be improved (serial portion)
- k = Speedup factor of the improved portion

Key Concepts

1. Serial Portion

- Part of the program that cannot be parallelised
- Acts as a bottleneck
- Limits maximum achievable speedup

2. Parallel Portion

- Part that can be improved or parallelised
- Benefits from multiple processors or enhancements

3. Speedup Limit

Even with infinite processors:

$$S_{\max} = 1 / 1 - P$$

This means:

- If 10% of a program is serial → maximum speedup = 10×
- If 1% is serial → maximum speedup = 100×

Worked Examples**Example 1:** Basic Speedup

Given:

- 70% of the program can be parallelised (P=0.7)
- Speedup factor k=4

$$S = 1 / ((1 - 0.7) + 0.7/4)$$

$$= 1/(0.3+0.175) = 1/0.475$$

$$\approx 2.115$$

Interpretation:

Even with 4× improvement, total speedup is only 2.11×.

Example 2: Infinite Processors

Given:

$$P = 0.9$$

$$S_{\max} = 1/(1-0.9)$$

$$= 1/0.1 = 10$$

Interpretation:

Maximum speedup is 10×, no matter how many processors are used.

Graphical Insight (Conceptual)

As the number of processors increases:

- Speedup increases initially
- Eventually plateaus due to serial portion

This shows diminishing returns in parallel systems.

Real-Life Scenario

Consider a web server:

- 80% of workload can be parallelized
- 20% is serial (e.g., database locking)

Even with powerful multi-core processors:

- The 20% serial portion limits total performance

Amdahl's Law in Multicore Systems

Modern CPUs (e.g., Intel, AMD):

- Use multiple cores to improve performance
- But:
 - Not all programs are parallelizable
 - Performance gain depends on PPP

Limitations of Amdahl's Law

1. Assumes Fixed Problem Size: does not account for increasing workload
2. Ignores Overheads:
 - Communication between processors
 - Synchronisation delays
3. Unrealistic Assumptions: perfect parallelism rarely exists

In-Text Questions

1. What happens when clock speed increases?
2. Do all instructions take the same number of clock cycles?
3. Can CPI be less than 1?
4. Why do memory instructions have higher CPI?
5. Can reducing CPI always improve performance?
6. Why is MIPS considered a misleading performance metric?
7. Why are benchmarks preferred over MIPS?
8. Why does performance improvement saturate in parallel systems?
9. What happens when $P=1$?
10. Why is Amdahl's Law important in system design?

In-Text Answers

1. More cycles are executed per second, potentially improving performance.
2. No, it depends on the instruction complexity and architecture.
3. Yes, in modern pipelined processors, multiple instructions may complete per cycle, resulting in $CPI < 1$.
4. Because accessing memory takes longer than register operations.
5. Yes, but overall performance also depends on clock speed and instruction count.
6. Because it ignores instruction complexity, varies across architectures, and does not reflect actual program execution time.

7. Because benchmarks evaluate actual program performance, including memory, I/O, and real workloads.
8. Because the serial portion of the program cannot be parallelized and becomes a bottleneck.
9. The program is fully parallelizable, and speedup can scale linearly with processors.
10. It helps designers focus on optimizing the most impactful parts of a system.

2.2.6 Conclusion

Performance metrics are fundamental to understanding and improving computer systems. While simple metrics like clock speed and MIPS provide insights, deeper analysis using CPI, benchmarking, and Amdahl's Law is necessary for accurate evaluation. These concepts guide system design, optimisation, and real-world decision-making.

Summary

- Performance is measured using execution time and efficiency.
- Clock speed and cycles determine processing rate.
- CPI measures instruction efficiency.
- MIPS provides a basic performance metric but has limitations.
- Benchmarking gives a realistic performance evaluation.
- Amdahl's Law shows limits of system improvement.

Self-Assessment Questions

1. What is clock speed?
2. Define CPI.
3. What is MIPS?
4. What is benchmarking?
5. What does Amdahl's Law show?

Unit Activities

1. Calculate execution time for:
Clock rate = 1.5 GHz
Cycles = 3 billion
2. Given:
Instruction count = 500
Cycles = 2000
Compute CPI.
3. Compare two processors using:
Clock speed
CPI
Execution time
4. Apply Amdahl's Law:
60% improved
Speedup factor = 3

Tutor-Marked Assignments

1. Explain the relationship between clock speed, CPI, and execution time with examples.
2. Compare MIPS and benchmarking as performance measures.
3. Solve performance problems involving CPI and execution time.
4. Apply Amdahl's Law to a real-world system improvement.
5. Discuss limitations of performance metrics in modern computing systems.
6. Design a benchmarking strategy for evaluating a new processor.

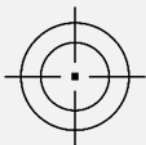
References For Further Reading

Hennessy, J. L., & Patterson, D. A. – Computer Architecture: A Quantitative Approach
Stallings, W. – Computer Organization and Architecture
Mano, M. M. – Computer System Architecture
Tanenbaum, A. S. – Structured Computer Organization
Patterson & Hennessy – Computer Organization and Design
SPEC Benchmark Organization – Standard Performance Evaluation Corporation

Unit 3 — Improving System Performance Through Pipelining

INTRODUCTION

There is a limit to the speed any hardware can attain, and the cost of faster circuits is quite high; we are left with the option of re-arranging the hardware with a view to improving the performance of the system. This unit is therefore concerned with one of such methods of re-arranging hardware in order to improve performance, known as pipelining. Various aspects of pipelining, including types, characteristics, hazards, advantages, and disadvantages of pipelining, are discussed among other topics.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the basic concepts of pipelining.
- Explain the various types of pipelining and their characteristics.
- Describe the design and execution of a pipeline supported by some practical examples.
- Describe with practical examples the various types of pipeline hazards and how to resolve them.

2.3.1 Overview of Pipelining

Have you ever visited an industrial plant and seen the assembly lines over there? How a product passes through the assembly line, and while passing, it is worked on, at different phases simultaneously. For example, take a car manufacturing plant. At the first stage, the automobile chassis is prepared, in the next stage, workers add the body to the chassis, further, the engine is installed, then painting work is done and so on.

The group of workers, after working on the chassis of the first car, doesn't sit idle. They start working on the chassis of the next car. And the next group take the chassis of the car and adds a body to it. The same thing is repeated at every stage after finishing the work on the current car body they take on the next car body, which is the output of the previous stage.

Here, though the first car is completed in several hours or days, due to the assembly line arrangement, it becomes possible to have a new car at the end of the assembly line in every clock cycle.

Similarly, the concept of pipelining works. The output of the first pipeline becomes the input for the next pipeline. It is like a set of data processing unit connected in series to utilize processor up to its maximum.

An instruction in a process is divided into 5 subtasks likely,

Instruction Fetch	Instruction Decode	Operand Fetch	Instruction Execute	Operand Store
----------------------	-----------------------	------------------	------------------------	------------------

- In the first subtask, the instruction is fetched.
- The fetched instruction is decoded in the second stage.
- In the third stage, the operands of the instruction are fetched.
- In the fourth, arithmetic and logical operation are performed on the operands to execute the instruction.
- In the fifth stage, the result is stored in memory.

Pipelining is a process of arrangement of hardware elements of the CPU such that its overall performance is increased. Simultaneous execution of more than one instruction takes place in a pipelined processor.

Let us see a real-life example that works on the concept of pipelined operation. Consider a water bottle packaging plant. Let there be 3 stages that a bottle should

pass through, Inserting the bottle(I), Filling water in the bottle(F), and Sealing the bottle(S). Let us consider these stages as stage 1, stage 2 and stage 3 respectively. Let each stage take 1 minute to complete its operation.

Now, in a non-pipelined operation, a bottle is first inserted in the plant, after 1 minute it is moved to stage 2 where water is filled. Now, in stage 1 nothing is happening. Similarly, when the bottle moves to stage 3, both stage 1 and stage 2 are idle. But in pipelined operation, when the bottle is in stage 2, another bottle can be loaded at stage 1. Similarly, when the bottle is in stage 3, there can be one bottle each in stage 1 and stage 2. So, after each minute, we get a new bottle at the end of stage 3. Hence, the average time taken to manufacture 1 bottle is:

Without pipelining = $9/3$ minutes = 3

```

I F S | | | | |
| | | I F S | | |
| | | | | | I F S (9 minutes)

```

With pipelining = $5/3$ minutes = 1.67m

```

I F S | |
| I F S |
| | I F S (5 minutes)

```

Types of Pipelining

In 1977, Handler and Ramamoorthy classified pipeline processors depending on their functionality.

Arithmetic Pipelining

It is designed to perform high-speed floating-point addition, multiplication and division. Here, the multiple arithmetic logic units are built in the system to perform the parallel arithmetic computation in various data formats. Examples of the arithmetic pipelined processor are Star-100, TI-ASC, Cray-1, and Cyber-205.

Arithmetic pipelines are usually found in most of the computers. They are used for floating point operations, multiplication of fixed-point numbers, etc. For example, The input to the Floating-Point Adder pipeline is:

$$X = A * 2^a$$

$$Y = B * 2^b$$

Here A and B are mantissas (significant digit of floating-point numbers), while a and b are exponents.

The floating-point addition and subtraction is done in 4 parts:

1. Compare the exponents.
2. Align the mantissas.
3. Add or subtract mantissas
4. Produce the result.

Registers are used for storing the intermediate results between the above operations.

Instruction Pipelining

Here, a number of instructions are pipelined and the execution of current instruction is overlapped by the execution of the subsequent instruction. It is also called instruction lookahead.

In this a stream of instructions can be executed by overlapping fetch, decode and execute phases of an instruction cycle. This type of technique is used to increase the throughput of the computer system.

An instruction pipeline reads instruction from the memory while previous instructions are being executed in other segments of the pipeline. Thus we can execute multiple instructions simultaneously. The pipeline will be more efficient if the instruction cycle is divided into segments of equal duration.

Processor Pipelining

Here, the processors are pipelined to process the same data stream. The data stream is processed by the first processor and the result is stored in the memory block. The result in the memory block is accessed by the second processor. The second processor reprocesses the result obtained by the first processor and passes the refined result to the third processor and so on.

Uni-function Vs. Multifunction Pipelining

The pipeline performing the precise function every time is uni-functional pipeline. On the other hand, the pipeline performing multiple functions at a different time or multiple functions at the same time is multifunction pipeline.

Static vs Dynamic Pipelining

The static pipeline performs a fixed-function each time. The static pipeline is uni-functional. The static pipeline executes the same type of instructions continuously. Frequent change in the type of instruction may vary the performance of the pipelining.

Dynamic pipeline performs several functions simultaneously. It is a multifunction pipelining.

Scalar vs Vector Pipelining

Scalar pipelining processes the instructions with scalar operands. The vector pipeline processes the instruction with vector operands.

Pipeline Characteristics

1. Throughput: Number of items (cars, instructions, operations) that exit the pipeline per unit time. Example: 1 inst /clock cycle, 10 cars/ hour, 10 floating point operations /cycle.
2. Stage time: The pipeline designer's goal is to balance the length of each pipeline stage (balanced pipeline). In general,
3. Stage time = Time per instruction on non-pipelined machine/ number of stages.
In many instances, stage time = max (times for all stages).
4. CPI : Pipeline yields a reduction in cycles per instruction
 - CPI approx = stage time.

2.3.2 Pipeline Stages

RISC processor has 5-stage instruction pipeline to execute all the instructions in the RISC instruction set. Following are the 5 stages of RISC pipeline with their respective operations:

1. Stage 1 (Instruction Fetch)

In this stage the CPU reads instructions from the address in the memory whose value is present in the program counter.

2. Stage 2 (Instruction Decode)

In this stage, instruction is decoded and the register file is accessed to get the values from the registers used in the instruction.

3. Stage 3 (Instruction Execute)

In this stage, ALU operations are performed.

4. Stage 4 (Memory Access)

In this stage, memory operands are read and written from/to the memory that is present in the instruction.

5. Stage 5 (Write Back)

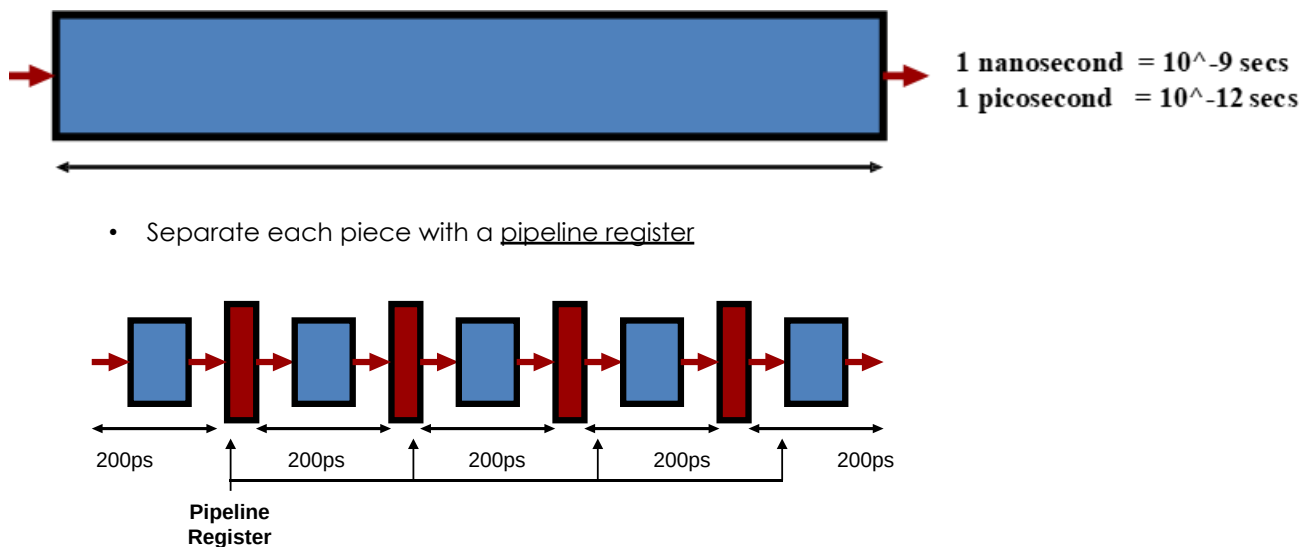
In this stage, computed/fetched value is written back to the register present in the instructions.

2.3.3 Design of a basic pipeline

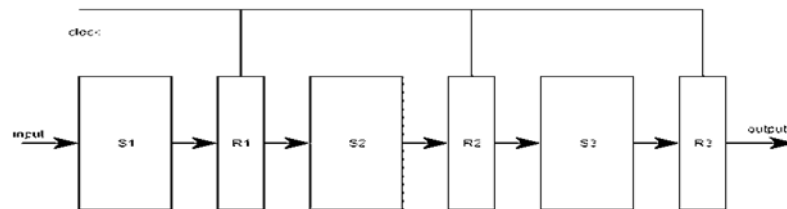
In a pipelined processor, a pipeline has two ends, the input end and the output end. Between these ends, there are multiple stages/segments such that output of one stage is connected to input of next stage and each stage performs a specific operation.

Interface registers are used to hold the intermediate output between two stages. These interface registers are also called latch or buffer. All the stages in the pipeline along with the interface registers are controlled by a common clock.

Key idea: break big computation up into pieces



In pipeline system, each segment consists of an input register followed by a combinational circuit. The register is used to hold data and combinational circuit performs operations on it. The output of combinational circuit is applied to the input register of the next segment.



Pipeline system is like the modern-day assembly line setup in factories. For example, in a car manufacturing industry, huge assembly lines are setup and at each point, there are robotic arms to perform a certain task, and then the car moves on ahead to the next arm.

Execution in a pipelined processor

Execution sequence of instructions in a pipelined processor can be visualized using a space-time diagram. For example, consider a processor having 4 stages and let there be 2 instructions to be executed. We can visualize the execution sequence through the following space-time diagrams:

Non-overlapped execution:

Stage / Cycle	1	2	3	4	5	6	7	8
S1	I ₁				I ₂			
S2		I ₁				I ₂		
S3			I ₁				I ₂	
S4				I ₁				I ₂

Total time = 8 Cycle

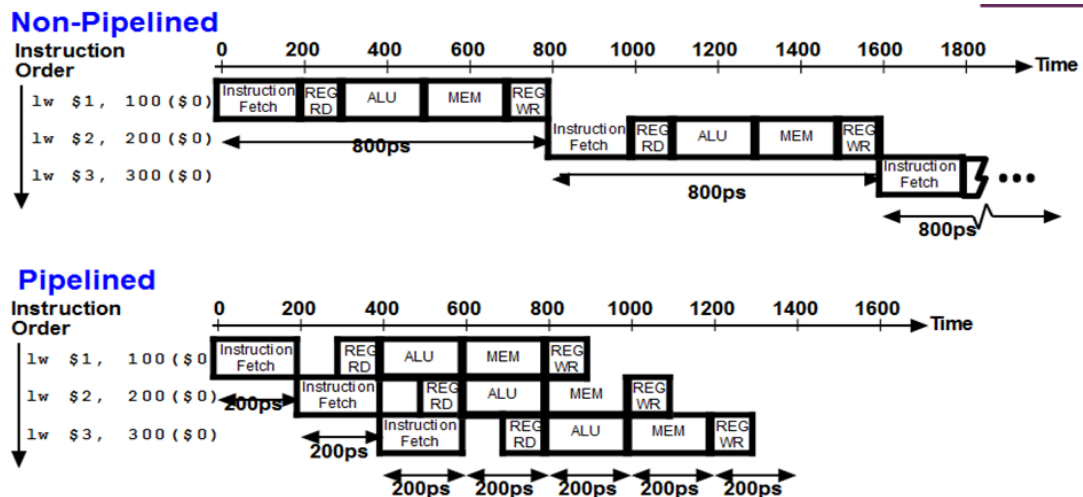
Overlapped execution:

Stage / Cycle	1	2	3	4	5
S1	I ₁	I ₂			
S2		I ₁	I ₂		
S3			I ₁	I ₂	
S4				I ₁	I ₂

Total time = 5 Cycle

Sample Solved Examples on Pipelines

Example 1:



Example 2

- 4-stage pipeline
- sub operation in each stage; $t_p = 20\text{ns}$
- 100 tasks to be executed
- 1 task in non-pipelined system; $20 \times 4 = 80\text{ns}$

Pipelined System

$$(k + (n - 1)) \cdot t_p = (4 + 99) \cdot 20 = 2060\text{ns}$$

Non-Pipelined System

$$n \cdot t_p = 100 \cdot 80 = 8000\text{ns}$$

Speedup

$$S_k = 8000 / 2060 = 3.88$$

4-Stage Pipeline is basically identical to the system with 4 identical function units

Example 3

- General: 40% ALU, 20% branch, 40% memory
- Design 1: Non-pipelined. 10ns clock cycles. ALU operations and branches take 4 cycles, memory operations take 5 cycles. In other words, ALU operations and branches take $4 \times 10 = 40\text{ns}$
- Design 2: Pipelined. Clock skew and Setup add 1 ns overhead to clock cycle.
- What is the speedup?

Solution

Design 1:

Average instruction execution time = clock cycletime *CPI

$$= 10\text{ns} * (4 * 0.4 + 4 * 0.2 + 5 * 0.4) = 10 * (1.6 + 0.8 + 2.0)$$

$$= 44\text{ns}$$

Design 2:

Average instruction time at steady state is clock cycle time:

$$= 10\text{ns} + 1\text{ns (for setup and clock skew)} = 11\text{ns}$$

$$\text{Speed up} = 44/11 = 4$$

Example 4

Assume times for each functional unit of a pipeline to be: 10ns, 8ns, 10ns, 10ns and 7ns.

Overhead 1ns per stage. Compute the speed of the data path.

Pipelined: Stage time = MAX(10,8,10,10,7) + overhead

$$= 10 + 1 = 11\text{ns.}$$

This is the average instruction execution time at steady state.

Non-pipelined: 10+8+10+10+7 = 45ns

$$\text{Speedup} = 45/11 = 4.1 \text{ times}$$

2.3.4 Pipeline Hazards/Limitations

Hazards reduce the performance from the ideal speedup gained by pipelines. Some other texts refer to hazards as dependencies, and there are mainly three types possible in a pipelined processor. These are:

- Structural Dependency
- Control Dependency
- Data Dependency

These dependencies may introduce stalls in the pipeline.

Stall: A stall is a cycle in the pipeline without new input.

Structural dependency

This dependency arises due to the resource conflict in the pipeline. A resource conflict is a situation in which more than one instruction tries to access the same resource in the same cycle. In other words, this is a situation in which hardware cannot support all possible combinations of instructions in simultaneous overlapped execution. A resource can be a register, memory, or ALU. Some examples are provided below.

Example 1:

Instruction / Cycle	1	2	3	4	5
I ₁	IF(Mem)	ID	EX	Mem	
I ₂		IF(Mem)	ID	EX	
I ₃			IF(Mem)	ID	EX
I ₄				IF(Mem)	ID

In the above scenario, in cycle 4, instructions I₁ and I₄ are trying to access the same resource (Memory), which introduces a resource conflict.

To avoid this problem, we have to keep the instruction on hold until the required resource (memory in our case) becomes available. This wait will introduce stalls in the pipeline as shown below:

Cycle	1	2	3	4	5	6	7	8
I ₁	IF(Mem)	ID	EX	Mem	WB			
I ₂		IF(Mem)	ID	EX	Mem	WB		
I ₃			IF(Mem)	ID	EX	Mem	WB	
I ₄				–	–	–	IF(Mem)	

Solution for structural dependency

To minimise structural dependency stalls in the pipeline, we use a hardware mechanism called Renaming.

Renaming: According to renaming, we divide the memory into two independent modules used to store the instruction and data separately, called Code memory (CM) and Data memory (DM), respectively. CM will contain all the instructions, and DM will contain all the operands that are required for the instructions.

Instruction/ Cycle	1	2	3	4	5	6	7
I ₁	IF(CM)	ID	EX	DM	WB		
I ₂		IF(CM)	ID	EX	DM	WB	
I ₃			IF(CM)	ID	EX	DM	WB
I ₄				IF(CM)	ID	EX	DM
I ₅					IF(CM)	ID	EX
I ₆						IF(CM)	ID
I ₇							IF(CM)

Control Dependency (Branch Hazards)

This type of dependency occurs during the transfer of control instructions such as BRANCH, CALL, JMP, etc. On many instruction architectures, the processor will not know the target address of these instructions when it needs to insert the new instruction into the pipeline. Due to this, unwanted instructions are fed to the pipeline.

Consider the following sequence of instructions in the program:

100: I1

101: I2 (JMP 250)

102: I3

250: BI1

Expected output: I1 -> I2 -> BI1

NOTE: Generally, the target address of the JMP instruction is known after ID stage only.

Instruction/ Cycle	1	2	3	4	5	6
I ₁	IF	ID	EX	MEM	WB	
I ₂		IF	ID (PC:250)	EX	Mem	WB
I ₃			IF	ID	EX	Mem
BI ₁				IF	ID	EX

Output Sequence: I1 -> I2 -> I3 -> BI1

So, the output sequence is not equal to the expected output, which means the pipeline is not implemented correctly.

To correct the above problem, we need to stop the Instruction fetch until we get the target address of the branch instruction. This can be implemented by introducing a delay slot until we get the target address.

Instruction/ Cycle	1	2	3	4	5	6
I ₁	IF	ID	EX	MEM	WB	
I ₂		IF	ID (PC:250)	EX	Mem	WB
Delay	–	–	–	–	–	–
BI ₁				IF	ID	EX

Output Sequence: I₁ -> I₂ -> Delay (Stall) -> BI₁

As the delay slot performs no operation, this output sequence is equal to the expected output sequence. But this slot introduces a stall in the pipeline.

Solution for Control dependency:

Branch Prediction is the method through which stalls due to control dependency can be eliminated. In this at 1st stage, a prediction is made about which branch will be taken. For branch prediction, the branch penalty is zero.

Branch penalty: The number of stalls introduced during the branch operations in the pipelined processor is known as the branch penalty.

NOTE: As we see that the target address is available after the ID stage, so the number of stalls introduced in the pipeline is 1. Suppose the branch target address were present after the ALU stage, there would have been 2 stalls. Generally, if the target address is present after the kth stage, then there will be (k – 1) stalls in the pipeline.

Total number of stalls introduced in the pipeline due to branch instructions = Branch frequency * Branch Penalty

Data Dependency (Data Hazard)

Let us consider an ADD instruction S, such that

S : ADD R1, R2, R3

Addresses read by S = I(S) = {R2, R3}

Addresses written by S = O(S) = {R1}

Now, we say that instruction S₂ depends in instruction S₁, when

This condition is called the Bernstein condition.

Three cases exist:

- Flow (data) dependence: $O(S_1) \cap I(S_2)$, $S_1 \rightarrow S_2$ and S₁ writes after something read by S₂
- Anti-dependence: $I(S_1) \cap O(S_2)$, $S_1 \rightarrow S_2$ and S₁ reads something before S₂ overwrites it

- Output dependence: $O(S1) \cap O(S2)$, $S1 \rightarrow S2$ and both write the same memory location.

Example 1:

Let there be two instructions, I1 and I2, such that:

I1 : ADD R1, R2, R3

I2 : SUB R4, R1, R2

When the above instructions are executed in a pipelined processor, then a data dependency condition will occur, which means that I2 tries to read the data before I1 writes it; therefore, I2 incorrectly gets the old value from I1.

Instruction / Cycle	1	2	3	4
I ₁	IF	ID	EX	DM
I ₂		IF	ID(Old value)	EX

To minimise data dependency stalls in the pipeline, **operand forwarding** is used.

Operand Forwarding: In operand forwarding, we use the interface registers present between the stages to hold intermediate output so that dependent instructions can access the new value from the interface register directly.

Considering the same example:

I1 : ADD R1, R2, R3

I2 : SUB R4, R1, R2

Instruction / Cycle	1	2	3	4
I ₁	IF	ID	EX	DM
I ₂		IF	ID	EX

Example 2 :

add \$s0, \$t0, \$t1

sub \$t2, \$s0, \$t3

In the example, the second instruction is dependent on the result

in \$s0 of the first instruction:

if \$s0 = -5 before add

\$s0 = 8 after add

then the value 8 should be used in the second instruction sub.

Draw the multiple clock cycle pipeline diagram for the execution:

Execution order Time	CC1	CC2	CC3	CC4	CC5	CC6
add \$s0, \$t0, \$t1	IF	----ID-----	EX-----	MEM---	WB	
value in \$s0	5	-5	-5	-5	-5/8	8
sub \$t2, \$s0, \$t3		IF-----	ID-----	EX-----	MEM---	WB

In which clock cycle add writes to \$s0? Clock 5

In which clock cycle sub reads \$s0? Clock 3

For sub instruction, the value in \$s0 has to be read in its ID stage (CC3).

However, the value in \$s0 in CC3 is still -5 and not the correct value

8. We can only have the correct value in \$s0 at the end of clock 5 (CC5).

The dependency goes from CC5 ---> CC3 (backward)

Data hazards occur when instructions that exhibit data dependence modify data in different stages of a pipeline. Hazards cause delays in the pipeline. There are mainly three types of data hazards:

RAW - Read After Write. Most common: solved by data forwarding (Flow/True data dependency)

WAW - Write After Write (Output data Dependency)

Inst i (load) before inst j (add). Both write to the same register. But inst i does it before inst j. DLX avoids this by waiting for WB to write to registers. So no WAW hazard in DLX.

WAR - Write after Read (Anti-Data dependency):

inst j tries to write a destination before it is read by I, so I incorrectly get its value. This cannot happen in DLX since all inst read early (ID) but write late (WB).

But WAW happens in complex instruction sets that have auto-increment mode and require operands to be read late cycle experience WAW.

Let there be two instructions I and J, such that J follow I. We can illustrate the three types of data dependencies as follows:

RAW hazard occurs when instruction J tries to read data before instruction I writes it.

Example:

I: $R2 \leftarrow R1 + R3$

J: $R4 \leftarrow R2 + R3$

WAR hazard occurs when instruction J tries to write data before instruction I reads it.

Eg:

I: $R2 \leftarrow R1 + R3$

J: $R3 \leftarrow R4 + R5$

WAW hazard occurs when instruction J tries to write output before instruction I writes it.

Eg:

I: $R2 \leftarrow R1 + R3$

J: $R2 \leftarrow R4 + R5$

WAR and WAW hazards occur during the out-of-order execution of the instructions.

2.3.5 Conclusion

This unit is concerned with the method of rearranging the hardware of the computer in order to improve performance in terms of throughput. This is known as pipelining. Various aspects of pipelining, including types, characteristics, hazards, advantages, and disadvantages of pipelining, are discussed among other topics.

Summary

In this unit, you have learnt various topics that relate to improving system performance through pipelining, which is a method of rearranging the hardware of the computer in order to improve throughput. Various aspects of pipelining, including types, characteristics, hazards, advantages, and disadvantages of pipelining, are discussed among other topics. You also had the opportunity to study numerous examples on the subject.

Self-Assessment Questions

1. What is pipelining in computer architecture?
2. List the five stages of a typical instruction pipeline.
3. What is pipeline speedup?
4. What is the ideal speedup of a k-stage pipeline?
5. Define pipeline latency.
6. What is a pipeline hazard?
7. List three types of pipeline hazards.
8. What is forwarding in pipelining?
9. What is a pipeline stall?
10. What is the formula for pipeline execution time?
11. Calculate cycles for 8 instructions in a 4-stage pipeline.
12. What is pipeline throughput?
13. Differentiate between the instruction pipeline and the arithmetic pipeline.
14. What causes pipeline inefficiency?
15. Calculate speedup for:
 - Non-pipelined = 40 cycles
 - Pipelined = 10 cycles

Unit Activities

Activity 1: Pipeline Stage Identification

Given the instruction sequence below, map each instruction to the 5 pipeline stages:

Instructions:

I1: LOAD R1, 0(R2)

I2: ADD R3, R1, R4

I3: SUB R5, R3, R6

Stages:

- IF (Instruction Fetch)
- ID (Instruction Decode)
- EX (Execute)
- MEM (Memory Access)
- WB (Write Back)

Activity 2: Pipeline Speedup Calculation

A non-pipelined processor takes 5 cycles per instruction. A pipelined processor has 5 stages. Calculate the time taken to execute 10 instructions:

- Without pipelining
- With pipelining

Expected Solution:

- Non-pipelined = $10 \times 5 = 50$ cycles
- Pipelined = $5 + (10 - 1) = 14$ cycles

Activity 3: Identifying Pipeline Hazards

Analyse the following instructions:

I1: LOAD R1, 0(R2)

I2: ADD R3, R1, R4

Questions:

- What type of hazard occurs?
- Suggest a solution

Expected Answer:

- Data hazard (RAW – Read After Write)
- Solution: Forwarding or pipeline stall

Activity 4: Hazard Classification

Classify the following hazards:

Scenario	Hazard Type
Two instructions need the same ALU	?
Branch instruction changes the flow	?
Instruction depends on the previous result	?

Expected Answers:

- Structural hazard
- Control hazard
- Data hazard

Activity 5: Pipeline Efficiency Calculation

A pipeline has 6 stages and executes 20 instructions.

Calculate:

- Total cycles
- Efficiency

Solution Guide:

- Total cycles = $k + (n - 1) = 6 + 19 = 25$
- Efficiency = $n / \text{total cycles} = 20 / 25 = 0.8$ (80%)

Tutor-Marked Assignment

1. A pipelined processor has a clock rate of 2.5 GHz and executes a program with 1.5 million instructions. The pipeline has five stages, and instructions are issued at a rate of one per clock cycle. Ignore penalties due to branch instructions and out-of-sequence executions.
 - What is the speedup of this processor for this program compared to a
 - nonpipelined processor, making the same assumptions used in Section 14.4?
 - What is the throughput (in MIPS) of the pipelined processor?
2. A nonpipelined processor has a clock rate of 2.5 GHz and an average CPI (cycles per instruction) of 4. An upgrade to the processor introduces a five-

stage pipeline. However, due to internal pipeline delays, such as latch delay, the clock rate of the New processor has to be reduced to 2 GHz.

- What is the speedup achieved for a typical program?
- What is the MIPS rate for each processor?

References/Further Readings

Computer Organisation and Architecture: Designing for Performance by William Stallings, Tenth edition (2016). Read all sections in Chapter 14

http://home.ustc.edu.cn/~leedsong/reference_books_tools/Computer%20Organization%20and%20Architecture%2010th%20-%20William%20Stallings.pdf

Other internet resources may also be used.

Module Three

Computer Memory Organisation

Unit 1 — Computer Memory Overview

INTRODUCTION

Memory unit is essential component of digital computer since it is needed for storing programs and data. Memory unit that communicates directly with CPU is called Main memory. Devices that provide backup storage are called auxiliary memory. Only programs and data currently needed by processor reside in the main memory. All other information is stored in auxiliary memory and transferred to main memory when needed.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the concept of data representation in a digital computer
- List and describe the units of representation in a computer

3.1.1 Computer Memory Overview

The memory unit is an essential component of a digital computer since it is needed for storing programs and data. A memory unit that communicates directly with the CPU is called the main memory. Devices that provide backup storage are called auxiliary memory. Only programs and data currently needed by the processor reside in the main memory. All other information is stored in auxiliary memory and transferred to main memory when needed. In order to have a better understanding of computer memory, two aspects are discussed further, namely: characteristics and hierarchy of computer memory.

Characteristics of Computer Memory

Although seemingly simple in concept, computer memory exhibits perhaps the widest range of types, technologies, organisations, performances, and costs of any feature of a computer system. No single technology is optimal in satisfying the memory requirements for a computer system.

The complex subject of computer memory is made more manageable if we classify memory systems according to their key characteristics. The most important of these are listed below.

Location	Performance
Internal (e.g., processor registers, cache, main memory)	Access time
External (e.g., optical disks, magnetic disks, tapes)	Cycle time
Capacity	Transfer rate
Number of words	Physical Type
Number of bytes	Semiconductor
Unit of Transfer	Magnetic
Word	Optical
Block	Magneto-optical
Access Method	Physical Characteristics
Sequential	Volatile/nonvolatile
Direct	Erasable/nonerasable
Random	Organization
Associative	Memory modules

Location

The term location in the table above refers to whether memory is internal or external to the computer.

- **Internal memory** is often equated with main memory, but there are other forms of internal memory. The processor requires its own local memory, in the form of registers (e.g., see Figure 2.3). Further, as we will see, the control unit portion of the processor may also require its own internal memory. Cache is another form of internal memory.
- **External memory** consists of peripheral storage devices, such as disk and tape, that are accessible to the processor via I/O controllers.

Capacity

An obvious characteristic of memory is its capacity. For internal memory, this is typically expressed in terms of bytes (1 byte = 8 bits) or words. Common word lengths are 8, 16, and 32 bits. External memory capacity is typically expressed in terms of bytes.

A related concept is the unit of transfer. For internal memory, the unit of transfer is equal to the number of electrical lines into and out of the memory module. This may be equal to the word length, but is often larger, such as 64, 128, or 256 bits. To clarify this point, consider three related concepts for internal memory:

- **Word Size:** The “natural” unit of organisation of memory.
The size of a word is typically equal to the number of bits used to represent an integer and to the instruction length. Unfortunately, there are many exceptions. For example, the CRAY C90 (an older model CRAY supercomputer) has a 64-bit word length but uses a 46-bit integer representation. The Intel x86 architecture has a wide variety of instruction lengths, expressed as multiples of bytes, and a word size of 32 bits.
- **Addressable units:** In some systems, the addressable unit is the word. However, many systems allow addressing at the byte level. In any case, the relationship between the length in bits A of an address and the number N of addressable units is $2^A = N$.
- **Unit of transfer:** For main memory, this is the number of bits read out of or written into memory at a time. The unit of transfer need not equal a word or an addressable unit. For external memory, data are often transferred in much larger units than a word, and these are referred to as blocks.

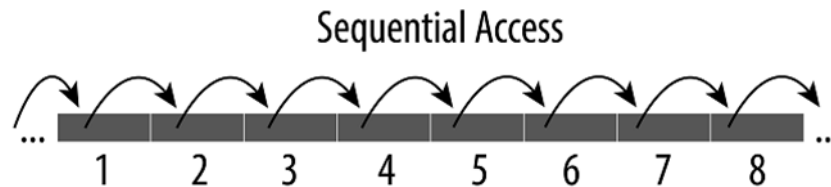
Access Methods

There are four main methods of accessing units of data based on the hardware implementation of the storage devices. They are discussed as follows:

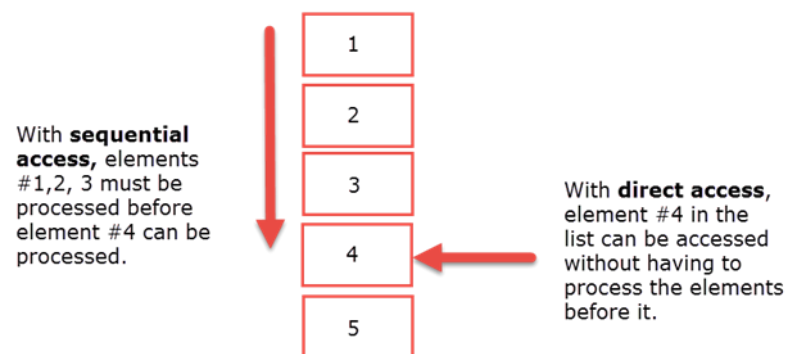
- **Sequential access:** Memory is organised into units of data, called records. Access must be made in a specific linear sequence. Stored addressing

information is used to separate records and assist in the retrieval process. A shared read–write mechanism is used, and this must be moved from its current location to the desired location, passing and rejecting each intermediate record.

Thus, the time to access an arbitrary record is highly variable. An example is Tape.



- **Direct access:** As with sequential access, direct access involves a shared read–write mechanism. However, individual blocks or records have a unique address based on physical location. Access is accomplished by direct access to reach a general vicinity, plus sequential searching, counting, or waiting to reach the final location. Again, access time is variable. An example is a hard disk drive.



- **Random access:** Each addressable location in memory has a unique, physically wired-in addressing mechanism. The time to access a given location is independent of the sequence of prior accesses and is constant. Thus, any location can be selected at random and directly addressed and accessed. Main memory and some cache systems are random-access.



- **Associative:** This is a random-access type of memory that enables one to make a comparison of desired bit locations within a word for a specified match, and to do this for all words simultaneously. Thus, a word is retrieved based on a portion of its contents rather than its address. As with ordinary

random-access memory, each location has its own addressing mechanism, and retrieval time is constant independent of location or prior access patterns. Cache memories may employ associative access.

Performance

From a user's point of view, the two most important characteristics of memory are capacity and performance. Three performance parameters are used:

- **Access time (latency):** For random-access memory, this is the time it takes to perform a read or write operation, that is, the time from the instant that an address is presented to the memory to the instant that data has been stored or made available for use.

For non-random-access memory, access time is the time it takes to position the read–write mechanism at the desired location.

- **Memory cycle time:** This concept is primarily applied to random-access memory and consists of the access time plus any additional time required before a second access can commence. This additional time may be required for transients to die out on signal lines or to regenerate data if they are read destructively. Note that memory cycle time is concerned with the system bus, not the processor.

Cycle time = access time + recovery time

- **Transfer rate:** This is the rate at which data can be transferred into or out of a memory unit. For random-access memory, it is equal to $1/(\text{cycle time})$. For non-random-access memory, the following relationship holds:

$$T_n = TA + N/R$$

where

T_n = Average time to read or write n bits

TA = Average access time

N = Number of bits

R = Transfer rate, in bits per second (bps)

Physical Types

A variety of physical types of memory have been employed. The most common today are semiconductor memory, magnetic surface memory, used for disk and tape, and optical and magneto-optical. Below are examples of memory devices found in each category.

- Semiconductor – RAM
- Magnetic – Disk & Tape

- Optical – CD & DVD
- Others

Bubble (old) – memory that made a "bubble" of charge in an opposite direction to that of the thin magnetic material on which it was mounted

Hologram (new) – much like the hologram on your credit card, laser beams are used to store computer-generated data in three dimensions. (10 times faster with 12 times the density)

Physical Characteristics

Several physical characteristics of data storage are important.

- In a volatile memory, information decays naturally or is lost when electrical power is switched off.
- In a nonvolatile memory, information once recorded remains without deterioration until deliberately changed; no electrical power is needed to retain information.
 - Magnetic-surface memories are nonvolatile.
 - Semiconductor memory (memory on integrated circuits) may be either volatile or nonvolatile.

Nonerasable memory cannot be altered, except by destroying the storage unit. Semiconductor memory of this type is known as read-only memory (ROM). Of necessity, a practical non-erasable memory must also be nonvolatile.

Organization

For random-access memory, the organisation is a key design issue. In this context, organisation refers to the physical arrangement of bits to form words. The obvious arrangement is not always used, as is explained in Chapter 5.

3.1.2 Memory Hierarchy

The design constraints on a computer's memory can be summed up by three questions:

- How much?
- How fast?
- How expensive?

The question of how much is somewhat open-ended. If the capacity is there, applications will likely be developed to use it. The question of how fast it is, in a sense, is easier to answer. To achieve the greatest performance, the memory must be able to

keep up with the processor. That is, as the processor is executing instructions, we would not want it to have to pause waiting for instructions or operands. The final question must also be considered. For a practical system, the cost of memory must be reasonable in relation to other components.

As might be expected, there is a trade-off among the three key characteristics of memory: capacity, access time, and cost. A variety of technologies are used to implement memory systems, and across this spectrum of technologies, the following relationships hold:

- Faster access time, greater cost per bit;
- Greater capacity, smaller cost per bit;
- Greater capacity, slower access time.

The dilemma facing the designer is clear. The designer would like to use memory technologies that provide for large-capacity memory, both because the capacity is needed and because the cost per bit is low. However, to meet performance requirements, the designer needs to use expensive, relatively lower-capacity memories with short access times.

The way out of this dilemma is not to rely on a single memory component or technology, but to employ a memory hierarchy. A typical hierarchy is illustrated in the figure below. As one goes down the hierarchy, the following occur:

- Decreasing cost per bit;
- Increasing capacity;
- Increasing access time;
- Decreasing frequency of access to the memory by the processor.

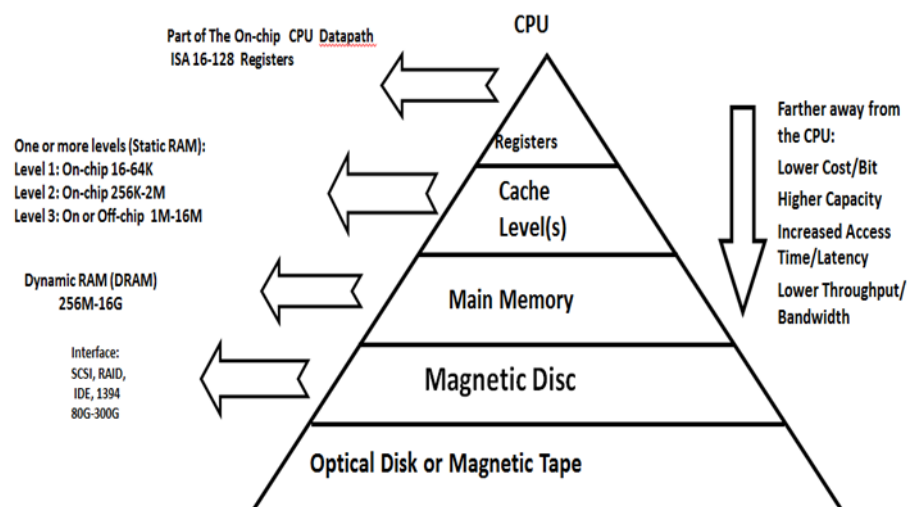


Figure 18: Memory Hierarchy

Thus, smaller, more expensive, faster memories are supplemented by larger, cheaper, slower memories. The key to the success of this organization is item (d): decreasing frequency of access.

The use of two levels of memory to reduce average access time works in principle, but only if conditions (a) through (d) apply. By employing a variety of technologies, a spectrum of memory systems exists that satisfies conditions (a) through (c). Fortunately, condition (d) is also generally valid.

The basis for the validity of condition (d) is a principle known as locality of reference. During the course of execution of a program, memory references by the processor, for both instructions and data, tend to cluster. Programs typically contain a number of iterative loops and subroutines. Once a loop or subroutine is entered, there are repeated references to a small set of instructions. Similarly, operations on tables and arrays involve access to a clustered set of data words.

Over a long period of time, the clusters in use change, but over a short period of time, the processor is primarily working with fixed clusters of memory references.

3.1.3 Conclusion

Computer memory is responsible for storage of data. It is also by far, the only component of the computer that exhibits quite a number of characteristics which have been studied in this unit. It is also instructive to note that there is a system of hierarchy in computer memory for which the smallest and closest to processor happens to be registers followed by cache memory then main memory. One of the reasons for the hierarchy is to enhance performance of the computer system.

Summary

In this unit, you learnt the basic concepts and different aspects of the computer memory. Most of the unit dwelt on the characteristics of computer memory such as location, physical type, size, performance among others. The unit concluded with a discussion on computer memory hierarchy.

Self-Assessment Questions

1. What is computer memory?
2. List four key characteristics of computer memory.

3. Differentiate between primary and secondary memory.
4. Why is memory hierarchy important in computer systems?
5. What is the relationship between memory speed, cost, and size?
6. Explain the role of cache memory in the memory hierarchy.

Unit Activities

Activity 1: Memory Classification Exercise

Divide students into groups and assign each group a type of computer memory (e.g., RAM, ROM, cache, secondary storage).

Each group should:

- Describe the memory type
- Identify its characteristics (location, speed, size, volatility)
- Present real-life examples (e.g., smartphone memory, hard disk)

Outcome: Students understand different memory types and their roles in a system.

Activity 2: Performance Comparison Task

Provide students with data on access time, cost, and size of different memory types.

Ask them to:

- Compare at least three memory types
- Explain why faster memory is smaller and more expensive

Outcome: Students understand trade-offs in memory design.

Activity 3: Real-World Scenario Analysis

Present a scenario:

“A computer is slow when opening applications but runs fast afterward.”

Students should:

- Identify which memory components are involved
- Explain the role of cache and RAM in improving performance

Outcome: Students apply theoretical knowledge to practical situations.

Tutor-Marked Assignment

1. What are the differences among sequential access, direct access, and random access?
2. What is the general relationship among access time, memory cost, and capacity?
3. How does the principle of locality relate to the use of multiple memory levels?

References/Further Readings

Computer Organisation and Architecture: Designing for Performance by William Stallings, Tenth edition (2016). Read all sections in Chapter 4

http://home.ustc.edu.cn/~leedsong/reference_books_tools/Computer%20Organization%20and%20Architecture%2010th%20-%20William%20Stallings.pdf

Computer Organisation and Design: The Hardware/Software Interface: RISC-V Edition - Read all sections in Chapter 3

Other internet resources may also be used.

Unit 2 — Semiconductor Memory

INTRODUCTION

In earlier computers, the most common form of random-access storage for computer main memory employed an array of doughnut-shaped ferromagnetic loops referred to as cores. Hence, main memory was often referred to as core, a term that persists to this day. The advent of, and advantages of, microelectronics have long since vanquished the magnetic core memory. Today, the use of semiconductor chips for main memory is almost universal. In this unit, key aspects of semiconductor technology are explored.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand basic concept of Semiconductor Memory
- Describe the forms and types of Semiconductor memory
- Discuss the classification of read-only memory (ROM)

3.2.1 Semiconductor Memory

The basic element of a semiconductor memory is the memory cell. Although a variety of electronic technologies are used, all semiconductor memory cells share certain properties:

- They exhibit two stable (or semistable) states, which can be used to represent binary 1 and 0.
- They are capable of being written into (at least once), to set the state.
- They are capable of being read to sense the state.

Most commonly, the cell has three functional terminals capable of carrying an electrical signal. The select terminal, as the name suggests, selects a memory cell for a read or write operation. The control terminal indicates read or write. For writing, the other terminal provides an electrical signal that sets the state of the cell to 1 or 0. For reading, that terminal is used for the output of the cell's state. The details of the internal organisation, functioning, and timing of the memory cell depend on the specific integrated circuit technology used.

Forms of Semiconductor Memory

All of the memory types that we will explore in this chapter are random-access. That is, individual words of memory are directly accessed through wired-in addressing logic.

The table below lists the major types of semiconductor memory. The most common is referred to as random-access memory (RAM). This is, in fact, a misuse of the term, because all of the types listed in the table are random access. One distinguishing characteristic of memory that is designated as RAM is that it is possible to both read data from the memory and write new data into the memory easily and rapidly.

Memory Type	Category	Erase	Write Mechanism	Volatility
Random-access memory (RAM)	Read-write memory	Electrically, byte-level	Electrically	Volatile
Read-only memory (ROM)	Read-only memory	Not possible	Masks	Nonvolatile
Programmable ROM (PROM)			Electrically	
Erasable PROM (EPROM)	UV light, chip-level			
Electrically Erasable PROM (EEPROM)	Electrically, byte-level			
Flash memory	Electrically, block-level			

Both the reading and writing are accomplished through the use of electrical signals. The other distinguishing characteristic of traditional RAM is that it is volatile. A RAM must be provided with a constant power supply. If the power is interrupted, then the data are lost. Thus, RAM can be used only as temporary storage. The two traditional forms of RAM used in computers are DRAM and SRAM. Newer forms of RAM, discussed in a separate section, are nonvolatile.

Forms of Semiconductor Memory - Dynamic RAM (DRAM)

RAM technology is divided into two technologies: dynamic and static. A dynamic RAM (DRAM) is made of cells that store data as charge on capacitors. The presence or absence of charge in a capacitor is interpreted as a binary 1 or 0. Because capacitors have a natural tendency to discharge, dynamic RAMs require periodic charge refreshing to maintain data storage. The term dynamic refers to this tendency of the stored charge to leak away, even with power continuously applied.

Although the DRAM cell is used to store a single bit (0 or 1), it is essentially an analogue device. The capacitor can store any charge value within a range; a threshold value determines whether the charge is interpreted as 1 or 0.

Forms of Semiconductor Memory - Static RAM (SRAM)

A static RAM (SRAM) is a digital device that uses the same logic elements as the processor. In a SRAM, binary values are stored using traditional flip-flop logic-gate configurations. A static RAM will hold its data as long as power is supplied to it.

As in the DRAM, the SRAM address line is used to open or close a switch.

SRAM vs DRAM (Comparison)

The table below provides a summary of the comparison of the two types of RAM

SRAM	DRAM
Uses capacitor for storing information	Uses Flip flop
More cells per unit area due to smaller cell size.	Needs more space for same capacity
Cheap and smaller in size	Expensive and bigger in size
Slower and analog device	Faster and digital device
Requires refresh circuit	No need
Used in main memory	Used in cache

Types of ROM

As the name suggests, a read-only memory (ROM) contains a permanent pattern of data that cannot be changed. A ROM is nonvolatile; that is, no power source is required to maintain the bit values in memory. While it is possible to read a ROM, it is not possible to write new data into it. An important application of ROMs is micro-programming; other potential applications include:

- Library subroutines for frequently wanted functions
- System programs
- Function tables

For a modest-sized requirement, the advantage of ROM is that the data or program is permanently in main memory and never needs to be loaded from a secondary storage device. A ROM is created like any other integrated circuit chip, with the data actually wired into the chip as part of the fabrication process. This presents two problems:

- The data insertion step includes a relatively large fixed cost, whether one or thousands of copies of a particular ROM are fabricated.
- There is no room for error. If one bit is wrong, the whole batch of ROMs must be thrown out.

When only a small number of ROMs with particular memory content is needed, a less expensive alternative is the **programmable ROM (PROM)**. Like the ROM, the PROM is **nonvolatile** and may be written into only once. For the PROM, the writing process is performed electrically and may be performed by a supplier or customer at a time later than the original chip fabrication. Special equipment is required for the writing or “programming” process. PROMs provide flexibility and convenience. The ROM remains attractive for high-volume production runs. Another variation on **read-only memory** is the read-mostly memory, which is useful for applications in which read operations are far more frequent than write operations but for which nonvolatile storage is required. There are three common forms of read-mostly memory: EPROM, EEPROM, and flash memory.

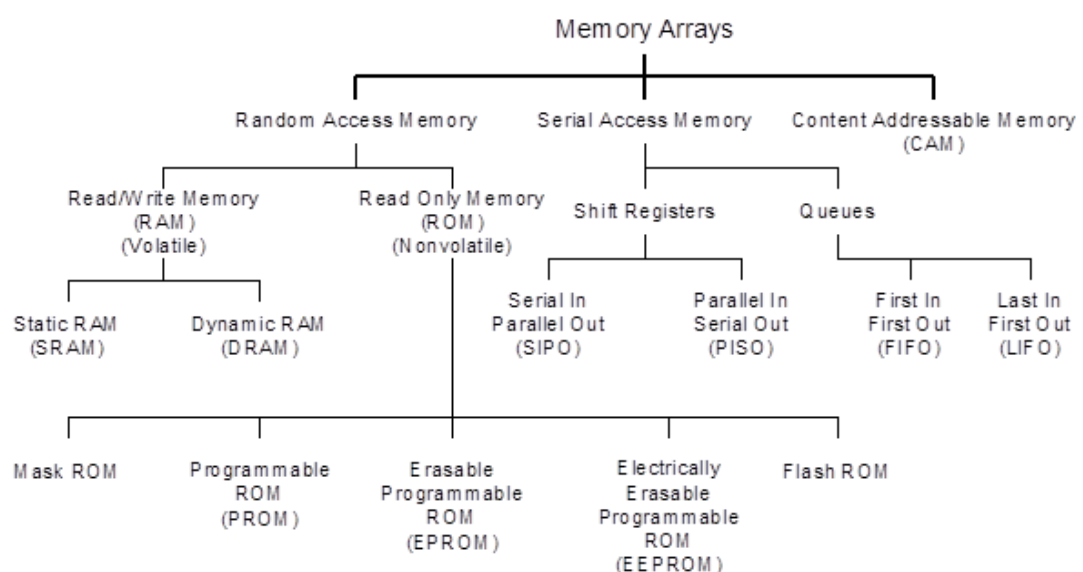
The optically **erasable programmable read-only memory (EPROM)** is read and written electrically, as with PROM. However, before a write operation, all the storage cells must be erased to the same initial state by exposure of the packaged chip to ultraviolet radiation. Erasure is performed by shining an intense ultraviolet light through a window that is designed into the memory chip. This erasure process can be performed repeatedly; each erasure can take as much as 20 minutes to perform. Thus, the EPROM can be altered multiple times and, like the ROM and PROM, holds

its data virtually indefinitely. For comparable amounts of storage, the EPROM is more expensive than PROM, but it has the advantage of multiple update capability.

A more attractive form of read-mostly memory is **electrically erasable programmable read-only memory (EEPROM)**. This is a read-mostly memory that can be written into at any time without erasing prior contents; only the byte or bytes addressed are updated. The write operation takes considerably longer than the read operation, on the order of several hundred microseconds per byte. The EEPROM combines the advantage of non-volatility with the flexibility of being updatable in place, using ordinary bus control, address, and data lines. EEPROM is more expensive than EPROM and is also less dense, supporting fewer bits per chip.

Another form of semiconductor memory is flash memory (so named because of the speed with which it can be reprogrammed). First introduced in the mid-1980s, flash memory is intermediate between EPROM and EEPROM in both cost and functionality. Like EEPROM, flash memory uses an electrical erasing technology. An entire flash memory can be erased in one or a few seconds, which is much faster than EPROM. In addition, it is possible to erase just blocks of memory rather than an entire chip. Flash memory gets its name because the microchip is organised so that a section of memory cells is erased in a single action or "flash." However, flash memory does not provide byte-level erasure. Like EPROM, flash memory uses only one transistor per bit, and so achieves the high density (compared with EEPROM) of EPROM.

The figure below is summary of the memory arrays that we have.



3.2.2 Conclusion

Early computers' memory was made up of an array of doughnut-shaped ferromagnetic loops called cores. However, the emergence of microelectronics has since replaced the magnetic core memory. Today, the use of semiconductor chips for main memory is almost universal. In this unit, we explored key aspects of semiconductor technology.

Summary

In this unit, you have studied different aspects of semiconductor memory, from what it is to the forms of semiconductor memory, namely DRAM and SRAM. You also learned the different classifications of ROM, which is an example of a semiconductor memory. Some of the types of ROM discussed include PROM, EPROM, EEPROM, MROM and Flash.

Self-Assessment Exercise

1. Define DRAM.
2. Define SRAM.
3. State two key differences between DRAM and SRAM.
4. Why is DRAM commonly used as main memory?
5. Why is SRAM used in cache memory?
6. What is the major disadvantage of DRAM?
7. What is ROM?
8. What is MROM?
9. What is PROM?
10. What is EPROM?
11. What is Flash Memory?
12. Why is EPROM not commonly used in modern systems?
13. Give two applications of Flash memory.

Unit Activities

Activity 1: Comparison Table Construction

Students should create a table comparing:

- DRAM
- SRAM
- ROM types (PROM, EPROM, EEPROM, Flash, MROM)

Include features such as speed, cost, volatility, and applications.

Activity 2: Diagram Drawing

Students should draw and label:

- A DRAM cell
- An SRAM cell

Then explain how data is stored in each.

Tutor-Marked Assignment

1. What are the key properties of semiconductor memory?
2. What are two interpretations of the term random-access memory?
3. What is the difference between DRAM and SRAM in terms of application?
4. What is the difference between DRAM and SRAM in terms of characteristics such as speed, size, and cost?
5. Explain why one type of RAM is considered to be analogue and the other digital.
6. What are some applications for ROM?
7. What are the differences among EPROM, EEPROM, and flash memory?

References/Further Readings

Computer Organisation and Architecture: Designing for Performance by William Stallings, Tenth edition (2016). Read all sections in Chapter 5

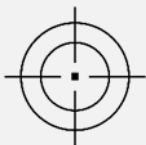
http://home.ustc.edu.cn/~leedsong/reference_books_tools/Computer%20Organization%20and%20Architecture%2010th%20-%20William%20Stallings.pdf

Computer Organisation and Design: The Hardware/Software Interface: RISC-V Edition - Read all sections in Chapter 1

Unit 3 — Cache Memory Design and Organisation

INTRODUCTION

Analysis of a large number of programs shows that references to memory at any given interval of time tend to be confined to a few localised areas in memory. This is known as the locality of reference. If the active portion of the program and data are placed in fast memory, then the average execution time of the program can be reduced. Such fast memory is called cache memory, and it is placed between the main memory and the CPU.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

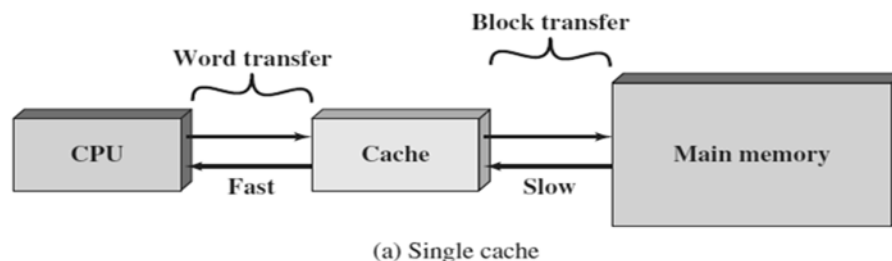
- Describe the basic concept of cache memory in terms of its function or usefulness.
- Explain typical cache organisation and elements of cache design
- Demonstrate cache mapping functions with practical solved examples
- Describe the concept of multilevel cache

3.3.1 Overview of Cache Memory

Cache memory is designed to combine the memory access time of expensive, high-speed memory combined with the large memory size of less expensive, lower-speed memory. The concept is illustrated in Figure 4.3a. There is a relatively large and slow main memory together with a smaller, faster cache memory. The cache contains a copy of portions of main memory. When the processor attempts to read a word of memory, a check is made to determine if the word is in the cache. If so, the word is delivered to the processor. If not, a block of main memory, consisting of some fixed number of words, is read into the cache, and then the word is delivered to the processor.

Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that there will be future references to that same memory location or to other words in the block.

Figure 19 depicts the structure of a cache/main-memory system. Main memory consists of up to 2^n addressable words, with each word having a unique n -bit address. For mapping purposes, this memory is considered to consist of a number of fixed-length blocks of K words each. That is, there are $M = 2^n/K$ blocks in main memory. The cache consists of m blocks, called **lines**.³ Each line contains K words,



In referring to the basic unit of the cache, the term line is used, rather than the term block, for two reasons:

- To avoid confusion with a main memory block, which contains the same number of data words as a cache line; and
- Because a cache line includes not only K words of data, just as a main memory block, but also includes tag and control bits.

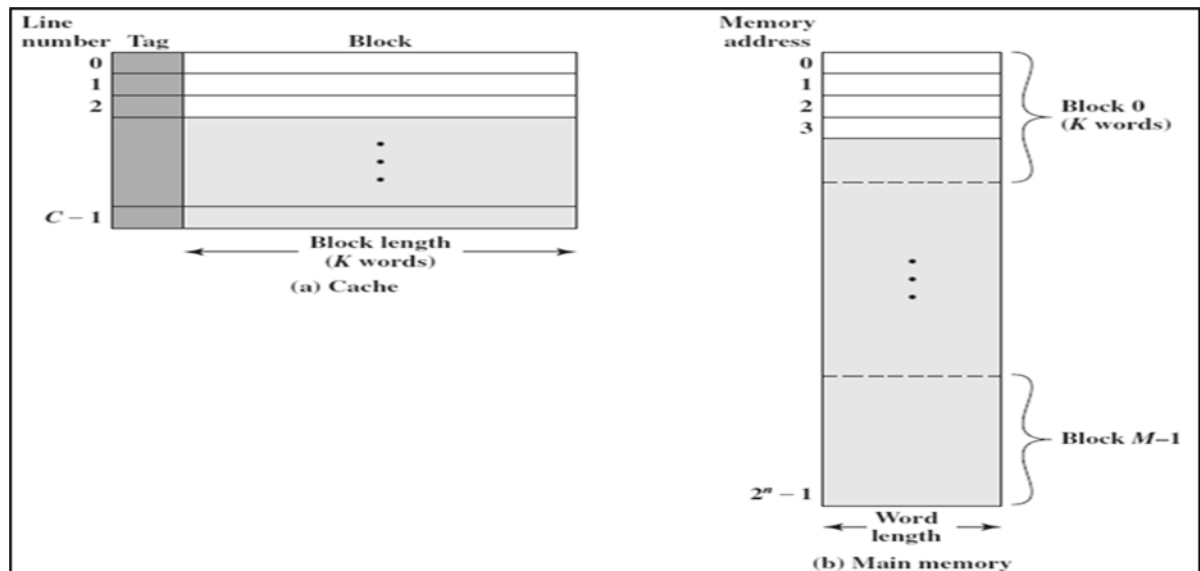


Figure 19: Cache - Memory Structure

3.3.2 Typical Cache Organization

In this organization, the cache connects to the processor via data, control, and address lines. The data and address lines also attach to data and address buffers, which attach to a system bus from which main memory is reached. When a cache hit occurs, the data and address buffers are disabled and communication is only between processor and cache, with no system bus traffic. When a cache miss occurs, the desired address is loaded onto the system bus and the data are returned through the data buffer to both the cache and the processor. In other organizations, the cache is physically interposed between the processor and the main memory for all data, address, and control lines. In this latter case, for a cache miss, the desired word is first read into the cache and then transferred from cache to processor.

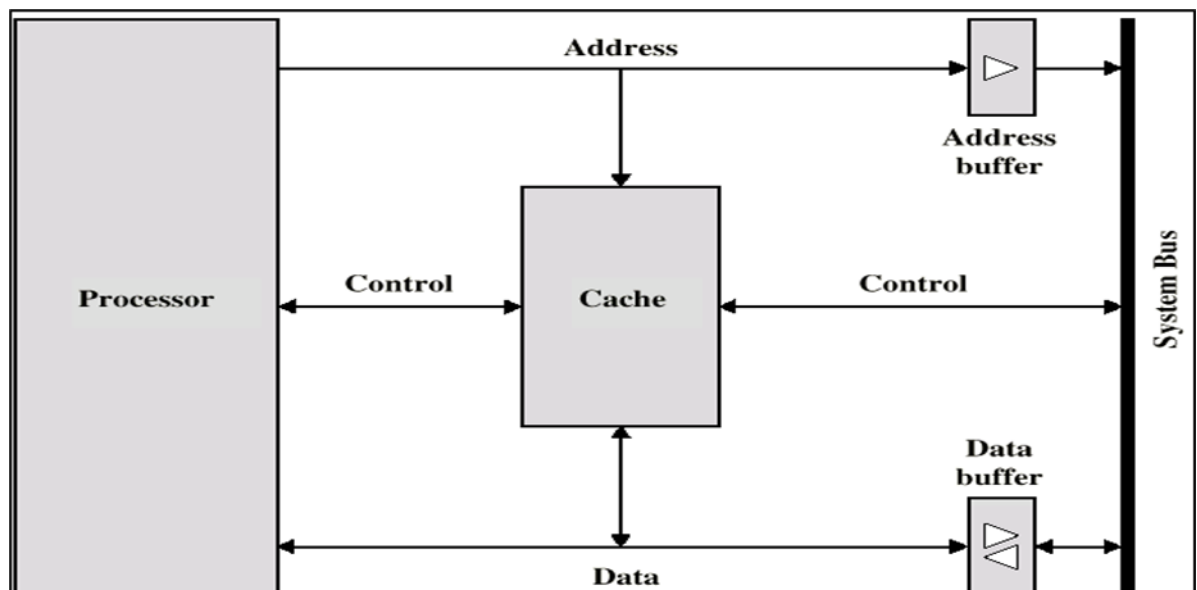


Figure 20: Cache – Processor Operation

3.3.3 Elements of Cache Design

This section provides an overview of cache design parameters and reports some typical results. We occasionally refer to the use of caches in high-performance computing (HPC). HPC deals with supercomputers and their software, especially for scientific applications that involve large amounts of data, vector and matrix computation, and the use of parallel algorithms. Cache design for HPC is quite different from that of other hardware platforms and applications. Indeed, many researchers have found that HPC applications perform poorly on computer architectures that employ caches.

Other researchers have since shown that a cache hierarchy can be useful in improving performance if the application software is tuned to exploit the cache. Although there are a large number of cache implementations, there are a few basic design elements that serve to classify and differentiate cache architectures. The table below lists key elements.

Cache Addresses	Write Policy
Logical	Write through
Physical	Write back
Cache Size	Line Size
Mapping Function	Number of Caches
Direct	Single or two level
Associative	Unified or split
Set associative	
Replacement Algorithm	
Least recently used (LRU)	
First in first out (FIFO)	
Least frequently used (LFU)	
Random	

Figure 21: Elements of Cache Design

Cache Size

The size of the cache should be small enough so that the overall average cost per bit is close to that of main memory alone and large enough so that the overall average access time is close to that of the cache alone. There are several other motivations for minimising cache size. The larger the cache, the larger the number of gates involved in addressing the cache. The result is that large caches tend to be slightly slower than small ones—even when built with the same integrated circuit

technology and put in the same place on the chip and circuit board. The available chip and board area also limits cache size. Because the performance of the cache is very sensitive to the nature of the workload, it is impossible to arrive at a single “optimum” cache size. Table 4.3 lists the cache sizes of some current and past processors.

3.3.4 Mapping Function

Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines. Further, a means is needed for determining which main memory block currently occupies a cache line. The choice of the mapping function dictates how the cache is organised. Three techniques can be used: direct, associative, and set-associative. We examine each of these in turn. In each case, we look at the general structure and then a specific example.

For all three cases, the example includes the following elements:

- The cache can hold 64 kB.
- Data are transferred between main memory and the cache in blocks of 4 bytes each. This means that the cache is organised as $16K = 214$ lines of 4 bytes each.
- The main memory consists of 16 MB, with each byte directly addressable by a 24-bit address ($2^{24} = 16M$). Thus, for mapping purposes, we can consider the main memory to consist of 4M blocks of 4 bytes each.

Direct Mapping Function

The simplest technique, known as direct mapping, maps each block of main memory into only one possible cache line. The mapping is expressed as

$$i = j \text{ modulo } m$$

where

i = cache line number

j = main memory block number

m = number of lines in the cache

The figure below shows the mapping for the first m blocks of main memory. Each block of main memory maps into one unique line of the cache. The next m blocks

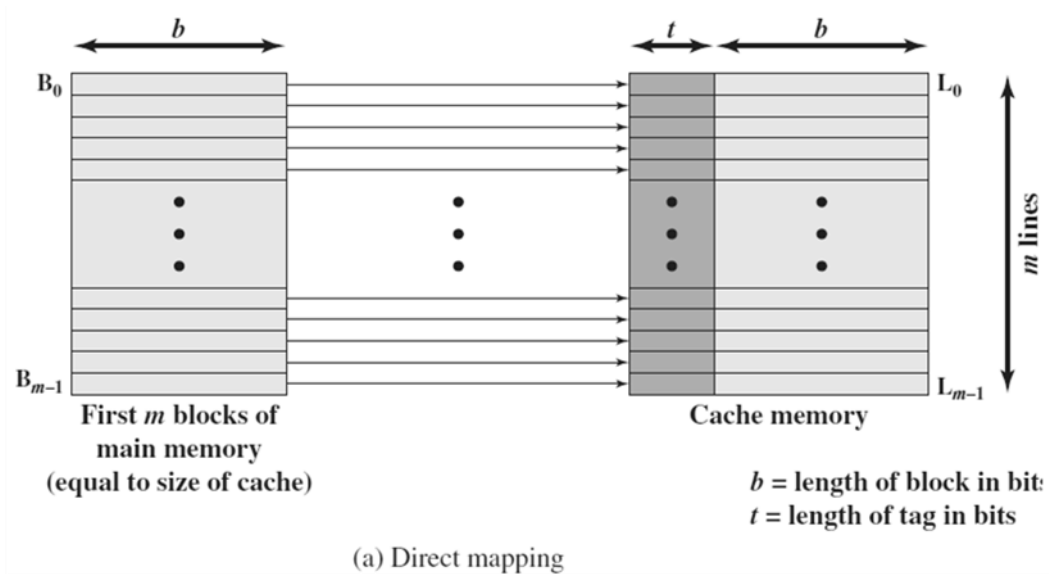


Figure 22: Direct Mapping

Worked Example on Direct Mapping:

Consider a direct-mapped cache of size 16 KB with a block size of 256 bytes. The size of the main memory is 128 KB. Find-

1. Number of bits in tag
2. Tag directory size

Solution

Given-

Cache memory size = 16 KB

Block size = Frame size = Line size = 256 bytes

- Main memory size = 128 KB

We consider that the memory is byte addressable.

Number of Bits in Physical Address

We have,

Size of main memory

= 128 KB = 217 bytes

Thus, the number of bits in the physical address = 17 bits

Number of Bits in Block Offset

We have,

Block size = 256 bytes = 28 bytes

Thus, the number of bits in the block offset = 8 bits

Number of Bits in Line Number

Total number of lines in cache = Cache size / Line size

= 16 KB/256 bytes

= 214 bytes / 28 bytes = 26 lines

Thus, the number of bits in the line number = 6 bits

Number of Bits in Tag

= Number of bits in physical address – (Number of bits in line number + Number of bits in block offset)

= 17 bits – (6 bits + 8 bits)

= 17 bits – 14 bits = 3 bits

Thus, the number of bits in the tag = 3 bits

Tag Directory Size

Tag directory size

= Number of tags x Tag size = Number of lines in cache x Number of bits in tag

= 26 x 3 bits = 192 bits = 24 bytes

Thus, the size of the tag directory = 24 bytes

Example 2:

Consider a direct-mapped cache of size 512 KB with a block size of 1 KB. There are 7 bits in the tag. Find-

1. Size of main memory
2. Tag directory size

Solution

Given-

- Cache memory size = 512 KB
- Block size = Frame size = Line size = 1 KB
- Number of bits in tag = 7 bits

We consider that the memory is byte addressable.

Number of Bits in Block Offset

We have a block size

= 1 KB

= 210 bytes

Thus, the number of bits in the block offset = 10 bits

Number of Bits in Line Number

Total number of lines in cache

= Cache size / Line size

= 512 KB / 1 KB = 29 lines

Thus, the number of bits in the line number = 9 bits

Number of Bits in Physical Address

Number of bits in physical address = Number of bits in tag + Number of bits in line number + Number of bits in block offset

= 7 bits + 9 bits + 10 bits = 26 bits

Thus, the number of bits in the physical address = 26 bits

Size of Main Memory

We have the number of bits in the physical address = 26 bits

Thus, the size of the main memory

= 226 bytes = 64 MB

Tag Directory Size

Tag directory size = Number of tags x Tag size

= Number of lines in cache x Number of bits in tag

= $29 \times 7 \text{ bits} = 3584 \text{ bits} = 448 \text{ bytes}$

Thus, the size of the tag directory = 448 bytes

Associative Mapping

Associative mapping overcomes the disadvantage of direct mapping by permitting each main memory block to be loaded into any line of the cache (Figure 4.8b). In this case, the cache control logic interprets a memory address simply as a Tag and a Word field. The Tag field uniquely identifies a block of main memory. To determine whether a block is in the cache, the cache control logic must simultaneously examine every line's tag for a match.

Worked Examples on Associative Mapping

Example 1:

Consider a fully associative mapped cache of size 16 KB with a block size of 256 bytes. The size of the main memory is 128 KB. Find-

1. Number of bits in tag
2. Tag directory size

Solution

Given-

- Cache memory size = 16 KB
- Block size = Frame size = Line size = 256 bytes
- Main memory size = 128 KB

We consider that the memory is byte addressable.

Number of Bits in Physical Address

We have,

Size of main memory

= 128 KB

= 217bytes

Thus, the number of bits in the physical address = 17 bits

Number of Bits in Block Offset

We have a block size

= 256 bytes

= 28 bytes

Thus, the number of bits in the block offset = 8 bits

Number of Bits in Tag

= Number of bits in physical address – Number of bits in block offset

= 17 bits – 8 bits = 9 bits

Thus, the number of bits in the tag = 9 bits

Number of Lines in Cache

Total number of lines in cache

= Cache size / Line size

= 16 KB / 256 bytes

= 214bytes / 28bytes = 26lines

Tag Directory Size

= Number of tags x Tag size

= Number of lines in cache x Number of bits in tag

= 26x 9 bits

= 576 bits = 72 bytes

Thus, the size of the tag directory = 72 bytes

Example 2:

Consider a fully associative mapped cache with a block size of 4 KB. The size of the main memory is 16 GB. Find the number of bits in the tag.

Solution

Given-

- Block size = Frame size = Line size = 4 KB

- Size of main memory = 16 GB

We consider that the memory is byte addressable.

Number of Bits in Physical Address

We have,

Size of main memory

= 16 GB

= 234 bytes

Thus, the number of bits in the physical address = 34 bits

Number of Bits in Block Offset

We have,

Block size

= 4 KB = 212 bytes

Thus, the number of bits in the block offset = 12 bits

Number of Bits in Tag

= Number of bits in physical address – Number of bits in block offset

= 34 bits – 12 bits = 22 bits

Thus, the number of bits in the tag = 22 bits

Set associative mapping

This form of mapping is an enhanced form of direct mapping where the drawbacks of direct mapping are removed. Set-associative addressing addresses the problem of possible thrashing in the direct-mapped method. It does this by saying that instead of having exactly one line that a block can map to in the cache, we will group a few lines together, creating a set. Then a block in memory can map to any one of the lines of a specific set. Set-associative mapping allows each word that is present in the cache to have two or more words in the main memory for the same index address. Set associative cache mapping combines the best of direct and associative cache mapping techniques.

Worked Examples on Set-Associative Mapping

Example 1: Consider a 2-way set-associative mapped cache of size 16 KB with a block size of 256 bytes. The size of the main memory is 128 KB. Find-

- Number of bits in tag
- Tag directory size

Solution

Given

- Set size = 2
- Cache memory size = 16 KB
- Block size = Frame size = Line size = 256 bytes
- Main memory size = 128 KB

We consider that the memory is byte addressable.

Number of Bits in Physical Address

We have,

Size of main memory

= 128 KB = 217 bytes

Thus, the number of bits in the physical address = 17 bits

Number of Bits in Block Offset

We have,

Block size

= 256 bytes = 28 bytes

Thus, the number of bits in the block offset = 8 bits

Number of Lines in Cache

Total number of lines in cache

= Cache size / Line size

= 16 KB / 256 bytes

= 214 bytes / 28 bytes = 64 lines

Thus, the number of lines in cache = 64 lines

Number of Sets in Cache

Total number of sets in cache

= Total number of lines in cache / Set size

= $64 / 2 = 32$ sets

= 25 sets

Thus, the number of bits in the set number = 5 bits

Number of bits in tag

= Number of bits in physical address – (Number of bits in set number + Number of bits in block offset)

= 17 bits – (5 bits + 8 bits)

= 17 bits – 13 bits

= 4 bits

Thus, the number of bits in the tag = 4 bits

Tag Directory Size

= Number of tags x Tag size

= Number of lines in cache x Number of bits in tag

= 64×4 bits

= 256 bits

= 32 bytes

Thus, the size of the tag directory = 32 bytes

Example 2:

Consider an 8-way set-associative mapped cache of size 512 KB with a block size of 1 KB. There are 7 bits in the tag. Find-

- Size of main memory
- Tag directory size

Solution

Given-

- Set size = 8
- Cache memory size = 512 KB
- Block size = Frame size = Line size = 1 KB
- Number of bits in tag = 7 bits

We consider that the memory is byte addressable.

Number of Bits in Block Offset

We have,

Block size

= 1 KB

= 210 bytes

Thus, the number of bits in the block offset = 10 bits

Number of Lines in Cache

Total number of lines in cache

= Cache size / Line size

= 512 KB / 1 KB

= 512 lines

Thus, the number of lines in cache = 512 lines

Number of Sets in Cache

Total number of sets in cache

= Total number of lines in cache / Set size

= 512 / 8 = 64 sets

= 26 sets

Thus, the number of bits in the set number = 6 bits

Number of Bits in Physical Address

= Number of bits in tag + Number of bits in set number + Number of bits in block offset

= 7 bits + 6 bits + 10 bits = 23 bits

Thus, the number of bits in the physical address = 23 bits

Size of Main Memory

We have,

Number of bits in physical address = 23 bits

Thus, the size of the main memory

= 223 bytes = 8 MB

Tag directory size

= Number of tags x Tag size

= Number of lines in cache x Number of bits in tag

= 512 x 7 bits

= 3584 bits

= 448 bytes

Thus size of tag directory=448bytes

3.3.5 Conclusion

Cache memory is designed to combine the memory access time of expensive, high-speed memory with the large memory size of less expensive, lower-speed Memory. The whole idea behind the introduction of cache memory was to improve the performance of a computer by reducing the idle time of a processor.

Summary

In this unit, you have learnt different aspects of cache memory, including its organisation, elements of design, multilevel caches, and the three cache mapping functions, namely direct, associative and set-associative.

Self-Assessment Questions

1. What is cache memory?
2. Why is cache memory needed?
3. What are the main components of cache organisation?
4. What is a cache hit?
5. What is a cache miss?
6. Define hit ratio.
7. What is a tag in cache memory?
8. What is cache associativity?
9. What is a replacement policy?
10. Name common replacement policies.
11. What is a write-through policy?
12. What is the write-back policy?
13. Calculate AMAT if:
 - Hit time = 1 ns
 - Miss rate = 0.05
 - Miss penalty = 100 ns
14. Compare direct and set-associative mapping in terms of performance.
15. Why does increasing cache size improve performance?
16. Explain the trade-off between block size and miss rate.

Unit Activities

Activity 1: Cache Organisation Breakdown

Given a cache system with the following specifications:

- Cache size = 8 KB
- Block size = 16 bytes

Questions:

- How many cache blocks are there?
- Identify the fields in a memory address (Tag, Index, Offset)

Expected Outcome:

Students should compute:

- Number of blocks = $8192 / 16 = 512$ blocks
- Understand address partitioning

Activity 2: Direct Mapping Exercise

A cache has 8 lines. Main memory has 32 blocks.

- Map the following memory blocks into cache:
 - Block 0, 8, 16, 24

Questions:

- Which cache line will each block map to?
- Identify any conflicts

Expected Answer:

- Use formula: Cache line = Block number mod number of lines
- All map to line 0 → conflict occurs

Activity 3: Associative Mapping Analysis

Task: Assume a fully associative cache with 4 lines.

- Place memory blocks: 2, 4, 6, 8

Questions:

- Where can each block be placed?
- What replacement policy would you use when the cache is full?

Expected Outcome:

- Any block can go into any line
- Replacement: LRU, FIFO, or Random

Activity 4: Set-Associative Mapping

Task: A cache has:

- 8 lines
- 2-way set associative

Questions:

- How many sets are there?
- Map block 10 into the cache

Expected Answer:

- Sets = $8 / 2 = 4$
- Set index = Block mod number of sets

Activity 5: Cache Performance Scenario

Task: A processor accesses memory with:

- Cache hit time = 2 ns
- Miss penalty = 50 ns
- Hit ratio = 0.9

Question:

Calculate average memory access time (AMAT)

Solution:

AMAT = Hit time + Miss rate $\times$ Miss penalty

$$= 2 + (0.1 \times 50) = 7 \text{ ns}$$

Tutor-Marked Assignment

1. What are the differences among direct mapping, associative mapping, and set-associative mapping?
2. For a direct-mapped cache, a main memory address is viewed as consisting of three fields. List and define the three fields.
3. For an associative cache, a main memory address is viewed as consisting of two fields. List and define the two fields.
4. For a set-associative cache, a main memory address is viewed as consisting of three fields. List and define the three fields.
5. What is the distinction between spatial locality and temporal locality?
6. In general, what are the strategies for exploiting spatial locality and temporal locality?

References/Further Readings

Computer Organisation and Architecture: Designing for Performance by William Stallings,
Tenth edition (2016). Read all sections in Chapter 5

http://home.ustc.edu.cn/~leedsong/reference_books_tools/Computer%20Organization%20and%20Architecture%2010th%20-%20William%20Stallings.pdf

Computer Organisation and Design: The Hardware/Software Interface: RISC-V Edition -
Read all sections in Chapter 1

Other internet resources may also be used.

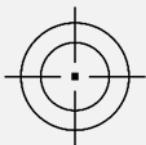
Module Four

Basic and Advanced Processor Architectures

Unit 1 — RISC Versus CISC Architectures

INTRODUCTION

In this unit, we consider the organisation of computer processors with the aim of understanding the available types of processors. Thus, two classifications of processors are considered, namely Reduced Instruction Set Computers (RISC) and Complex Instruction Set Computers (CISC). Each of the categories is discussed in great detail, including their characteristics, advantages and disadvantages in various sections of the unit. The unit concludes with a comparison of the two categories.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Differentiate between RISC and CISC architectures
- Explain the design principles of RISC architectures,
- Describe the characteristics of CISC architectures,
- Compare the performance implications of RISC and CISC processors, considering factors such as CPI, clock speed, and instruction count.

4.1.1 Overview of Processor Organisation

A processor is the logic circuitry that responds to and processes the basic instructions that drive a computer. CPU instructions are numbers stored in memory. An instruction set is a set of instructions a programmer can give to a machine to perform operations. These instructions are specific to the CPU architecture of a given computer. However, there are common basic operations that all CPUs should be able to carry out. These operations include reading instructions from memory, decoding the instructions, executing the instructions, and writing back the result of the operation to memory.

Central Processing Unit Architecture operates the capacity to work from "Instruction Set Architecture" to where it was designed. The architectural designs of CPU are RISC (Reduced Instruction Set Computing) and CISC (Complex Instruction Set Computing). CISC has the ability to execute addressing modes or multi-step operations within one instruction set. It is the design of the CPU where one instruction performs many low-level operations. For example, memory storage, an arithmetic operation and loading from memory. RISC is a CPU design strategy based on the insight that a simplified instruction set gives higher performance when combined with a microprocessor architecture which has the ability to execute the instructions by using fewer microprocessor cycles per instruction.

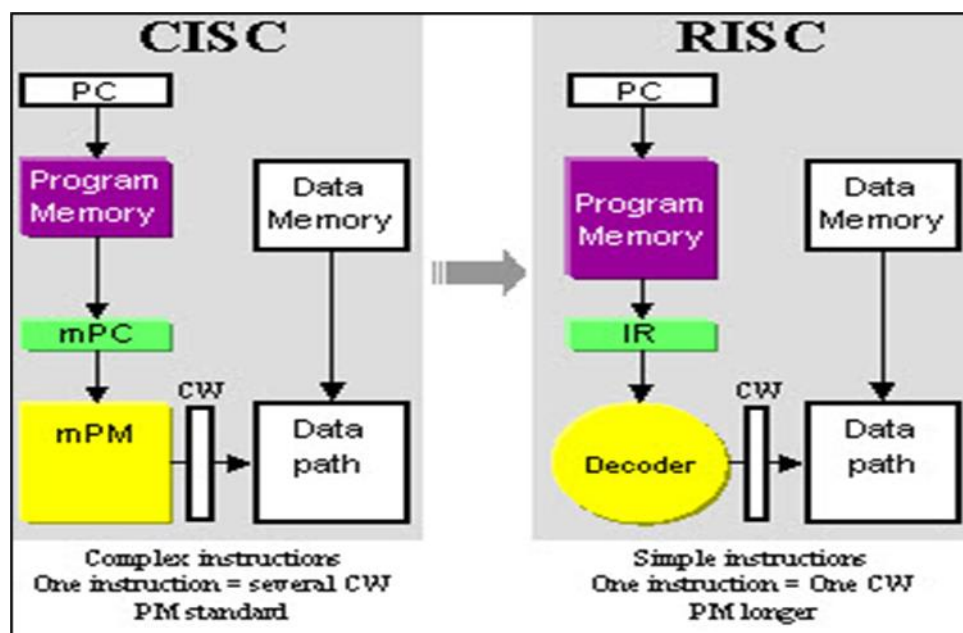


Figure 23: RISC and CISC Architectures

Hardware designers invent numerous technologies and tools to implement the desired architecture in order to fulfil these needs. Hardware architecture may be implemented to be either hardware-specific or software-specific, but according to the

application, both are used in the required quantity. As far as the processor hardware is concerned, there are two types of concepts to implement the processor hardware architecture. They are:

- Complex Instruction Set Computers (CISC) and
- Reduce Instruction Set Computers (RISC)

In this unit, the two categories of processor organisation are discussed in detail.

4.1.2 Complex Instruction Set Computers (CISC)

The CISC approach attempts to minimise the number of instructions per program, sacrificing the number of cycles per instruction. Computers based on the CISC architecture are designed to decrease the memory cost. Because the large programs need more storage, thus increasing the memory cost, and large memory becomes more expensive. To solve these problems, the number of instructions per program can be reduced by embedding the number of operations in a single instruction, thereby making the instructions more complex.

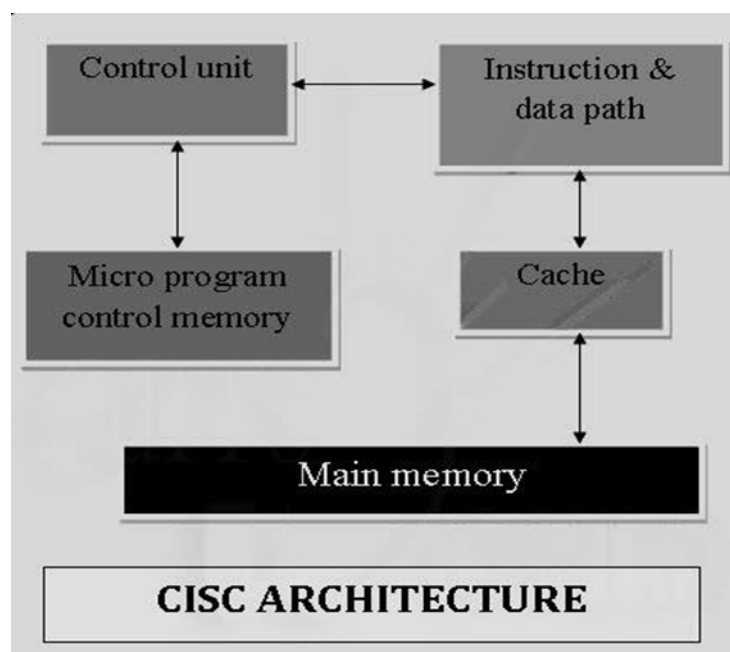


Figure 24: CISC Architecture

Complex instruction set computing is a CPU design where single instructions can execute several low-level operations (such as a load from memory, an arithmetic operation, and a memory store) or are capable of multi-step operations or addressing modes within single instructions.

CISC existed close to the beginning of general computing. Since the earliest machines were programmed in assembly language and memory was slow and

expensive, IBM designed an instruction set to allow early programmers to easily program a hundred complex instructions rather than thousands of individual instructions.

CISC was developed to make compiler development simpler. It shifts most of the burden of generating machine instructions to the processor. For example, instead of having to make a compiler write long machine instructions to calculate a square-root, a CISC processor would have a built-in ability to do this. CISC is called "complex" because of the complex work performed per instruction. The concept is to encode the intention directly e.g. "add X and Y and put the result in Z" (for X,Y,Z memory address).

Computers typically use CISC while tablets, smartphones and other devices use RISC. So, the higher efficiency of the RISC architecture makes it desirable in these applications where cycles and power are usually in short supply. In CISC, instructions are executed by microcode. A CISC instruction set typically includes many instructions of different sizes and execution cycles, which makes CISC instructions harder to pipeline.

Properties of CISC Processors

1. Richer instruction set, some simple, some very complex.
2. Instructions generally take more than 1 clock cycle to execute.
3. Instructions are of a variable size.
4. Instruction is an interface with memory in multiple mechanisms with complex addressing modes.
5. No pipelining.
6. Microcode control makes the CISC instruction set possible and flexible.
7. CISC processors work well with simpler compilers.

Advantages of CISC Processors

With CISC processors, microprogramming is as easy as assembly language to implement, and much less expensive than hardwiring a control unit. Also, as each instruction became more capable, fewer instructions could be used to implement a given task. This made more efficient use of the relatively slow main memory. Because micro-program instruction sets can be written to match the constructs of high-level languages, the compiler does not have to be as complicated.

Disadvantages of CISC Processors

Complex instructions are infrequently used by programmers and compilers. Memory references, loads and stores are slow and account for a significant fraction of all instructions. Procedure and function calls are a major bottleneck in passing arguments, storing, and retrieving values in registers. Instruction sets and chips of new-generation hardware become more complex with each generation of computers.

Examples of CISC Processors

- IBM 370/168 – It was introduced in the year 1970. CISC design is a 32-bit processor with four 64-bit floating-point registers.
- VAX 11/780 – CISC design is a 32-bit processor, and it supports a number of addressing modes and machine instructions from Digital Equipment Corporation.
- Intel 80486 – It was launched in the year 1989, and it is a CISC processor, which has instructions varying in length from 1 to 11, and it has 235 instructions.

Some other examples of processors that fall under this category are listed below.

CISC Processors
IBM 370 168
VAX 11 780
Microvax II
INTEL 80386
INTEL 80286
Sun-3 5
PDP-11

4.1.3 Reduced Instruction set Computer (RISC)

RISC (Reduced Instruction Set Computer) is used in portable devices due to its power efficiency. For example, Apple iPod and Nintendo DS. RISC is a type of microprocessor architecture that uses highly-optimized set of instructions. RISC does the opposite, reducing the cycles per instruction at the cost of the number of instructions per program. Pipelining is one of the unique features of RISC. It is performed by overlapping the execution of several instructions in a pipeline fashion. It has a high-performance advantage over CISC.

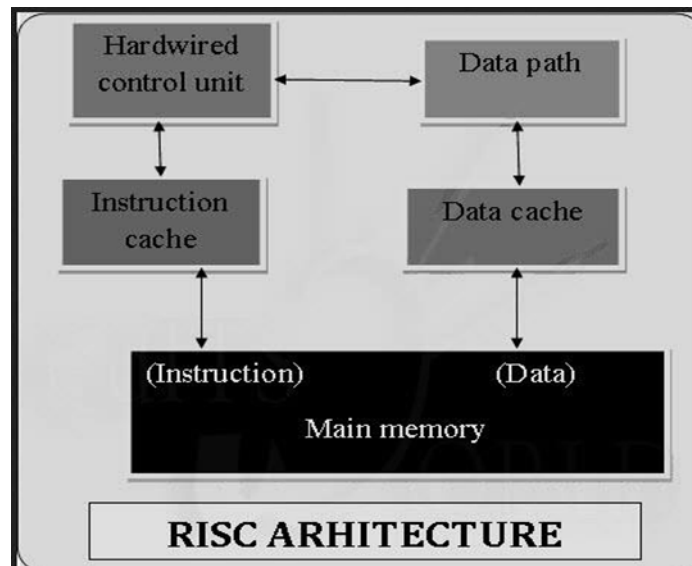


Figure 25: RISC Architecture

Properties of RISC Processors

- Simple Instructions are used in RISC architecture.
- RISC helps and supports few simple data types and synthesizes complex data types.
- RISC utilizes simple addressing modes and fixed length instructions for pipelining.
- RISC permits any register to use in any context.
- One Cycle Execution Time
- The amount of work that a computer can perform is reduced by separating "LOAD" and "STORE" instructions.
- RISC contains Large Number of Registers in order to prevent various number of interactions with memory.
- In RISC, Pipelining is easy as the execution of all instructions will be done in a uniform interval of time i.e. one clock.
- In RISC, more RAM is required to store assembly level instructions.
- Reduced instructions need a less number of transistors in RISC.
- RISC uses Harvard memory model means it is Harvard Architecture.
- A compiler is used to perform the conversion operation means to convert a high-level language statement into the code of its form.

Advantages of RISC architecture

- RISC (Reduced instruction set computing) architecture has a set of instructions, so high-level language compilers can produce more efficient code

- It allows freedom of using the space on microprocessors because of its simplicity.
- Many RISC processors use the registers for passing arguments and holding the local variables.
- RISC functions use only a few parameters, and the RISC processors cannot use the call instructions, and therefore, use a fixed length instruction which is easy to pipeline.
- The speed of the operation can be maximized and the execution time can be minimized.
- Very less number of instructional formats, a few numbers of instructions and a few addressing modes are needed.

Disadvantages of RISC architecture

- Mostly, the performance of the RISC processors depends on the programmer or compiler as the knowledge of the compiler plays a vital role while changing the CISC code to a RISC code
- While rearranging the CISC code to a RISC code, termed as a code expansion, will increase the size. And, the quality of this code expansion will again depend on the compiler, and also on the machine's instruction set.
- The first level cache of the RISC processors is also a disadvantage of the RISC, in which these processors have large memory caches on the chip itself. For feeding the instructions, they require very fast memory systems.

Examples of RISC Processors

Below is the list of processors that fall under the category of reduced instruction set computers.

RISC Processors
MIPS R2000
SUN SPARC
INTEL i860
MOTOROLA 8800
POWERPC 601
IBM RS. 6000
MIPS R4000

Comparison of RISC and CISC

MUL instruction is divided into three instructions

"LOAD" – moves data from the memory bank to a register

"PROD" – finds product of two operands located within the registers

"STORE" – moves data from a register to the memory banks

The main difference between RISC and CISC is the number of instructions and its complexity.

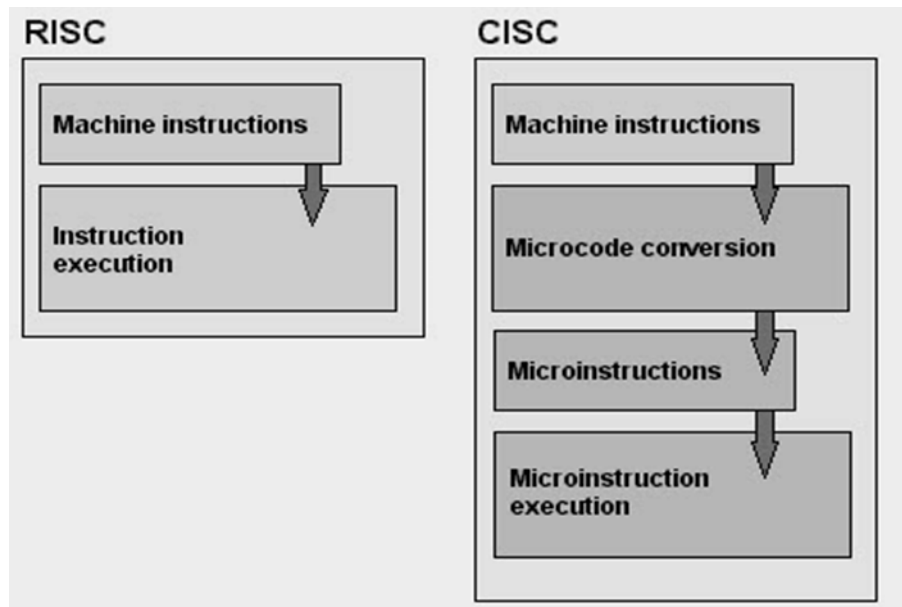


Figure 26: RISC versus CISC

CISC	RISC
It has a microprogramming unit.	It has a hard-wired unit of programming.
The instruction set has various different instructions that can be used for complex operations.	The instruction set is reduced, and most of these instructions are very primitive.
Performance is optimized with emphasis on hardware.	Performance is optimized which emphasis on software
Only single register set	Multiple register sets are present
They are mostly less or not pipelined	This type of processors are highly pipelined
Execution time is very high	Execution time is very less
Code expansion is not a problem.	Code expansion may create a problem.
Decoding of instructions is complex.	The decoding of instructions is simple.
It requires external memory for calculations	It doesn't require external memory for calculations
Examples of CISC processors are the System/360, VAX, AMD, and Intel x86 CPUs.	Common RISC microprocessors are ARC, Alpha, ARC, ARM, AVR, PA-RISC, and SPARC.
Single-cycle for each instruction	Instructions can take several clock cycles

Heavy use of RAM (can cause bottlenecks if RAM is limited)	More efficient use of RAM than RISC
Simple, standardized instructions	Complex and variable-length instructions
A small number of fixed-length instructions	A large number of instructions
Limited addressing modes	Compound addressing modes
Important applications are Security systems, Home automation.	Important applications are : Smartphones, PDAs.
Varying formats (16-64 bits for each instruction).	fixed (32-bit) format
Unified cache for instructions and data.	Separate data and instruction cache.

4.1.4 Conclusion

A processor is the logic circuitry that responds to and processes the basic instructions that drive a computer. Instruction set is a set of instructions a programmer can give to a machine to perform operations. These instructions are specific to the CPU architecture of a given computer. However, there are common basic operations that all CPUs should be able to carry out. These operations include read instruction from memory, decode the instruction, execute instruction, and write back the result of the operation to memory. In this unit, different aspects of processor organization are discussed with emphasis on two major types of processor organization namely RISC and CISC.

Summary

In this unit you have studied different aspects of processor organization as well as instruction set. The two major processor organization namely RISC and CISC were discussed based on a number of their characteristics. Finally, a comparison between the two was done.

Self-Assessment Exercise

1. Define RISC architecture.
2. Define CISC architecture.

3. State two advantages of RISC over CISC.
4. State two advantages of CISC over RISC.
5. What is the impact of memory usage in RISC vs CISC?
6. Why is RISC commonly used in mobile devices?

Unit Activities

Activity 1: Instruction Analysis

- Write a simple program (e.g., addition of two numbers)
- Represent it using:
 - RISC-style instructions
 - CISC-style instructions
- Compare number of instructions used

Activity 2: Architecture Identification

- Identify whether the following processors are RISC or CISC:
 - ARM
 - x86
 - MIPS

Tutor-Marked Assignment

1. Compare RISC and CISC architectures in terms of:
 - Instruction set design
 - Execution speed
 - Hardware complexity
 - Power consumption
2. Explain how pipelining improves performance in RISC architectures. Use diagrams where necessary.
3. Discuss the role of microprogrammed control in CISC processors.
4. Evaluate the statement: "RISC architectures are always faster than CISC architectures." Provide arguments for and against.
5. A program written for a CISC machine uses 500 instructions. The equivalent RISC program uses 800 instructions but executes twice as fast per instruction.

- Which system performs better? Justify with calculations.
6. Discuss real-world applications where CISC architectures are preferred over RISC.

References/Further Readings

Textbooks

Computer Organization and Design – David A. Patterson & John L. Hennessy

Computer Architecture: A Quantitative Approach – John L. Hennessy & David A. Patterson

Structured Computer Organization – Andrew S. Tanenbaum

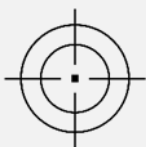
Online Resources

MIT OpenCourseWare (Computer Architecture courses)

Unit 2 — Parallel Architectures and Interconnection Networks

INTRODUCTION

Modern computing systems demand high performance to handle large-scale applications such as scientific simulations, artificial intelligence, big data analytics, and real-time systems. Traditional single-processor systems are no longer sufficient to meet these demands.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain the concept of parallel processing and its importance
- Identify and differentiate types of parallel architectures
- Describe various interconnection networks and topologies
- Analyse communication mechanisms in parallel systems

4.2.1 Parallel Processing Concepts

Parallel processing is the simultaneous execution of multiple computations using multiple processing units.

Key Concepts

- **Concurrency vs Parallelism**
 - Concurrency: Multiple tasks in progress
 - Parallelism: Tasks executed simultaneously
- **Granularity**
 - Fine-grained: Small tasks, frequent communication
 - Coarse-grained: Larger tasks, less communication
- **Synchronization**
 - Ensures correct execution order among processes.
- **Communication**

Processors exchange data via:

 - Shared memory
 - Message passing

Forms of Parallelism

- **Data Parallelism:** Same operation on different data
- **Task Parallelism:** Different tasks executed simultaneously

Scenario

A weather forecasting system divides a large geographical region into smaller sections and processes them simultaneously using multiple processors.

4.2.2 Parallel Architectures

Parallel architectures define how multiple processing units are organised, coordinated, and interconnected to execute tasks simultaneously. The design of these architectures determines system performance, scalability, cost, and efficiency.

Parallel architectures are broadly classified using:

- Instruction/Data stream models (Flynn's taxonomy)
- Memory organisation
- System structure

Flynn's Classification of Parallel Architectures

Proposed by Michael J. Flynn, this is the most widely used classification system.

SISD (Single Instruction, Single Data)

One processor executes a single instruction stream on a single data stream.

Characteristics

- Sequential execution
- No parallelism
- Traditional von Neumann architecture

Example

Early single-core computers

SIMD (Single Instruction, Multiple Data)

A single control unit issues the same instruction to multiple processing elements operating on different data.

Key Features

- High data parallelism
- Synchronised execution
- Efficient for repetitive tasks

Examples

- Vector processors
- GPUs
- Image processing systems

Advantages

- High throughput
- Simple control logic

Disadvantages

- Not suitable for tasks with varying operations

MISD (Multiple Instruction, Single Data)

Multiple processors execute different instructions on the same data stream.

Characteristics

- Rare in practice
- Used in fault-tolerant systems

Example

Systems performing redundant computations for reliability

MIMD (Multiple Instruction, Multiple Data)

Multiple processors execute different instructions on different data independently.

Characteristics

- Asynchronous execution
- High flexibility
- Most modern systems use this model

Examples

- Multi-core processors
- Distributed systems
- Cloud computing systems

4.2.3 Memory-Based Classification

Shared Memory Architecture

All processors share a common global memory.

Types

- Uniform Memory Access (UMA): Equal access time for all processors
- Non-Uniform Memory Access (NUMA): Access time depends on memory location

Advantages

- Easier programming
- Efficient communication

Disadvantages

- Scalability issues
- Memory contention

Example

Multi-core CPUs

Distributed Memory Architecture

- Each processor has its own local memory
- Communication via message passing

Characteristics

- No shared global memory
- Explicit communication

Advantages

- Highly scalable
- No memory contention

Disadvantages

- Complex programming
- Communication overhead

Examples

- Computer clusters
- Supercomputers

Hybrid Architecture

Combines shared and distributed memory

Example

Clusters of multi-core processors

Advantages

- Combines the benefits of both architectures
- Improved scalability

4.2.4 Instruction-Level Parallelism (ILP)

Parallelism is achieved by executing multiple instructions simultaneously within a processor.

Techniques

- Pipelining
- Superscalar execution
- Out-of-order execution

Example

Modern CPUs execute multiple instructions per cycle

4.2.5 Data-Level Parallelism (DLP)

The same operation is applied to multiple data elements simultaneously.

Example

Vector addition

4.2.6 Task-Level Parallelism (TLP)

Different tasks executed concurrently.

Example

Web server handling multiple requests

4.2.7 Interconnection Networks and Topologies

Processors in parallel systems must communicate. This is achieved using interconnection networks.

Types of Interconnection Networks

- **Shared Bus**
 - All processors share a common communication line
 - Simple but suffers from congestion
- **Crossbar Switch**
 - Provides dedicated paths between processors and memory
 - High performance but expensive
- **Multistage Networks**
 - Uses switches arranged in stages
 - Balanced cost and performance

Network Topologies

Mesh Topology

- Processors arranged in a grid
- Each node connects to neighbours

Ring Topology

- Nodes connected in a circular fashion
- Data travels in one direction

Star Topology

- Central node connects all others

Hypercube

- High-dimensional cube structure
- Efficient communication

Diagram Description (Conceptual)

- Mesh: Like a chessboard
- Ring: Circular loop
- Star: Hub and spokes

4.2.8 Communication Mechanisms in Parallel Systems

Shared Memory Communication

- All processors access a common memory
- Communication occurs via read/write operations

Advantages:

- Easy to program

Disadvantages:

- Memory contention

Message Passing

Processors communicate by sending messages

Advantages:

- Scalable

- Suitable for distributed systems

Disadvantages:

- Complex programming

Synchronization Mechanisms

Ensures correct execution order

Techniques:

- Locks
- Semaphores
- Barriers

Scenario

In a banking system:

- One processor updates the account balance
- Another reads it

Without synchronisation incorrect results

With locks → correct execution

Performance Metrics in Parallel Computing

- **Execution Time:** total time taken to complete a task
- **Throughput:** number of tasks completed per unit time
- **Latency:** time taken to send a message
- **Scalability:** ability to maintain performance with increased processors

In-Text Questions

1. What does high throughput indicate?
2. What problem occurs without synchronization?
3. Why is SIMD suitable for image processing?
4. Why is MIMD the most widely used architecture today?
5. What is the main limitation of shared memory systems?
6. Why is hypercube topology efficient?

In-Text Answers

1. The system can process many tasks efficiently.
2. Race conditions.
3. Because the same operation (e.g., filtering) is applied to many pixels simultaneously.
4. Because it supports independent task execution and is suitable for general-purpose computing.
5. Memory contention and limited scalability.
6. It reduces communication steps between nodes due to multiple paths.

4.2.9 Conclusion

Parallel architectures are fundamental to modern computing systems. By effectively distributing tasks and optimizing communication between processors, these systems achieve significant performance improvements. However, challenges such as synchronization, communication overhead, and scalability must be carefully managed.

Summary

- Parallel processing improves performance by executing tasks simultaneously
- Architectures include SIMD, MIMD, shared, and distributed systems
- Interconnection networks enable communication
- Performance depends on speedup, efficiency, and scalability
- Amdahl's Law limits achievable gains

Self-Assessment Questions

1. What is parallel processing?
2. Define speedup.
3. What is scalability?
4. Name two topologies.

5. What is Amdahl's Law?
6. What is efficiency?

Unit Activities

Activity 1: Simulate parallel execution of a task using multiple threads and compare with sequential execution.

Activity 2: Draw and analyze different network topologies (mesh, ring, hypercube).

Activity 3: Compute speedup and efficiency for different processor counts.

Activity 4: Case study analysis of a supercomputer architecture.

Activity 6: Design a simple parallel algorithm for matrix multiplication.

Tutor-Marked Assignments (Tmas)

1. Explain parallel processing concepts with examples.
2. Compare SIMD and MIMD architectures.
3. Discuss interconnection networks and their importance.
4. Evaluate mesh and hypercube topologies.
5. Solve performance problems using Amdahl's Law.
6. Discuss factors affecting scalability in parallel systems.

REFERENCES FOR FURTHER READING

Computer Architecture: A Quantitative Approach – John L. Hennessy & David A. Patterson

Structured Computer Organization – Andrew S. Tanenbaum

Introduction to Parallel Computing – Ananth Grama

IEEE Computer Society

ACM Digital Library

MIT OpenCourseWare

Answers to Self-Assessment Questions

Module One

4.3.1 Unit 1 - Self-Assessment Answers

1. Answer: It is the conceptual design that specifies the functional behavior of a computer system, including instruction sets and data formats.
2. Answer: It is the physical implementation of the system, including hardware components and their interconnections.
3. Answer: Architecture defines functionality (what), while organization defines implementation (how).
4. Answer: Instruction set and addressing modes.
5. Answer: Control unit design and memory hardware.
6. Answer: Because it defines how software interacts with hardware.
7. Answer: Yes, due to differences in organization.
8. Answer: Instruction Set Architecture, a key part of computer architecture.

4.3.2 Unit 2 - Self-Assessment Answers

1. Answer 1: A group of 8 bits.
2. Answer2: 10100_2
3. Answer3: 14_{10}
4. Answer4: 0 to 255
5. Answer5: $+3 \rightarrow 0011$
 Invert $\rightarrow 1100$
 $+1 \rightarrow 1101$
6. Answer6: Compact representation of binary data.

4.3.3 Unit 3 - Self-Assessment Answers

1. Answer1:
 $4 \rightarrow 100$
 $5 \rightarrow 0101$
 BCD = 0100 0101
2. Answer2: $7 + 3 = 10 \rightarrow 1010$
3. Answer3: ASCII = 65 (Decimal) = 01000001 (Binary)
4. Answer4:
 Large range of values

Suitable for scientific computations

5. Answer5: Bias = 3
6. Answer6: It requires more bits than pure binary representation, making it less efficient.

4.3.4 Unit 4 - Self-Assessment Answers

1. Answer1: ISA is the interface between hardware and software that defines the set of instructions a processor can execute, including formats, data types, and addressing modes.
2. Answer2:
 - Opcode (operation code)
 - Operand(s)
 - Addressing mode field
3. Answer3: An opcode specifies the operation to be performed (e.g., ADD, SUB, LOAD).
4. Answer4:
 - Data transfer
 - Arithmetic
 - Logical
 - Control
5. Answer5: The operand value is directly included in the instruction.
6. Answer6:
 - Direct: Address field contains actual memory address
 - Indirect: Address field points to another address where data is stored
7. Answer7: Control (branch) instruction.
8. Answer8: Operands are located in CPU registers specified in the instruction.
9. Answer9: The effective address is obtained by adding a base address to an index register.
10. Answer10: They define how instructions are structured and interpreted by the CPU.
11. Answer11: The control unit interprets the instruction and determines the required operation.
12. Answer12: Fixed formats simplify decoding and improve speed, while variable formats increase flexibility but may slow execution.

13. Answer13: It determines how efficiently operands are accessed, impacting performance and flexibility.

Module Two

4.4.1 Unit 1 - Self-Assessment Answers

1. Answer 1: It holds the address of the memory location to be accessed.
2. Answer 2: IR holds current instruction; PC holds address of next instruction.
3. Answer 3: Because control logic is implemented in software (microcode).
4. Answer 4: Addition and logical AND.
5. Answer 5: Fetch, Decode, Execute, Memory Access, Write Back.

4.4.2 Unit 2 - Self-Assessment Answers

1. Answer 1: Number of cycles per second.
2. Answer 2: Average cycles per instruction.
3. Answer 3: Million Instructions Per Second.
4. Answer 4: Using standard programs to measure performance.
5. Answer 5: Limits of performance improvement.

4.4.3 Unit 3 - Self-Assessment Answers

1. Answer 1: Pipelining is a technique that allows multiple instructions to be processed simultaneously by overlapping different stages of execution.
2. Answer 2:
 - Instruction Fetch (IF)
 - Instruction Decode (ID)
 - Execute (EX)
 - Memory Access (MEM)
 - Write Back (WB)
3. Answer3:

It is the ratio of execution time without pipelining to execution time with pipelining.
4. Answer4: k (number of pipeline stages)
5. Answer5: The time taken for a single instruction to pass through all pipeline stages.

6. Answer 6: A condition that prevents the next instruction from executing in the pipeline.
7. Answer 7:
 - Data hazards
 - Structural hazards
 - Control hazards
8. Answer 8: A technique that passes data directly between pipeline stages to avoid delays.
9. Answer 9: A delay is inserted into the pipeline to resolve hazards.
10. Answer 10: Total cycles = $k + (n - 1)$, where k = stages, n = instructions
11. Answer 11: $4 + (8 - 1) = 11$ cycles
12. Answer 12: The number of instructions completed per unit time.
13. Answer 13:
 - Instruction pipeline processes instructions
 - Arithmetic pipeline performs arithmetic operations in stages
14. Answer 14: Hazards, stalls, and unbalanced stage delays.
15. Answer 15: Speedup = $40 / 10 = 4$

Module Three

4.5.1 Unit 1 - Self-Assessment Answers

1. Answer 1: Computer memory is a component used to store data, instructions, and information temporarily or permanently for processing and retrieval by the CPU.
2. Answer 2:
 - Location (internal or external)
 - Physical type (semiconductor, magnetic, optical)
 - Size (capacity)
 - Performance (access time, cycle time, transfer rate)
3. Answer 3: Primary memory: Fast, directly accessible by CPU, volatile (e.g., RAM).
 Secondary memory: Slower, non-volatile, used for long-term storage (e.g., hard disk).

4. Answer 4: Memory hierarchy balances speed, cost, and capacity by organizing memory into levels, ensuring frequently used data is accessed quickly while maintaining large storage capacity at lower cost.
5. Answer 5: Faster memory is more expensive and has smaller capacity, while slower memory is cheaper and has larger capacity.
6. Answer 6: Cache memory stores frequently used data and instructions close to the CPU, reducing access time and improving overall system performance.

4.5.2 Unit 2 - Self-Assessment Answers

1. Answer 1: Dynamic Random-Access Memory (DRAM) is a type of semiconductor memory that stores each bit of data in a capacitor, which must be periodically refreshed to maintain the stored information.
2. Answer 2: Static Random-Access Memory (SRAM) is a type of semiconductor memory that uses flip-flops to store data and does not require refreshing as long as power is supplied.
3. Answer 3: DRAM requires refreshing; SRAM does not, and DRAM is slower and cheaper; SRAM is faster and more expensive.
4. Answer 4: Because it is cost-effective and has higher storage density, making it suitable for large memory capacity.
5. Answer 5: Because it is faster and has lower access time, which improves system performance.
6. Answer 6: It requires continuous refreshing, which adds complexity and reduces speed.
7. Answer 7: Read-Only Memory (ROM) is a type of non-volatile semiconductor memory used to store permanent data or instructions that are not meant to be modified frequently.
8. Answer 8: Mask Read-Only Memory (MROM) is a type of ROM programmed during the manufacturing process and cannot be modified afterwards.
9. Answer 9: Programmable Read-Only Memory (PROM) is a type of ROM that can be programmed once by the user after manufacturing.
10. Answer 10: Erasable Programmable Read-Only Memory (EPROM) is a ROM that can be erased using ultraviolet (UV) light and reprogrammed.
11. Answer 11: Flash memory is a type of EEPROM that erases and writes data in blocks, making it faster and widely used in storage devices.
12. Answer 12: Because it requires UV light for erasure, making it inconvenient compared to EEPROM and Flash memory.

13. Answer 13: USB drives and Solid-state drives (SSDs)

4.5.3 Unit 3 - Self-Assessment Answers

1. Answer 1: A small, high-speed memory that stores frequently accessed data to reduce access time.
2. Answer 2: To bridge the speed gap between CPU and main memory.
3. Answer 3:
 - Cache lines (blocks)
 - Tag
 - Index
 - Offset
4. Answer 4: When requested data is found in cache.
5. Answer 5: When requested data is not found in cache.
6. Answer 6: The fraction of memory accesses found in cache.
7. Answer 7: A field used to identify if a memory block is present in a cache line.
8. Answer 8: The number of locations in a set where a block can be placed.
9. Answer 9: A method used to decide which block to replace when the cache is full.
10. Answer 10:
 - Least Recently Used (LRU)
 - First-In-First-Out (FIFO)
 - Random
11. Answer 11: Data is written to both cache and main memory simultaneously.
12. Answer 12: Data is written to the cache first and updated in memory later.
13. Answer 13: $AMAT = 1 + (0.05 \times 100) = 6 \text{ ns}$
14. Answer 14: Direct mapping is simpler but suffers from conflicts; set-associative reduces conflicts but increases complexity.
15. Answer 15: It reduces the miss rate by storing more data.
16. Answer 16: Larger blocks reduce compulsory misses but may increase conflict and capacity misses.

Module Four

4.6.1 Unit 1 - Self-Assessment Answers

1. Answer 1: RISC (Reduced Instruction Set Computer) is a processor design that uses a small, highly optimised set of instructions that execute rapidly.
2. Answer 2: CISC (Complex Instruction Set Computer) is a processor design that uses a large set of complex instructions capable of performing multiple operations.
3. Answer3:
 - Faster instruction execution
 - Easier pipelining
4. Answer 4:
 - Reduced instruction count
 - More powerful individual instructions
5. Answer 5: RISC uses more instructions (higher memory usage), while CISC uses fewer instructions (lower memory usage).
6. Answer 6: Because it is power-efficient and provides high performance per watt.

4.6.2 Unit 2 - Self-Assessment Answers

1. Answer 1: Execution of multiple tasks simultaneously using multiple processors.
2. Answer 2: Ratio of sequential execution time to parallel execution time.
3. Answer 3: Ability to maintain performance with increased resources.
4. Answer 4: Mesh and Ring.
5. Answer 5: A formula that limits achievable speedup based on the sequential portion.
6. Answer 6: Speedup divided by the number of processors.

GLOSSARY

Absolute address: a variable's or routine's actual address in memory.

Abstraction a model that renders lower-level details of computer systems temporarily invisible to facilitate the design of sophisticated systems.

Address a value used to delineate the location of a specific data element within a memory array.

Addressing mode one of the several addressing regimes delimited by their varied use of operands and/or addresses.

Amdahl's law a rule stating that the performance enhancement possible with a given improvement is limited by the amount that the improved feature is used. It is a quantitative version of the law of diminishing returns.

AND a logical bit-by-bit operation with two operands that calculates a 1 only if there is a 1 in both operands.

Application Binary Interface (ABI) the user portion of the instruction set plus the operating system interfaces used by application programmers. It defines a standard for binary portability across computers.

Architectural registers are the instruction set of visible registers of a processor; for example, in RISC-V, these are the 32 integer and 32 floating-point registers.

An arithmetic logic unit (ALU) hardware that performs addition, subtraction, and usually logical operations such as and and or.

Assemble a program that translates a symbolic version of an instruction into the binary version.

An assembler directive is an operation that tells the assembler how to translate a program,

but does not produce machine instructions; it always begins with a period.

Assembly language a symbolic language that can be translated into binary machine language.

A basic block is a sequence of instructions without branches (except possibly at the end) and without branch targets or branch labels (except possibly at the beginning).

A binary digit is also called a binary bit. One of the two numbers in base 2, 0 or 1, that are the components of information.

Block (or line) the minimum unit of information that can be either present or not present in a cache.

Bus in logic design, a collection of data lines that are treated together as a single logical signal; also, a shared collection of lines with multiple sources and uses.

Cache memory a small, fast memory that acts as a buffer for a slower, larger memory.

Cache miss a request for data from the cache that cannot be filled because the data are not present in the cache.

Central Processing Unit (CPU) also called central processor unit or processor. The active part of the computer, which contains the datapath and control and which adds numbers, tests numbers, signals i/o devices to activate, and so on.

Clock cycle also called tick, clock tick, clock period, clock, or

Cycle. The time for one clock period, usually of the processor clock.

Clock Cycles Per Instruction (CPI) average number of clock cycles per instruction for a program or program fragment.

Clock period the length of each clock cycle.

Cloud computing refers to large collections of servers that provide services over the internet; some providers rent dynamically varying numbers of servers as a utility.

Cluster a set of computers connected over a local area network that function as a single large multiprocessor.

Clusters collections of computers connected via i/o over standard network switches to form a message-passing multiprocessor.

Coarse-Grained Multithreading a version of hardware multithreading that implies switching between threads only after significant events, such as a last-level cache miss.

Combinational element an operational element, such as an and gate or an alu

Combinational logic a logic system whose blocks do not contain memory and hence compute the same output given the same input.

Compiler a program that translates high-level language statements into assembly language statements.

Conditional branch an instruction that tests a value, and that allows for a subsequent transfer of control to a new address in the program based on the outcome of the test

Context switch a changing of the internal state of the processor to allow a different process to use the processor that includes saving the state needed to return to the currently executing process.

Control the component of the processor that commands the datapath, memory, and i/o devices according to the instructions of the program.

Control hazard also called branch hazard. Arises when the proper instruction cannot execute in the proper pipeline clock cycle because the instruction that was fetched is not

the one that is needed; that is, the flow of instruction addresses is not what the pipeline expected.

Control signal a signal used for multiplexor selection or for directing the operation of a functional unit; contrasts with a

Data signal, which contains information that is operated on by a functional unit.

CPI cycle per instruction

CPU Execution Time also called cpu time. The actual time the cpu spends computing for a specific task.

Data hazard also called a pipeline data hazard. When a planned instruction cannot execute in the proper clock cycle because data that are needed to execute the instruction are not yet available.

Data transfer instruction a command that moves data between memory and registers.

Data-level parallelism parallelism achieved by performing the same operation on independent data.

Datapath the component of the processor that performs arithmetic operations.

Datapath element a unit used to operate on or hold data within a processor. In the risc-v implementation, the datapath elements include the instruction and data memories, the register file, the alu, and adders.

Decoder a logic block that has an n-bit input and 2^n outputs, where only one output is asserted for each input combination.

Direct-Mapped Cache a cache structure in which each memory location is mapped to exactly one location in the cache.

Dynamic Random Access Memory (DRAM) memory built as an integrated circuit; it provides random access to any location. Access times are 50 nanoseconds

Dynamically Linked Libraries (DLLS) library routines that are linked to a program during execution.

Embedded computer a computer inside another device used for running one predetermined application or collection of software.

Error Detection Code a code that enables the detection of an error in data, but not the precise location and, hence, correction of the error.

Exception also called an interrupt. An unscheduled event that disrupts program execution; used to detect overflow.

Exponent in the numerical representation system of floating-point arithmetic, the value that is placed in the exponent field.

External label also called global label. A label referring to an object that can be referenced from files other than the one in which it is defined.

Fine-grained multithreading a version of hardware multithreading that implies switching between threads after every instruction.

Finite-state machine a sequential logic function consisting of a set of inputs and outputs, a next-state function that maps the current state and the inputs to a new state, and an output function that maps the current state and possibly the inputs to a set of asserted outputs.

Flash memory a non-volatile semiconductor memory. It is cheaper and slower than dram but more expensive per bit and faster than magnetic disks. Access times are about 5 to 50 microseconds and cost per gigabyte in 2012 was \$0.75 to \$1.00.

Flip-flop a memory element for which the output is equal to the value of the stored state inside the element and for which the internal state is changed only on a clock edge.

Floating point computer arithmetic that represents numbers in which the binary point is not fixed.

Fully associative cache a cache structure in which a block can be placed in any location in the cache.

Gate A device that implements basic logic functions, such as AND or OR.

Hardware description language A programming language for describing hardware, used for generating simulations of a hardware design and also as input to synthesis tools that can generate actual hardware.

Hardware multithreading Increasing utilization of a processor by switching to another thread when one thread is stalled.

Hexadecimal Numbers in base 16.

High-level programming language A portable language such as C, C++, Java, or Visual Basic that is composed of words and algebraic notation that can be translated by a compiler into assembly language.

Hit rate The fraction of memory accesses found in a level of the memory hierarchy.

Hit time The time required to access a level of the memory hierarchy, including the time needed to determine whether the access is a hit or a miss.

Implementation Hardware that obeys the architecture abstraction.

Input device A mechanism through which the computer is fed information, such as a microphone.

Instruction A command that computer hardware understands and obeys.

Instruction count The number of instructions executed by the program.

Instruction format A form of representation of an instruction composed of fields of binary numbers.

Instruction-level parallelism The parallelism among instructions.

Instruction mix A measure of the dynamic frequency of instructions across one or many programs.

Instruction set architecture Also called architecture. An abstract interface between the hardware and the lowest-level software that encompasses all the information necessary to write a machine language program that will run correctly, including instructions, registers, memory access, I/O, and so on.

Integrated circuit Also called a chip. A device combining dozens to millions of transistors.

Interrupt An exception that comes from outside of the processor. (Some architectures use the term interrupt for all exceptions.)

Interrupt handler A piece of code that is run as a result of an exception or an interrupt.

Latency (pipeline) The number of stages in a pipeline or the number of stages between two instructions during execution.

Least recently used (LRU) A replacement scheme in which the block replaced is the one that has been unused for the longest time.

Least significant bit The rightmost bit in a RISC-V doubleword.

Linker Also called link editor. A systems program that combines independently assembled machine language programs and resolves all undefined labels into an executable file.

Liquid crystal display A display technology using a thin layer of liquid polymers that can be used to transmit or block light according to whether a charge is applied.

Load-use data hazard A specific form of data hazard in which the data being loaded by a load

instruction have not yet become available when they are needed by another instruction.

Loader A systems program that places an object program in main memory so that it is ready to execute.

Local miss rate The fraction of references to one level of a cache that miss; used in multi-level hierarchies.

Lock A synchronization device that allows access to data to only one processor at a time.

Machine language Binary representation used for communication within a computer system.

Magnetic disk Also called hard disk. A form of non-volatile secondary memory composed of rotating platters coated with a magnetic recording material. Because they are rotating mechanical devices, access times are about 5 to 20 milliseconds, and the cost per gigabyte in 2012 was \$0.05 to \$0.10.

Main memory Also called primary memory. Memory used to hold programs while they are running; typically consists of DRAM in today's computers.

Memory The storage area in which programs are kept when they are running, and that contains the data needed by the running programs.

Memory hierarchy A structure that uses multiple levels of memories; as the distance from the processor increases, the size of the memories and the access time both increase.

Microarchitecture The organization of the processor, including the major functional units, their interconnection, and control.

Million instructions per second (MIPS) A measurement of program execution speed based on the number of millions of instructions. MIPS is computed as the instruction count divided by the product of the execution time and 10⁶.

Minterms Also called product terms. A set of logic inputs joined by conjunction (AND operations); the product terms form the first logic stage of the programmable logic array (PLA).

Miss Penalty The time required to fetch a block into a level of the memory hierarchy from the lower level, including the time to access the block, transmit it from one level to the other, insert it in the level that experienced the miss, and then pass the block to the requestor.

Miss Rate The fraction of memory accesses not found in a level of the memory hierarchy.

Most Significant Bit The leftmost bit in a RISC-V doubleword.

Multicore Microprocessor A microprocessor containing multiple processors (“cores”) in a single integrated circuit. Virtually all microprocessors today in desktops and servers are multicore.

Multilevel Cache A memory hierarchy with multiple levels of caches, rather than just a cache and main memory.

Multiprocessor: A computer system with at least two processors. This computer is in contrast to a uniprocessor, which has one, and is increasingly hard to find today.

NAND gate An inverted AND gate.

Nonvolatile Memory: A form of memory that retains data even in the absence of a power source and that is used to store programs between runs. A DVD disk is nonvolatile.

NOP An instruction that does no operation to change state.

NOR A logical bit-by-bit operation with two operands that calculates the NOT of the OR of the two operands. That is, it calculates a 1 only if there is a 0 in both operands.

NOR gate An inverted OR gate.

Normalized A number in floating-point notation that has no leading 0s.

NOT A logical bit-by-bit operation with one operand that inverts the bits; that is, it replaces every 1 with a 0, and every 0 with a 1.

One’s Complement A notation that represents the most negative value by $10 \dots 000_2$ and the most positive value by $01 \dots 11_2$, leaving an equal number of negatives and positives but ending up with two zeros, one positive ($00 \dots 00_2$) and one negative ($11 \dots 11_2$). The term is also used to mean the inversion of every bit in a pattern: 0 to 1 and 1 to 0.

Opcode The field that denotes the operation and format of an instruction.

OpenMP An API for shared memory multiprocessing in C, C++, or Fortran that runs on UNIX and Microsoft platforms. It includes compiler directives, a library, and runtime directives.

OR A logical bit-by-bit operation with two operands that calculates a 1 if there is a 1 in either operand.

Output Device A mechanism that conveys the result of a computation, such as a display, to a user or to another computer.

Parallel Processing Program A single program that runs on multiple processors simultaneously.

PC-Relative Addressing An addressing regime in which the address is the sum of the program counter (PC) and a constant in the instruction.

Personal Computer (PC) A computer designed for use by an individual, usually incorporating a graphics display, a keyboard, and a mouse.

Physical Address An address in main memory.

Physically Addressed Cache A cache that is addressed by a physical address.

Pipeline Stall Also called bubble. A stall initiated in order to resolve a hazard.

Pipelining An implementation technique in which multiple instructions are overlapped in execution, much like an assembly line.

Pixel The smallest individual picture element. Screens are composed of hundreds of thousands to millions of pixels, organized in a matrix.

Prefetching A technique in which data blocks needed in the future are brought into the cache early by the use of special instructions that specify the address of the block.

Procedure A stored subroutine that performs a specific task based on the parameters with which it is provided.

Process Includes one or more threads, the address space, and the operating system state. Hence, a process switch usually invokes the operating system, but not a thread switch.

Program counter (PC) The register containing the address of the instruction in the program being executed.

Programmable ROM (PROM) A form of read-only memory that can be programmed when a designer knows its contents.

Protection A set of mechanisms for ensuring that multiple processes sharing the processor, memory, or I/O devices cannot interfere, intentionally or unintentionally, with one another by reading or writing each other's data. These mechanisms also isolate the operating system from a user process.

Push Add element to stack.

read-only memory (ROM) A memory whose contents are designated at creation time, after which the contents can only be read. ROM is used as structured logic to implement a set of logic functions by using the terms in the logic functions as address inputs and the outputs as bits in each word of the memory.

Response time Also called execution time. The total time required for the computer to complete a task, including disk accesses, memory accesses, I/O activities, operating system overhead, CPU execution time, and so on.

secondary memory Non-volatile memory used to store programs and data between runs; typically consists of flash memory in PMDs and magnetic disks in servers.

semiconductor A substance that does not conduct electricity well.

sequential logic A group of logic elements that contain memory and hence whose value depends on the inputs as well as the current contents of the memory.

server A computer used for running larger programs for multiple users, often simultaneously, and typically accessed only via a network.

set-associative cache A cache that has a fixed number of locations (at least two) where each block can be placed.

shared memory multiprocessor (SMP) A parallel processor with a single physical address space.

silicon A natural element that is a semiconductor.

SIMD or Single Instruction stream, Multiple Data streams. The same instruction is applied to many data streams, as in a vector processor.

spatial locality the locality principle stating that if a data location is referenced, data locations with nearby addresses will tend to be referenced soon.

static random access memory (SRAM) A memory where data are stored statically (as in flip-flops) rather than dynamically (as in DRAM). SRAMs are faster than DRAMs, but less dense and more expensive per bit.

stored-program concept The idea that instructions and data of many types can be stored in memory as numbers and thus be easy to change, leading to the stored program computer.

structural hazard When a planned instruction cannot execute in the proper clock cycle

because the hardware does not support the combination of instructions that are set to execute.

system CPU time The CPU time spent in the operating system performing tasks on behalf of the program.

systems software - software that provides services that are commonly useful, including operating systems, compilers, loaders, and assemblers.

tag a field in a table used for a memory hierarchy that contains the address information required to identify whether the associated block in the hierarchy corresponds to a requested word.

task-level parallelism or process-level parallelism Utilizing multiple processors by running independent programs simultaneously.

temporal locality The principle stating that if a data location is referenced, then it will tend to be referenced again soon.

three Cs model A cache model in which all cache misses are classified into one of three categories: compulsory misses, capacity misses, and conflict misses.

throughput Also called bandwidth. Another measure of performance, it is the number of tasks completed per unit time.

truth table From logic, a representation of a logical operation by listing all the values of the

inputs and then in each case showing what the resulting outputs should be.

underflow (floating-point) A situation in which a negative exponent becomes too large to fit in the exponent field.

very-large-scale integrated (VLSI) circuit A device containing hundreds of thousands to millions of transistors.

very long instruction word (VLIW) A style of instruction set architecture that launches many operations that are defined to be independent in a single wide instruction, typically with many separate opcode fields.

virtual address An address that corresponds to a location in virtual space and is translated by address mapping to a physical address when memory is accessed

virtual machine A virtual computer that appears to have nondelayed branches and loads and a richer instruction set than the actual hardware.

virtual memory A technique that uses main memory as a “cache” for secondary storage.

virtually addressed cache A cache that is accessed with a virtual address rather than a physical address.

volatile memory Storage, such as DRAM, that retains data only if it is receiving power.